ENGINE DEVELOPMENT PLAN

Rocks · 3D 게임 엔진 전체 구현 계획

STL 비사용 Core · Object와 점진 GC · 웹 에디터 · GPU-driven 렌더러

설계 기준일 2026.11.10 · 개정 1.3 · 상태: 구현 제안 / 성능 미측정

추가 요구 반영: Deferred·Forward 정식 지원, 업스케일링, 태깅·컴포넌트·프리팹, Unreal 참고 체계 확대, 컷씬 제작 핵심화, Blueprint 제외·자체 언어 채택, 하이엔드 포스트이펙트 전체 기능군, UBT 계열 자체 빌드 도구·분산 빌드·반복 개발 최적화, 정식 헤드리스 모드, 필수 노드 기반 머티리얼 에디터.

1.2 교차 검증 결과: Core/Object, 렌더링·콘텐츠, 빌드·헤드리스 담당 서브에이전트 3개가 개정 1.1을 독립 검토하였다. 중복을 통합한 16건의 계약 누락·모순과 별도 보완 3건을 본문·완료 기준에 반영하였다. 수명 수정안은 담당 외 검토자가 다시 반례를 대조하였다. 첫 작업 Core/Object, Blueprint 제외·커스텀 언어·STL 비사용 Runtime 정책을 유지한다. 33.4절에 검토 근거·조치·남은 실행 검증을 기록한다. 본 문서는 구현 제안이며 엔진 실행·성능 검증 결과가 아니다.

1.3 추가 요구: Rocks 자체 명명, Windows/macOS 개발 환경, 5종 RHI, 머티리얼 에디터·템플릿·PIE, 가변/고정 tick·time dilation, texture streaming·내장 리소스, lightmap/probe, 필수 로그·프로파일러·장애 하네스를 반영한다. 수학은 검증된 구현의 어댑터와 SIMD 활용으로 구성하고 자체 알고리즘 재작성은 제외한다. 안정성·정확성, 확장성, 성능 순으로 판단하며 Core/Object가 첫 작업이다. 상세 모듈·API·작업은 별첨 1.1에 연결한다.

목차

1 목표와 실행 전략 1
2 기술 정책과 기준 플랫폼 4
3 모듈 구조와 의존성 9
4 Core 컨테이너와 메모리 14
5 Object와 청크 레지스트리 16
6 Nonmoving 병렬·점진 GC 19
7 Clang 메타 컴파일러와 직렬화 21
8 Task Graph와 스레드 실행 규약 23
9 에셋 파이프라인·쿠킹·DDC·비동기 로딩 26
10 Scene Graph·World·World Partition 30
11 렌더러 기반과 Deferred·Forward 정식 지원 32
12 업스케일링·AA·동적 해상도 37
13 GPU Scene·Static Mesh·VirtualGeometry 39
14 조명·DynamicGI·재질·후처리 39
15 Skeletal Mesh와 Animation 47
16 충돌·물리·캐릭터 이동 48
17 네트워크·Replication·Prediction 49
18 오디오·DSP Graph 51
19 Navigation과 AI 52
20 오브젝트 태깅·컴포넌트·프리팹·게임플레이 53
21 VFX·게임 UI·환경 시스템 55
22 웹 에디터와 네이티브 뷰포트 57
23 자체 게임플레이 언어·컴파일러·VM 61
24 Unreal 참고 체계의 추가 도입 63
25 컷씬 제작기·런타임 시퀀스·영상 출력 65
26 하이엔드 포스트이펙트 전체 기능군 68
27 자체 빌드 도구·분산 빌드·반복 개발 최적화 71
28 정식 헤드리스 실행·서버·자동화 77
29 통합 검증·프로파일링·성능 예산 79
30 단계별 마일스톤과 인력 84
31 위험·우선순위·대응 86
32 착수 4주 실행 항목과 요구 추적 88
33 전체 검토 결과·Unreal 비교·잔여 항목 89
34 참고 자료와 설계 근거 98

1 목표와 실행 전략

최우선 첫 작업은 Core 공통 라이브러리와 오브젝트 시스템이다. 기반의 정확성·수명·검증 체계를 먼저 완성한 뒤 에셋 임포트·웹 편집·쿠킹·독립 실행 게임의 첫 제품 목표로 확장한다. UE5급 기능 범위는 장기 목표로 유지하되, 가상화 지오메트리와 동적 전역 조명은 기본 제작 과정이 검증된 이후 독립적인 연구 단계로 확장한다.

참조 대화 「3D 엔진 설계」에서 조회 가능한 5개 사용자 요청과 응답을 반영하였다. 최근의 'STL 미사용' 요구를 이전의 'STL/PMR 사용' 제안보다 우선한다. C++23·자체 Core·청크 레지스트리·Nonmoving 병렬·점진 GC·Clang 메타와 웹 UI/네이티브 뷰포트 방향을 계승한다. 추가 요청에 따라 자체 Object 모델과 D3D11/D3D12/Vulkan/Metal/OpenGL 지원, Windows/macOS 개발 환경으로 범위를 갱신한다. Jolt는 어댑터 뒤의 물리 후보로 유지한다. 아래의 구체적인 API, 수치, 인력, 일정, 셀 크기와 제한값은 실행 가능한 계획을 위한 **추가 설계 제안**이며 기존 합의나 실측 결과가 아니다.

구분	제품 목표	완료 판정
P0 · 기반	메모리·객체·빌드·저장 규약의 정확성	GC 오수거 0건, 재현 가능한 에셋 빌드, 헤드리스 테스트 통과
P1 · 실용 엔진	기본 PBR, 편집·저장·쿠킹, 캐릭터·충돌·오디오·AI·멀티플레이	신규 작업 환경에서 샘플 게임 제작과 패키지 실행 재현
P2 · 고급 제작	GPU-driven, 월드 스트리밍, 전문 편집기, 대규모 콘텐츠	장시간 이동·편집·네트워크 부하에서 품질·예산 충족
P3 · 연구	Nanite/Lumen 유사 기능, 가상 그림자·텍스처, 파괴·고급 캐릭터	기준 경로 대비 화질·성능·제작 비용의 개선 입증

표 1

'UE5급'은 API 호환이나 기능·성능 동등성을 보장하는 표현으로 사용하지 않는다. 각 기능은 지원 대상, 콘텐츠 제한, 대체 경로, 수치 기준을 별도로 정의한다. Nanite와 Lumen이라는 명칭은 공개 기술을 설명하기 위한 비교 기준이며, 자체 구현의 내부 명칭은 VirtualGeometry와 DynamicGI로 통일한다.

구현 순서: Core 최소 기능 → 최소 메타 스키마와 객체 → 정확한 정지형 GC → 비동기 패키지 → 기본 렌더러와 편집기 → 실제 게임 → 병렬·점진화 → 고급 렌더링. 직렬 기준 구현을 유지하여 최적화 결과와 항상 대조한다.

1.1 첫 작업: Core 기반 라이브러리와 오브젝트 시스템

첫 번째 구현 작업은 Core다. Core는 공통 library와 ObjectRuntime을 포함하는 기반 시스템 범위로 정의하며 최우선 인력·검증 예산을 배정한다. 렌더러·에디터·자체 언어·전문 빌드 최적화보다 먼저 이 계약을 완성한다. ObjectRuntime은 별도 모듈이지만 작업 우선순위상 Core Foundation에 포함한다.

첫 작업의 최소 build/test bootstrap, CPU-only test runner, trace·compiler timing은 Core 검증에 필요한 도구로 함께 준비한다. 이 단계에서 웹 viewport·GPU renderer·분산 빌드 engine·게임플레이 VM을 구현의 임계 경로로 편입하지 않는다. 최소 TypeInfo/visitor는 수동 등록으로 객체 테스트를 먼저 시작하고, 안정된 계약에 Clang 생성기를 연결한다.

Foundation 책임 묶음	최종 구현 결과	검증 책임
F0 · 실행 기반	PlatformBase·고정 toolchain·assert/log·CPU-only runner	GPU/창/에디터 없이 생성·실행·오류 재현
F1 · 메모리	기준 heap/page·tag·실패·정렬·track, slab/thread cache는 선택 최적화	모든 기반 자료구조의 allocator/수명 계약
F2 · 공통 library	Array/Span/String/Name/HashMap·handle store·math·path/time/archive	iterator·소멸·복사/이동 ·overflow·Transform·serialization 정확성
F3 · 실행 시스템	Atomic/Thread/Mutex/Event·queue·Task DAG·기초 async IO·Trace	의존성·취소·drain·read/write phase 검증
F4 · 객체 기반	TypeInfo 최소형·Object·청크 Registry·generation·Root/Object/Weak 참조 ·ClassDefaults	공개 전 초기화·instancing·논리 파괴·회수 규약

Foundation 책임 묶음	최종 구현 결과	검증 책임
F5 · GC·데이터	Nonmoving full collector·generated/manual visitor 대조·Archive·schema fixture	오수거·dangling·영속 ID 손상 0건
F6 · Foundation Gate	병렬 기반 stress·Object/GC 수명 시험·문서화된 API·재현 trace	이후 시스템의 본격 구현 착수

표 2

F0–F6는 Foundation 책임 묶음이며 번호순으로 전체 기능을 완료하라는 선행 관계가 아니다. 실제 착수는 상세안 R0의 기준 heap·최소 Array·inline executor → R1 객체/정지형 GC → R2 전체 library·worker/IO·Archive 순서다. C07–C12에 C13–C20 또는 C28을 선행시키지 않으며 slab/thread cache는 C28 선택 최적화다.

병렬·점진 GC의 공개 목표는 유지하되 첫 collector는 정확한 정지형 기준 구현이다. 초기부터 barrier-aware API를 설계하여 나중에 raw pointer 필드를 일괄 교체하지 않게 한다. 병렬 marking/점진 budget 최적화는 full collector와 결과를 대조한 뒤 확대한다.

1.2 Foundation 완료 기준

Gate	필수 증거
CF01 · 독립성	Core/Object target이 World·Asset loader·Renderer·WebView·Script VM 없이 빌드·CPU-only 실행
CF02 · 정책	공개 헤더 독립 compile·금지 std 의존 0건, 예외/RTTI/allocator 경계 검사
CF03 · 메모리	alignment·overflow·failure injection·교차 스레드 free·leak/double-free fixture 통과
CF04 · 컨테이너	nontrivial 타입·iterator·relocation·hash collision·Transform shear/특이 행렬 fixture
CF05 · Task	prerequisite·cancel·terminal exactly-once·shutdown teardown·queue node 수명 검증
CF06 · Registry	generation·publication·ConstructionContext·실패 rollback·명시 destroy·오래된 handle 거부
CF07 · 참조	Root/Object/Weak·Borrow 경계·task root·destroy 보존·cutoff acquisition·World context 규약
CF08 · GC	무작위 graph의 reachable 누락 0건, barrier·cutoff·epoch 재사용·정리 재진입 fixture
CF09 · 기본값/복제	default subobject 인스턴스별 독립·내부 참조 remap·stable ID·재구성 검증
CF10 · 메타/Archive	최소 3개 schema migration·unknown field·길이/재귀 제한·단순 binary round-trip
CF11 · 진단	CPU/task/GC/메모리 trace·root chain·hang/crash 보고·재현 seed
CF12 · 통합	Windows·macOS의 지원 sanitizer·headless 반복·종료 후 메모리 안정 구간 복귀

표 3

CF01–CF12의 ID와 의미는 이 표를 정본으로 사용한다. 상세안 16.1절은 같은 ID에 CT 시험을 연결한다. 첫 Foundation의 필수 실행 플랫폼은 Windows/macOS다. Linux는 제품의 필수 후속 target으로 M3에서 Core CPU/headless suite를 다시 수행하고, M5 Linux 서버 통합 전에 backend·IO·thread·GC·shutdown 검증을 완료한다.

위 기준은 계획상의 gate이며 현재 구현 완료를 주장하지 않는다. API와 정확성 gate를 통과하기 전 새 렌더링 기능을 추가하여 Foundation 검증을 대체하지 않는다. GC의 0.5/1.0 ms 목표 등 성능 미달은 별도 측정 결과로 관리하고 정확성 통과와 구분한다.

2 기술 정책과 기준 플랫폼

2.1 STL 금지 범위

영역	적용 정책	확인 방법
엔진 소유 Runtime·공개 C++ API	std 컨테이너, 문자열, 알고리즘, 스마트 포인터, function, thread, mutex, pmr, ranges 미사용	금지 헤더·AST 검사와 공개 헤더 독립 컴파일
언어 지원	C++23 언어, concepts, constexpr, move, placement new, 기본 CRT 허용	툴체인·컴파일 옵션 고정
최소 표준 지원 헤더	cstddef, cstdint, new 허용. atomic은 Platform의 Atomic 구현 내부에만 허용하는 기본안	화이트리스트와 경계 검사
자체 Core	Array, HashMap, String, Function, UniquePtr, SharedPtr, Atomic, 동기화, 검증된 외부 수학 구현의 어댑터	할당기·컨테이너 규약 테스트
Host 도구	LLVM/Clang, DXC, 임포터 및 빌드 도구 내부 STL 허용. 게임 배포물과 분리	Host/Target 빌드 그래프 분리
ThirdParty 런타임	내부 STL은 허용하되 엔진 어댑터 밖으로 타입·예외·소유권을 노출하지 않음	버전별 헤더·심볼·할당 검토

표 4

이 정책은 '엔진 작성 코드와 API의 STL 비사용'을 의미한다. ThirdParty 내부 사용까지 포함한 '배포 바이너리 전체 STL 0'은 별도 요구 수준이다. 그 수준을 적용하면 Jolt 등의 의존성을 다시 검토하고, 교체·포크 비용을 일정에 반영해야 한다. C ABI로 감싸는 것만으로 내부 STL 의존성이 제거되지는 않는다. 플랫폼 Atomics는 검증된 표준 메모리 모델을 우선하며, intrinsic 대체는 별도 검증 후 적용한다.

엔진 Runtime은 예외와 C++ RTTI를 사용하지 않는 기본안으로 구성한다. Object Reflection은 자체 TypeInfo를 사용한다. ThirdParty의 예외·RTTI 옵션은 별도 빌드하며 예외가 엔진 경계를 통과하지 않게 변환한다. 일반 실패는 Result/Error, 손상된 불변식은 assert/fatal, 회복 불가능한 전체 메모리 부족은 진단 후 종료로 처리한다. 스트리밍처럼 예산 부족이 정상인 경로에는 TryAllocate를 제공한다.

2.2 좌표·시간·정밀도 규약

항목	기본안
플랫폼	Windows x64·macOS ARM64 개발/에디터·클라이언트, Linux x64 서버·기반 테스트. 추가 아키텍처는 개별 지원 manifest로 관리
도구	자체 BuildDriver(Rocks Build)이 target/module/action graph를 소유한다. CMake는 bootstrap·외부 도구, Ninja는 선택 local executor다. clang-cl 우선·MSVC 호환, SDK 버전 고정

항목	기본안
그래픽	D3D11/D3D12/Vulkan/Metal/OpenGL. 공통 material IR과 backend별 shader toolchain; 고급 capability는 독립 profile
좌표	오른손, +Z Up, +X Forward, 미터·라디안·초. 임포터에서 변환
행렬	column vector, p'=M×p, local-to-world=parent×local. HLSL 저장 규약과 전치 지점을 고정
깊이	reversed-Z, depth [0,1]. 투영·컬링·피킹·속도 버퍼의 동일 규약 검증
큰 월드	월드 위치 double, 회전·스케일 float, 렌더링은 카메라 상대 float. 물리는 지역 원점/구역 변환
시간	시뮬레이션 기본 60Hz, 단조 증가 64-bit Tick. 렌더 보간과 오디오 sample clock 분리
직렬화	명시적인 byte order, 고정 폭 정수, 길이 상한·체크섬. 구조체 메모리를 그대로 저장하지 않음
문자열	UTF-8 String/StringView, UI 입력·검색의 Unicode 정규화 정책 명시

표 5

CPU/GPU 구조체는 크기·정렬·필드 오프셋을 자동 검사한다. SIMD 정렬을 파일 포맷이나 플러그인 ABI에 암묵적으로 전파하지 않는다. 플랫폼 기능은 DeviceCaps로 조회하며 고급 기능이 없는 장치에는 전통 LOD·일반 그림자·probe 조명 경로를 제공한다.

2.3 Core Transform의 표현과 실패 규약

Core의 authoring LocalTransform은 translation(double)·quaternion·diagonal scale의 TRS로 정의하고, 계산용 WorldAffine은 parent×local의 affine 결과를 보존한다. 비균일 parent scale과 child rotation의 합성에는 shear가 생길 수 있으므로 world 결과를 매번 TRS로 분해하여 근사하지 않는다. 카메라 상대 GPU matrix는 이 affine 결과에서 생성한다.

KeepWorld reparent는 inverse(parent)×world를 계산한 뒤 local TRS 표현 가능성과 허용 오차를 검증한다. shear가 필요한 경우 NonRepresentableTransform으로 실패하여 원래 계층을 보존한다. 명시적인 geometry bake는 별도 편집 명령과 Undo를 제공하며 묵시 근사를 금지한다. singular/zero scale은 역행렬이 필요한 작업에서 실패하고, 음수 determinant의 winding·normal inverse-transpose·bounds를 일관되게 처리한다. physics는 허용 scale과 shape recook 정책을 별도 선언한다.

CF04에 uniform/nonuniform/negative/zero scale × parent/child rotation × reparent/Undo/prefab bake를 포함한다. 허용 경로는 world vertex·normal·bounds를 비교하고 불허 경로는 변경 없는 실패를 확인한다. 이 계약은 Scene 구현 이후가 아니라 첫 Core Math 단계에서 고정한다.

2.4 Rocks 명명 체계와 언어 경계 비교

엔진 이름은 **Rocks**로 확정한다. 자체 타입·모듈·도구의 식별자에 Unreal의 U/F/A 계열 분류 접두사를 채택하지 않는다. C++ 내부 기본 후보는 namespace rocks와 의미가 드러나는 PascalCase 타입이며, `rocks::Object`, `rocks::Entity`, `rocks::Component`, `rocks::TransformComponent`, `rocks::World` 로 표기한다. 예제는 namespace rocks 내부 표기를 생략할 수 있다. 인터페이스도 Allocator/InputBackend/RenderDevice처럼 역할로 명명하고 분류용 I/T/E/R 문자를 일괄 추가하지 않는다.

prefix를 모든 경계에서 금지하거나 강제하는 결론은 보류한다. 다른 언어의 namespace/module 표현, binding generator·debugger·symbol lookup·ABI 충돌을 검증하여 경계별로 결정한다. 아래는 자체 설계 대안의 비교이며 prefix만으로 ABI나 메모리 안전이 확보되는 것은 아니다.

대안	이점	비용·한계	우선 적용 판단
C++ namespace 중심	짧은 Object/Entity 이름·모듈별 역할·IDE 탐색, 내부 코드 가독성	namespace 없는 바인딩에는 이름 매핑 필요; C++ ABI를 외부에 그대로 노출하면 toolchain 결합	내부 기본 후보
모든 타입에 Rocks 접두사	평면 목록·검색에서 출처 식별, 일부 단순 바인딩 도구 편의	장황한 이름·리팩터링·namespace와 중복, C ABI 변환과 수명 규약은 여전히 필요	일괄 도입하지 않음
외부 ABI에만 rocks_	C/export symbol 충돌 회피·안정된 외부 이름, 내부 표기 유지	내부/외부 이름 매핑·진단 source map·wrapper 생성 필요	현재 권고안, 실제 glue 검증 후 확정
소수 export+versioned 함수 테이블	공개 symbol 수 제한·plugin capability 협상·ABI 확장	간접 호출·테이블 크기/버전 검사·debug 탐색 비용	독립 배포 plugin에 비교 실험

표 6

C language linkage의 동일 이름은 C++ namespace가 달라도 같은 entity를 가리킬 수 있으므로, namespace만으로 C export 충돌을 방지한다고 가정하지 않는다. 정확한 symbol decoration·calling convention은 target toolchain에서 확인한다. S68

경계 후보 예시는 C++의 `rocks::Object`에 대응하는 C ABI의 `rocks_object_handle`·`rocks_object_create`, namespace를 표현할 수 있는 언어의 `rocks.Object`, 평면 타입 이름이 필요한 바인딩의 `RocksObject`다. 마지막 형태는 해당 바인딩의 편의 선택이며 내부 타입까지 변경하는 근거가 아니다. macro·export와 같이 namespace로 구분할 수 없는 전역 식별자는 ROCKS_/rocks_ 프로젝트 표식을 사용할 수 있다. 중첩 namespace를 모든 언어에 그대로 복제할 필요는 없으며 타입·함수 ID로 원래 선언과 연결한다.

C ABI는 fixed-width POD·opaque handle·status code·명시 release 함수를 사용한다. Result·String·Array·vtable·예외를 경계에 직접 전달하지 않는다. allocator ownership·calling convention·packing·structSize·ABI version은 이름 정책과 독립 검증한다. serialization의 TypeId/FieldId는 namespace나 display name 변경으로 바뀌지 않는다.

정책 확정 Gate는 C++ public header·C harness·커스텀 언어 prototype의 생성/호출/파괴, 동명 외부 SDK symbol과의 link, 두 plugin 공존, symbol export 목록, 오류 stack/source map, rename 후 과거 asset 호환을 포함한다. 작은 API에서 namespace 중심·경계 prefix·함수 테이블을 비교하고 생성 코드량·개발자 탐색·호출 비용을 기록한다. 아직 구현하지 않은 glue의 편의성을 사실로 가정하여 전 타입을 일괄 개명하지 않는다.

도구 표시는 Rocks Studio(웹 에디터·native viewport), Rocks Build(build driver), Rocks Meta(메타 생성기), Rocks Lens(로그·프로파일러·상태 분석)로 통일한다. 명령은 `rocks build`, `rocks meta`, `rocks cook`, `rocks inspect`처럼 제품 이름 아래에 배치한다. 도구 브랜드의 Rocks 표시는 C++ 타입 분류 접두사와 별개다.

2.5 Windows·macOS 개발 환경의 동등 지원

Windows와 macOS는 모두 **정식 개발 플랫폼**이다. 두 OS에 전체 Rocks Studio, 웹 제작 화면과 네이티브 3D viewport, 프로젝트 생성, 빌드·메타 컴파일러·셰이더 cook·DDC·에셋 편집·PIE·테스트·headless를 제공한다. macOS를 Metal runtime 시험 전용으로 한정하지 않는다. Linux의 runtime·DedicatedServer·headless baseline은 별도 지원표로 관리하며, Linux 전체 Studio는 이 요구에서 자동 확정하지 않는다.

공통 Studio 기능과 데이터 계약은 유지하고 OS 통합부만 교체한다. Windows shell은 WebView2와 네이티브 viewport surface, macOS shell은 WKWebView와 AppKit/Metal surface를 사용한다. 웹 화면은 동일 명령·transaction·selection·undo 스키마를 사용하며 플랫폼마다 다른 frontend를 별도로 제작하지 않는다. WebView2는 Windows 앱 내 웹 콘텐츠 통합을, WKWebView는 AppKit 앱의 웹 view 통합을 제공한다. Microsoft WebView2, Apple WebKit for AppKit

Native viewport는 web canvas에 영상만 전송하는 대체품이 아니라 해당 OS의 실제 graphics surface다. focus·IME·단축키·clipboard·drag/drop·gamepad·고해상도 배율·다중 monitor·window 재배치·PIE input capture를 공통 acceptance로 검증한다. JS↔host 메시지는 versioned value/handle만 전달하고 raw pointer·native 객체 소유권을 전달하지 않는다. web process 재시작

·viewport close·PIE 종료 시 session/generation을 검사하여 늦은 요청을 폐기한다. OS UI thread와 Render owner lane의 경계를 유지한다.

개발 영역	Windows·macOS 공통 필수 계약	플랫폼별 실행·배포 규약
Core·Object·Task	동일 소스의 CPU-only build/test, allocator·handle·GC·shutdown 검증	각 OS의 native compiler·sanitizer/진단 도구로 시험하고 미지원 검사를 NOT RUN으로 기록
Build·MetaCompiler	같은 manifest/action graph·진단 스키마·증분 무효화 규약	Windows SDK와 Apple SDK를 독립 toolchain으로 고정; host 실행 파일과 target 산출물 분리
Asset·Shader·DDC	같은 import/cook API·content identity·원자적 게시·취소·복구	target compiler worker를 명시 선택; host 간 결과 동등성은 시험한 artifact만 공유
Studio·PIE	World/Session 분리, material/UI/cutscene/prefab 등 완성된 편집 기능의 양OS 제공	WebView2/WKWebView 차이를 adapter에 한정; macOS Metal viewport와 Windows native RHI 각각 검증
Headless·CI	`rocks build` , `rocks cook` , `rocks test` 등 동일 CLI 계약, exit code·JSON·timeout·graceful shutdown	CPU-only는 window/GPU/WebView가 없는 환경에서 수행; offscreen GPU suite는 별도 실행
배포·디버깅	version·symbol·crash artifact·지원 manifest 일치	Windows 서명·설치 검증과 macOS app bundle·서명·notarization 검증을 각각 native CI에서 수행

표 7

Toolhost와 target ABI는 `OS × CPU(x64/ARM64) × compiler/SDK × configuration × contract version` 으로 식별한다. x64와 ARM64의 executable·library·PCH/BMI·object·symbol을 교환하지 않는다. 각 지원 조합은 빌드 가능·실행 검증·배포 검증 상태를 별도 기록하며, 에뮬레이션 실행을 native 검증으로 대체하지 않는다. 프로세스/worker 경계에는 크기·정렬·endianness를 정의한 직렬화 메시지 또는 검증된 C ABI만 사용한다. C++ 객체 layout·예외·CRT allocator를 compiler/architecture가 다른 모듈 사이의 ABI로 노출하지 않는다. native plugin은 같은 target ABI로 별도 빌드한다. 이 규약은 초기 정적 모듈 구성에도 적용하며 동적 loader를 Foundation 선행 조건으로 만들지 않는다.

경로의 대소문자 충돌·Unicode 정규화·separator·symlink·긴 경로·파일 잠금·원자적 교체 차이를 Platform/FileSystem에서 검증한다. 공통 에셋 ID는 절대 경로에 의존하지 않으며 저장소에는 대소문자로만 구분되는 경로를 허용하지 않는다. 코드 생성은 target ABI·define을 명시 입력으로 받고 native host 구조체의 offset을 타 target의 값으로 재사용하지 않는다. DDC/action key에는 target·compiler/translator digest·SDK·옵션·binding/layout·schema 및 실제 입력 closure를 포함한다. host가 다르면 우선 분리하고, 정규화한 artifact의 byte 동일성을 검증한 변환만 공용 cache namespace를 사용한다. 영상의 허용 오차 비교를 cache 산출물의 동일성으로 대체하지 않는다.

셰이더 target과 compiler host의 구분. 5개 RHI의 첫 정식 renderer gate는 모두 필수지만, 모든 API를 모든 OS에서 실행한다는 의미는 아니다. Core/Foundation은 GPU·WebView·shader SDK에 의존하지 않는다. Studio 양OS 동등 지원은 제작 작업과 데이터의 동등 지원이며 hardware capability 차이의 은폐가 아니다.

target	산출물·기준 compiler 경로	Windows·macOS host 처리	필수 target 검증
D3D11	SM5 profile의 HLSL→FXC/D3DCompile→DXBC	Windows SDK worker를 기준으로 지정; macOS는 같은 action을 Windows worker에 전달 가능	Windows 실제 D3D11 device; DXIL을 DXBC로 대입 금지
D3D12	HLSL→고정 DXC→DXIL	검증된 native DXC build 또는 명시 Windows worker; macOS용 배포 binary 존재를 가정하지 않음	Windows 실제 D3D12 device·validator·SM/caps

target	산출물·기준 compiler 경로	Windows·macOS host 처리	필수 target 검증
Vulkan	HLSL→고정 DXC SPIR-V, 지원 target environment 고정	host별 검증된 compiler build/worker; 동일 source만으로 산출물 동등성 간주 금지	지정 Windows/Linux Vulkan device; 번역 layer 실행은 별도 profile
Metal	typed IR→MSL→Apple offline library 기준; DXIL→Metal Shader Converter는 검증 후 선택 경로	macOS Xcode 도구와 Windows용 Apple 도구의 version·host architecture·기능 지원표를 기록; 사용 불가 조합은 macOS worker	macOS 실제 Metal device·reflection/layout·PSO; compiler 성공만으로 GPU 지원 판정 금지
OpenGL	IR→GLSL target version/profile; 선택 정적 validator	host의 compiler/translator 결과와 target driver의 compile/link를 구분	지정 Windows/Linux core profile device; driver program binary는 호환 driver/device key 없이 공유 금지

표 8

FXC의 offline HLSL compile과 DXC의 DXIL/SPIR-V 경로는 별도 compiler 계약이다. Microsoft FXC, Microsoft DXC

Apple은 Windows용 Metal 제작 도구와 macOS/Windows용 Metal Shader Converter를 제공한다. 따라서 Metal cook 전체를 macOS 전용으로 단정하지 않는다. 다만 converter의 요구 OS·Argument Buffer tier 등은 일반 MSL 경로와 다를 수 있고, GPU binary archive에는 GPU/OS 호환 조건이 있다. 선택한 SDK·converter 버전의 제약을 manifest에 고정하고 미지원 GPU에는 검증된 MSL 경로 또는 명시 unsupported를 적용한다. macOS runtime·서명·배포 검증은 Windows의 shader compile 결과로 대신하지 않는다. Apple Metal 도구, Metal Shader Converter 조건, WWDC23 shader 제작 설명

Gate	필수 수용 기준
PL01 · C00–Foundation	Windows·macOS native CPU-only Core/Object suite; 실패 주입·잔존 allocation/lease·thread 종료, GPU/WebView link dependency 0건
PL02 · M3 도구	양OS clean checkout→project→build/meta/cook/test, 공백·Unicode·대소문자 충돌 fixture, cold/warm DDC와 취소/재시작
PL03 · M3/M4 renderer	5개 backend의 §11.5 기본 profile·Deferred/Forward·offscreen·실장비 검증; SDK/장비 미확보는 미완료
PL04 · M3/M4 Studio	양OS 웹+native viewport, asset 편집/undo·기본 material·PIE·저장/재개·input/IME/DPI·프로세스 실패 복구
PL05 · 해당 기능 첫 정식 단계	UI Designer·컷씬 등 후속 편집기의 같은 기능을 양OS에서 검증; Windows만 통과한 결과로 공통 기능 완료 선언 금지
PL06 · 배포	양OS native CI의 CPU/headless/GPU 결과 구분, 서명된 패키지의 깨끗한 장비 실행, symbol 복원·crash 수집·종료 후 자원 누수 0건

표 9

양OS 제작 환경과 5개 RHI의 추가 비용은 기존 단일 Windows/D3D12 중심 일정에 별도로 반영한다. Foundation 완료 뒤 toolchain·surface·shader 변환 위험을 확인하고, PL01–06과 API01–06의 실제 통과 범위에 따라 M3 이후 인력·장비·일정을 재산정한다. 기간 단축을 위해 macOS Studio나 5개 backend를 선택 후속으로 재분류하지 않는다. 성능 최적화 전에 양OS 정확성·실패 복구·누수 검증을 완료한다.

3 모듈 구조와 의존성

3.1 계층 구조

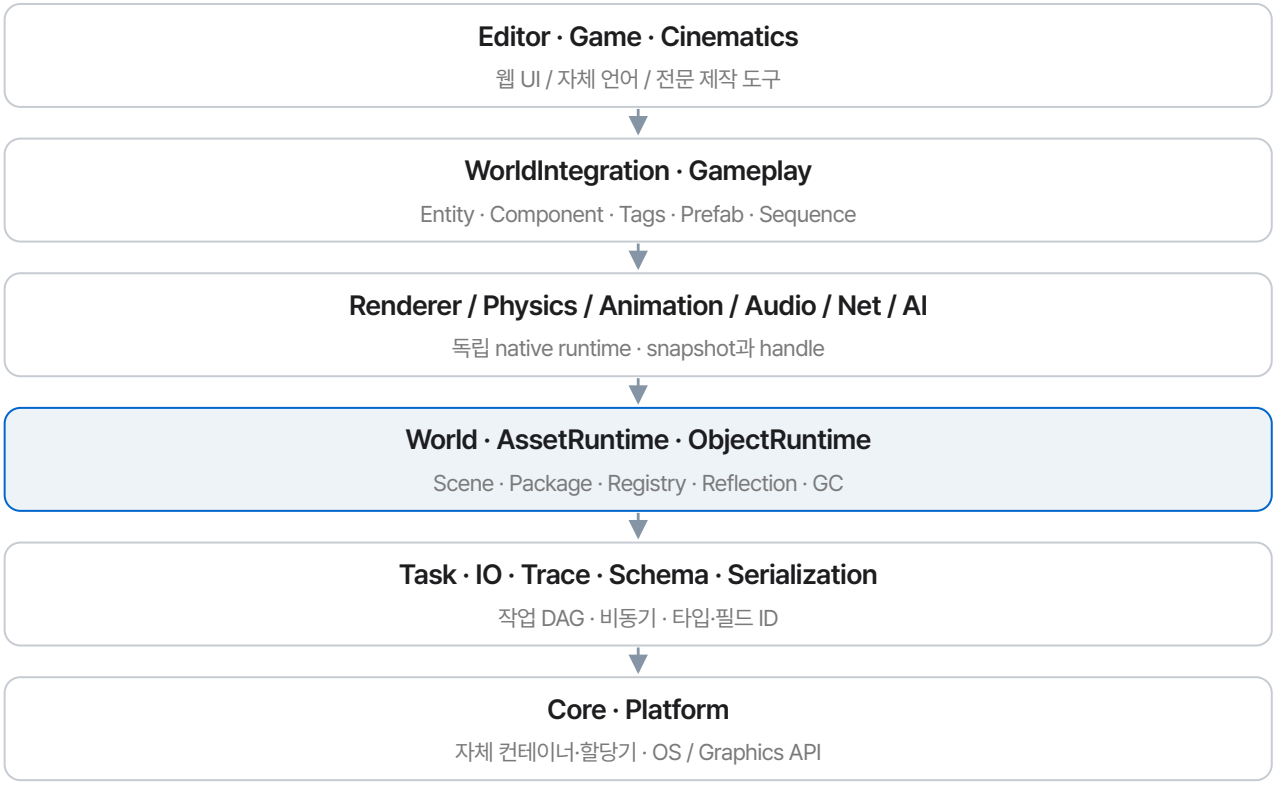


그림 1 · 모듈 의존성의 계층 (개념도)

그림의 아래쪽 모듈은 위쪽 모듈을 참조하지 않는다. 시스템 간 데이터 전달은 공통 계약 타입과 WorldIntegration의 조합으로 구현한다. Registry·GC·Reflection의 밀접한 내부 결합은 ObjectRuntime이라는 하나의 라이브러리 안에서 관리한다.

모듈	직접 의존성	책임과 금지 사항
EngineBase	최소 언어 지원 header	primitive·Guid POD·ABI. allocation 없음
PlatformBase / PlatformWindow	EngineBase·OS kernel / Core·PlatformBase·window SDK	Base는 메모리·파일·스레드·clock, Window는 창·입력. CPU-only는 Window 제외
Core	EngineBase·PlatformBase	컨테이너·할당기·문자열·수학·진단·Atomic. Window 의존 금지
Task / IO / Trace	Core	작업 DAG, 비동기 IO, 계측. 서로의 높은 계층에 의존하지 않음
MetaSchema / Serialization	Core	TypeId·FieldId·Archive 계약, 버전 규칙. Object 없음
ObjectRuntime	Core, Task, MetaSchema	Object, Registry, GC, Reflection 실행. Asset 의존 금지

모듈	직접 의존성	책임과 금지 사항
ObjectSerialization	ObjectRuntime, Serialization	R2 객체 Archive bridge. ObjectRuntime의 역참조 금지
AssetRuntime	IO, Task, ObjectRuntime, Serialization, ObjectSerialization	Package, LoadTicket, AssetId, DDC reader
World	AssetRuntime, ObjectRuntime, Task	Entity/Component, Scene Graph, Cell, 시스템 실행 순서
RHI	Core, Task, graphics SDK adapter	device·offscreen은 Window 불필요, interactive swapchain만 Window 사용. World·Object 없음
RenderGraph / Renderer	RHI, Task, Core, RenderData	pass·GPU Scene·조명·메시. World 순회 없음
MaterialSchema / Compiler / Runtime	Core·Schema / Host ShaderWorker / RenderData·RHI	graph 계약·host 컴파일·cooked 실행 분리; 14.6절 세부 의존
Physics / Audio / Navigation	Core, Task, 각 vendor adapter	독립 네이티브 runtime. Entity/Renderer 내부 주소 없음
Animation / VFX / UI	Core, Task, Asset 데이터 및 해당 backend	데이터 처리·실행. World 연결은 통합 계층에서 수행
NetCore / Replication	Core, IO / ObjectRuntime, MetaSchema	transport와 객체 replication 분리. Renderer 없음
ScriptRuntime	ObjectRuntime, MetaSchema, Task	bytecode VM·root provider·native binding. World·Renderer 의존 금지
Gameplay / WorldIntegration	World, ScriptRuntime 및 필요한 시스템	component binding·render/physics/audio command 변환
BuildDriver / BuildAgent	Host Core, manifest·toolchain·artifact store	action DAG·캐시·local/remote executor. Shipping 링크 금지
EditorBackend	WorldIntegration, Asset 도구, Transaction	편집 명령·PIE·검증. Runtime이 이 모듈을 참조하지 않음
EditorWeb / HostTools	RPC schema / Clang·DXC·임포터	프런트엔드·메타 생성·쿠킹. Shipping 링크 금지

표 10

RenderData, PhysicsData, AudioData는 안정적인 ID와 평범한 데이터 구조만 포함한다. World의 MeshComponent가 Renderer를 직접 소유하는 대신 WorldIntegration이 자원 핸들과 RenderInstanceId를 연결한다. 공유 계약 라이브러리로 상향 의존을 대체하되 모든 모듈이 참조하는 범용 서비스 모음으로 확대하지 않는다.

3.2 저장소와 빌드 단위

```
Engine/Base  PlatformBase  Core  Task  IO  Trace  MetaSchema  Serialization
Engine/ObjectRuntime/{Registry,Reflection,GC}  ObjectSerialization
Engine/AssetRuntime  World  WorldIntegration  Gameplay
Engine/RHI/{Null,D3D11,D3D12,Vulkan,Metal,OpenGL}  RenderGraph  Renderer
Engine/MaterialSchema  MaterialRuntime  MaterialAssetBridge
Tools/MaterialCompiler  MaterialCook  Editor/Material  Editor/Web/Material
Engine/Physics  Animation  Audio  Navigation  Network  AI  VFX  UI  ScriptRuntime
Editor/Backend  Editor/Web
Tools/{BuildDriver,BuildAgent,MetaCompiler,ScriptCompiler,Cooker,ShaderWorker,Packager}
ThirdParty/Adapters  Tests/{Unit,Stress,Integration,Render,Network}
Samples/VerticalSlice  Benchmarks  Build  Schemas
```

공개 헤더와 Private 구현을 분리한다. 모듈 manifest에 공개/비공개 의존성, Host/Target 구분, 초기화 순서, 버전, 선택 기능을 기록한다. 초기화는 명시적인 BootServices 순서로 수행하고 정적 전역 생성자의 상호 의존을 금지한다. Shutdown은 신규 게임/편집 요청 차단 → 작업 취소 요청 → World 비활성·자원 detach → teardown 작업과 callback drain → GPU fence·audio epoch·physics step 완료 → root/lease 반환·최종 GC → GC가 생성한 추가 cleanup/backend 작업 drain → owner/retire/queue closure 확인 → scheduler 종료 → type metadata/code lease와 trace writer 종료 → 모듈 역순 종료로 수행한다. teardown 전용 제출은 마지막 drain까지 허용하며 새로운 cleanup이 생기면 완료 집합이 닫힐 때까지 GC/drain을 반복한다. timeout은 실패이며 scheduler를 먼저 종료하거나 강제 free하지 않는다.

3.3 수학 라이브러리·SIMD·정밀도 정책

Rocks의 수학은 검증된 라이브러리를 채택하고, 공개 API와 데이터 규약을 자체 코드 스타일로 제공한다. 벡터·행렬·쿼터니언 연산, 역행렬, 분해, 보간, 초월함수와 SIMD 근사 알고리즘을 처음부터 작성하지 않는다. 저장형 wrapper, 좌표·행렬 규약 변환, 오류 정책, 선택 라이브러리 호출 adapter와 batch 실행 구성은 자체 구현 범위다. 기능이 부족하면 검증된 추가 라이브러리를 평가하며 임시 자체 수학 구현으로 보충하지 않는다.

라이브러리의 품질·성능 평가와 공개 API의 명명·코드 스타일은 별도 기준으로 관리한다. 공개 API는 `rocks::Vec3`, `rocks::Quat`, `rocks::Mat4`, `rocks::WorldPosition` 처럼 의미가 명확한 이름을 사용한다. `XMVECTOR` · `XMVector…` 와 vendor 호출 규약을 게임 코드에 노출하지 않는다. RTM도 C 스타일 API를 사용하므로 원형을 그대로 공개하지 않는다. 코드 가독성·일관성·디버깅·유지보수 비용을 성능과 함께 선정 기준에 포함한다.

후보와 공개 근거

후보·참고 범위	확인된 사실	Rocks에서 확인할 조건
DirectXMath — f32 연산 backend 후보	Microsoft 문서는 단정밀 벡터·행렬 중심이며 double vector를 지원하지 않는다고 설명한다. 공식 범위. 공식 저장소는 SSE/NEON 계열과 비 Windows용 `sal.h` 준비 사항을 설명한다. 공식 저장소	단정밀 범위와 f64 World 요구를 분리한다. macOS/Apple Clang 빌드·NEON 경로·SAL 의존을 실제 검증한다. Windows 전용이라고 단정하지 않으며 이름을 감추는 것만으로 adapter 비용이 0이라고 가정하지 않는다.
RTM — f32/f64 통합 backend 후보	공식 README에 float32·float64, row vector 규약, OS X Xcode ARM64 CI, ACL v1.1.0 수학 코드에서 시작한 이력이 명시되어 있다. 공식 README. 함수 인자 전달·행렬 순서는 별도 문서에 정의된다. API 규약	요구 연산별 f64·SIMD 구현과 오차·속도를 확인한다. CI 플랫폼 목록이나 ACL 기원이 모든 연산의 최적화·모든 상용 엔진의 채택을 증명하지 않는다. 공개 Rocks API와 convention 변환 비용도 비교한다.

후보·참고 범위	확인된 사실	Rocks에서 확인할 조건
PhysX math — 상용 미들웨어의 공개 수학 코드·정책 참고	NVIDIA의 공개 `PxVec3` 계열에는 저장 멤버, 유한값 검사, 정규화 계열이 존재한다. 공식 소스	작은 수학 subset의 독립 채택 가능성, foundation 의존, 선택 버전의 실제 정밀도·SIMD 범위를 확인한다. 물리 엔진 전체를 Core에 링크하지 않는다. 해당 공개 API만으로 모든 연산이 SIMD라고 판단하지 않는다.
Sony Vectormath — 공개된 상용 SDK 계열 참고	Bullet 팀의 2007-07-28 공식 공지는 Sony가 기존 PS3 개발자용 Vector Math를 BSD 조건으로 공개하였으며 벡터·행렬·쿼터니언과 SSE·PPU·SPU·scalar 구현을 제공하였다고 설명한다. 공식 공개 공지	공개된 역사적 코드만 평가한다. 현재 PlayStation SDK의 수학 코드·최신 NEON/f64 지원·현대 CPU 성능으로 일반화하지 않는다. 유지보수와 필요한 연산의 범위를 확인한 뒤 참고 또는 backend 후보로 제한한다.

표 11

검토 기준일은 2026-11-10이다. 라이브러리는 아직 선정하지 않았으며 **Rocks용 후보 벤치마크도 실행하지 않았다.** 공개 저장소를 볼 수 있다는 사실과 재사용·재배포 권한은 구분하고, 채택 시 고정 revision·개별 파일의 license/notice·수정 내역을 dependency manifest에 남긴다. 선택 수학 subset의 실제 header 의존을 검사하여 STL 컨테이너·런타임 의존이 엔진 정책을 우회하지 않게 한다. 수학 API·adapter·검증을 Core의 책임으로 두고 알고리즘 구현은 선정 라이브러리에 위임한다.

저장형·계산형·모듈 경계

`MathTypes`는 Core의 값 타입·정밀도·정렬·직렬화 의미를 정의하고, `MathOps`는 scalar 규모 호출, `MathBatch`는 연속 배열·SoA 입력을 처리한다. backend adapter와 SIMD register 타입은 내부 구현으로 둔다. Core Math는 Object·World·Renderer·Physics·RHI·window에 의존하지 않는다. World·physics·render origin 간 변환은 각 통합 계층에서 수행한다.

기본 `Vec3/Quat/Mat4`는 f32 local/render 계산용이며 f64용 `Vec3d/Mat4d` 및 `WorldPosition`을 구별한다. 실제 구조체 크기·정렬·쿼터니언 성분 순서를 명시하고 padding을 파일·네트워크로 그대로 저장하지 않는다. 공개 C ABI·plugin·reflection·VM·GPU upload에는 명시적인 plain storage와 변환 함수를 사용한다. `__m128`, NEON register, vendor vector·matrix 타입과 CPU별 ABI를 저장 포맷이나 module ABI로 전달하지 않는다. SIMD 정렬이 필요한 batch storage는 별도 타입·allocator 계약으로 지정한다.

일반 벡터 연산마다 virtual/function-pointer dispatch나 CPU feature 검사를 수행하지 않는다. 성능에 민감한 반복은 storage load → 여러 연산 → store를 batch 내부에서 수행하고 runtime dispatch는 batch kernel 진입에서만 선택한다. 공개 이름을 vendor typedef로 바꾸는 방식은 채택하지 않는다.

공개 storage/API 안정성과 vendor 타입·심벌의 공개 signature 미노출을 보장하되, vendor header의 완전 비공개와 모든 작은 연산의 inline·무비용 wrapper를 동시에 보장하지 않는다. 선택지는 내부 detail inline header를 통한 컴파일 의존 허용, 또는 private translation unit에 격리하고 LTO·batch kernel로 호출 비용을 줄이는 방식이다. 전자는 header 변경의 재컴파일 범위·macro 격리를, 후자는 작은 함수의 외부 호출 비용을 측정한다. load/store·shuffle·calling convention을 포함한 adapter 비용과 유지보수 결과로 결정한다.

좌표 규약·정확성·결정성

기존 column vector 규약 `p'=M×p`, `world=parent×local`을 유지한다. row vector 라이브러리를 사용하는 경우 adapter에서 곱 순서·전치 의미를 변환한다. 메모리의 row/column-major 배치, 벡터의 행/열 의미, 좌우수 좌표계는 각각 별도 항목이다. 각도 단위, quaternion 순서·합성 방향·정규화 전제, clip depth·reverse-Z·projection·normal inverse-transpose를 명세하고 shader/physics/importer 경계의 변환 지점을 고정한다. 라이브러리 선택을 이유로 프로젝트 전체 규약을 묵시 변경하지 않는다.

일반 정밀도 경로와 빠른 근사 경로를 구분한다. 예를 들어 checked normalization/inverse와 명시적인 estimate API를 제공하되 수치 연산은 검증된 backend를 호출한다. '정확'은 명세된 오차 조건을 충족한다는 뜻이며 모든 함수의 수학적 exact·correct rounding을 약속하지 않는다. 허용오차는 f32/f64, 연산, 입력 범위, 길이·각도 단위, 행렬 condition에 따라 정한다. 하나의 전역 epsilon을 모든 검사에 사용하지 않는다.

NaN/Inf, signed zero, subnormal, zero-length, near-singular/singular matrix, 비정규 quaternion, 음수·zero scale의 입력 정책을 명시한다. 실패 가능한 API는 실패를 반환하고 입력·상태를 유지하며, 잘못된 역행렬을 identity로 조용히 대체하지 않는다. import·script·editor 경계에서 검증하고 hot kernel의 사전조건은 debug assertion과 fixture로 확인한다. Transform의 shear·KeepWorld 실패 정책은 §2.3을 유지하고 검증된 분해 루틴으로 표현 가능성을 확인한다.

전역 위치·cell 경계 계산은 f64 또는 명시된 정수 cell+local 표현을 사용한다. 카메라/physics 지역 원점을 f64에서 먼저 차감한 뒤 유효 범위의 상대값만 f32로 변환한다. 애니메이션·렌더 데이터 전체를 무조건 f64로 확대하지 않는다. 렌더/시각 연산의 플랫폼 간 비교는 허용 오차 기준이며, 이것을 네트워크 lockstep의 bitwise 결정성으로 간주하지 않는다. 결정성이 필요한 부분은 연산 집합·실행 순서·RNG·compiler/FP mode·FMA contraction·FTZ/DAZ·target을 별도로 고정하고 검증한다. `fast-math`를 전체 Core에 일괄 적용하지 않는다.

SIMD·일괄 연산 정책

x64의 기본 backend는 SSE2 지원 범위로 구성한다. AVX2/FMA kernel은 선택 확장이며 CPUID의 필요한 개별 기능과 OSXSAVE, XGETBV/XCR0의 XMM/YMM 상태 활성화를 모두 확인한 뒤 실행한다. AVX2가 있다고 FMA를 무조건 가정하지 않는다. 검사 전 AVX 명령이 실행되지 않도록 baseline과 확장 translation unit·compiler flag를 분리하고 LTO 결과도 검증한다. Intel 공식 AVX 실행 조건

ARM64는 NEON backend를 검증하며 macOS ARM64를 별도 CI/실기기 항목으로 둔다. scalar 경로도 동일 의미의 검증·fallback에 사용하되, scalar 통과를 SIMD 최적화 완료로 기록하지 않는다. 하나의 Vec3를 SIMD register에 보관하는 연산과 여러 객체를 한 번에 처리하는 SoA batch를 구분한다. transform propagation, AABB/frustum, quaternion blend, skinning 준비, point/vector transform은 입력 배치·tail·alignment·aliasing을 명시한 batch API를 사용한다. 필요한 최적 kernel은 검증된 구현을 채택하고, 자체 근사식·역행렬·보간 알고리즘을 새로 작성하지 않는다.

평가는 단일 연산 microbenchmark와 위 엔진 workload를 함께 수행한다. 같은 정밀도·오차·컴파일 설정에서 adapter 포함 비용, warm/cold working set, 작은/큰 batch, 코드 크기, compile 시간, scalar/SSE2/AVX2/NEON 경로를 비교한다. 상수 folding·dead-code elimination을 방지하며 CPU·compiler·flags·revision과 입력을 보존한다. 특정 라이브러리나 넓은 SIMD가 항상 빠르다는 결론을 미리 정하지 않는다.

구현 순서와 수용 기준

첫 Core 단계에서 타입·규약·failure API를 확정하고 후보의 최소 adapter와 공통 시험을 작성한다. 정확성·플랫폼·스타일 평가 후 기본 backend를 선택한다. 필수 f64와 baseline SIMD 지원을 확인한 다음 실제 profile에 근거하여 batch·선택 ISA를 확장한다. World/RHI가 준비되기 전에는 CPU-only fixture로 수행하며 후보 비교를 이유로 수학 알고리즘 전체의 자체 구현을 시작하지 않는다.

기준	입력·검증	통과 조건
MX01 · API·의존	동일 vector/transform 예제를 후보 adapter로 작성, 공개 header·dependency 검사	`rocks::Vec3/Quat/Mat4` 스타일 유지, vendor 타입·호출 규약의 public ABI 노출 0건, 불필요한 STL/Window/RHI/Physics 의존 없음. 가독성·유지보수 판단 기록
MX02 · 일반·예외 수치	upstream tests, 검증된 reference 데이터·독립 고정밀 도구와 비교; NaN/Inf·±0·subnormal·zero length·singular·near-singular 포함	연산별 오차·실패 정책 충족, 일반/estimate 경로 결과를 구분, 실패 시 잘못된 성공값·상태 변경 없음
MX03 · 규약 왕복	비대칭 행렬·서로 교환되지 않는 회전/이동·quaternion·TRS/shear·normal·projection fixture	parent×local 순서·handedness·storage 변환 일치, CPU/reference shader 규약 일치. 실제 GPU pixel 검증은 RHI 단계에서 별도 수행
MX04 · 대규모 위치	먼 f64 world 좌표의 작은 상대 이동, origin 변경·cell 경계·연속 왕복	f64 차감 후 f32 변환, scene 요구 허용오차 충족, 입력 범위 초과·nonfinite 진단, 원점 변경에 의한 불연속 없음

기준	입력·검증	통과 조건
MX05 · ISA·플랫폼	Windows/Linux x64 baseline·선택 ISA, macOS ARM64; 미지원 CPU/OS feature masking	SSE2 baseline에서 불법 AVX 실행 0건, CPUID·OSXSAVE/XCR0 조건 일치, ARM64 NEON 경로 확인, SAL 등 후보 이식 의존과 지원 상태 명시
MX06 · batch·ABI	count 0/1/벡터폭±1·큰 batch, unaligned/alias 허용·금지 입력, AoS/SoA·module 경계	tail/OOB·UB 0건, 규약에 따른 alias 처리, scalar와 허용오차 내 일치, padding/register 타입의 직렬화 유출 없음
MX07 · 성능·반복 개발	transform·culling·blend fixture, 직접 vendor 호출 대 adapter, 작은/큰 batch·warm/cold	실행 결과·오차와 함께 시간 분포·코드 크기·compile 비용 보존, 측정 기반 채택 근거 확보. 미실행 결과를 성능 우위로 기재하지 않음
MX08 · 재현·업데이트	compiler·FMA/FP mode·thread 경로·backend revision 변경과 저장 데이터 재생	시각 허용오차와 별도 결정성 subset을 각각 판정, upstream 변경 후 회귀 실행, manifest·license/notice·변경 내역 추적

표 12

3.4 코드 서식·clang-format 표준

사람이 알아보기 좋은 코드 스타일 유지를 코드 작성과 리뷰의 명시적 원칙으로 적용한다. 읽기 쉬운 이름·적절한 빈 줄·짧고 명확한 함수 구조를 요구하며 formatter 통과만으로 가독성 검토를 대체하지 않는다.

엔진 소유 코드는 **탭 문자를 사용하지 않고 스페이스 4칸으로 들여쓴다.** 함수·타입 정의 사이, 검증·상태 갱신·결과 반환 등 의미가 다른 처리 단계 사이에는 빈 줄을 둔다. 관련된 짧은 선언은 묶되 여러 문장을 한 줄에 압축하지 않는다. 긴 함수·복잡한 조건·여러 책임을 서식만으로 해결하지 않고 작은 함수와 명확한 이름으로 분리한다.

`clang-format` 버전과 `.clang-format`을 저장소에 고정한다. 에디터 저장 시 포맷·선택 영역 포맷·로컬 검사·CI가 같은 실행 파일과 설정을 사용한다. `.editorconfig`는 `indent_style = space`, `indent_size = 4`, `tab_width = 4`, LF·파일 끝 개행·불필요한 행 끝 공백 제거를 명시한다. 열 제한 100자는 초기 기본값이며 줄바꿈 시 가독성을 우선한다. C++/Objective-C++는 같은 규칙을 적용한다.

첫 C00 산출물에 `.editorconfig`, `.clang-format`, `Docs/CodingStyle.md`를 포함한다. CT01에는 설정 파싱, 실제 tab 부재, 같은 입력의 포맷 재실행 동일성, 의도적으로 서식을 깨뜨린 fixture의 실패를 추가한다. PR에서는 `clang-format --dry-run --Werror`에 해당하는 검사를 수행하고 generated 파일은 생성 단계부터 같은 설정으로 출력한다. formatter 버전 변경은 별도 변경과 diff 검토로 처리한다. 외부 vendor 원본은 기본적으로 재포맷하지 않고 엔진 소유 adapter에 Rocks 규칙을 적용한다.

`SeparateDefinitionBlocks: Always`와 짧은 함수/블록/조건/반복문의 한 줄 압축 금지 설정으로 정의 사이의 간격을 유지한다. 자동 도구가 처리 단계의 의미를 판단하지는 못하므로 함수 내부의 논리적 빈 줄·책임 분리는 리뷰 항목으로 남긴다. 문서의 C++ API 예제에도 같은 formatter를 적용하되, 서식 통과를 API 컴파일·엔진 구현 테스트의 통과로 해석하지 않는다. clang-format 공식 설정

4 Core 컨테이너와 메모리

4.1 할당기 구성

첫 R0/R1은 단순 기준 heap/page와 잠금 기반 회계로 정확성을 검증한다. 크기 등급별 slab·thread cache·Object 전용 size-class pool은 C28 이후 측정 기반 최적화안이며 첫 객체 작업의 선행 조건이 아니다. Object 주소 비이동은 기준 heap에서도 유지한다. Registry 청크와 객체 본체는 서로 다른 메모리 영역이다. 큰 객체나 특수 정렬은 별도 page allocation으로 지원한다.

할당기	사용 대상	반환 규약
PageAllocator	대형 자원, slab backing	reserve/commit/decommit 구분, OS page 단위
Slab / Pool	Object 본체, 고정 작업 노드	원래 allocator와 size class로 반환, 원격 해제 큐
LinearArena	frame·pass·임포트 임시 데이터	Reset 시 전체 무효. 사용 작업과 fence 완료가 선행
StackArena	중첩 scratch scope	marker 역순 rollback, 비trivial 소멸자 별도 호출
UploadRing	GPU upload·constant buffer	요구 정렬과 각 사용 queue fence 완료 후 재사용
BudgetAllocator	에셋·지오메트리·오디오 pool	상한 초과 시 eviction 또는 요청 실패, 묵시적 무한 증가 금지

표 13

```
struct AllocationRequest {
    uint64 bytes;
    uint32 alignment;
    MemoryTag tag;
};

struct Allocator {
    virtual Result<void*> TryAllocate(AllocationRequest) noexcept = 0;
    virtual void Free(void* ptr, uint64 bytes, uint32 alignment) noexcept = 0;
    virtual ~Allocator() = default;
};

template <class T, class AllocPolicy = DefaultAlloc>
class Array {
public:
    Result<void> TryReserve(uint64 capacity);

    template <class... Args>
    Result<T*> EmplaceBack(Args&&...);
    Span<T> View() noexcept;
    void Clear() noexcept;
};
```

위 API는 시그니처 계약 예시이며 독립 컴파일용 완성 헤더가 아니다. 이후 코드도 동일하다. allocator는 그 allocator를 사용하는 컨테이너보다 오래 유지해야 한다. size×count와 alignment 올림의 정수 오버플로를 검사한다. Free(nullptr)는 무효 연산, 0-byte 요청은 명시된 최소 블록 또는 null 중 하나로 전역 통일한다. TryReserve 실패 시 원래 배열을 보존한다.

4.2 필수 자료구조와 정확성

타입	역할	무효화·동시성 규약
Array / InlineArray / StaticArray	연속 저장과 소규모 최적화	재할당 시 view·iterator 무효. 동시 쓰기 없음
Span / StringView	비소유 view	원본·arena보다 오래 보관 금지. 비동기 캡처 시 복사 또는 owner token 필요
ChunkedArray	주소가 고정된 청크와 슬롯	청크 포인터 publication 규약 필요. 일반 쓰기는 자동 thread-safe 아님

타입	역할	무효화·동시성 규약
SparseArray / SlotMap	슬롯 재사용, 세대 핸들	삭제 후 세대 증가. 순회와 삭제는 별도 단계
HashMap / HashSet	open addressing 기반 조회	rehash 시 참조 무효. 병렬 조회 중 변경 금지 또는 shard lock
BitArray	mark·가시성·residency	공유 bit word 갱신은 atomic 또는 word별 소유권
String / Name	UTF-8 소유 문자열, interned name	Name은 세션 내 ID. 영속 저장은 문자열 또는 안정 ID 사용
Function / Delegate	SBO callback, 이벤트	비소유 FunctionRef와 소유 Function 구분. 구독 해제 토큰·재진입 규칙
UniquePtr / SharedPtr	GC 외 네이티브 수명	SharedPtr은 GC 대체가 아님. 순환 소유 금지, atomic refcount 선택형
SPSC / MPSC queue	스레드 간 전달	지정된 producer/consumer 수만 허용. overflow·종료 정책 필수

표 14

T의 생성·복사·이동·파괴 횟수를 추적하는 검증 타입으로 각 조합을 검사한다. C++23에 일반적인 '임의 타입 byte relocation' 보장이 있다고 가정하지 않는다. portable 경로는 4.3절의 trivially-copyable 조건으로 한정하고, 확장 relocation은 명시 compiler capability와 엔진 trait를 함께 검사한다. 자기 주소에 의존하는 타입은 byte relocation을 금지한다. 비trivial 타입은 placement construction과 명시적 소멸을 수행한다.

동시 컨테이너를 일반 컨테이너의 변형으로 조기에 확장하지 않는다. 초기 Task queue는 잠금 기반으로 정확성을 확보하고, work-stealing deque와 lock-free free list는 ABA·메모리 회수 검증 후 교체한다. 할당 메타데이터는 태그·호출 위치·소유 모듈을 기록하며 shipping 계측 수준은 선택한다.

4.3 비예외 코드와 재배치의 정확성

Runtime container의 element construct/move/destruct 경로는 noexcept를 계약으로 요구한다. 실패 가능한 업무 초기화는 생성자 밖의 Result 기반 Initialize로 수행한다. 따라서 TryReserve는 allocation 실패를 복구할 수 있으나 throw하는 임의 T의 이동까지 복구하는 API는 아니다. 이 제약은 concept/trait 진단으로 강제한다.

portable byte relocation은 trivially-copyable complete element이며 저장소·정렬·객체 수명 조건을 만족하는 경우로 한정한다. implicit-lifetime이라는 사실만으로 값 보존·relocation을 승인하지 않는다. 그 밖의 타입은 noexcept move construction과 명시 소멸을 사용하고, compiler extension은 compiler/version/target capability별 별도 경로로 검증한다. overlap은 memmove 또는 placement 이동 알고리즘의 별도 수명 규약을 따른다. C++23 초안의 byte-copy 보장과 implicit-lifetime의 역할을 구분한다. S52

implicit-lifetime이지만 nontrivially-copyable인 타입, self-pointer·소유권·over-aligned 타입을 positive/negative fixture에 포함한다. 실행 중 우연히 정상인 결과만으로 언어 규약 준수를 판정하지 않는다. persistent file·network·FFI는 container 내부 layout과 독립된 schema를 사용한다.

5 Object와 청크 레지스트리

5.1 객체 모델과 핸들

Object는 reflection, editor 노출, 직렬화, GC가 필요한 객체의 기반이다. 매 정점·입자·physics body·render instance를 Object로 생성하지 않는다. Entity/Component는 관리 객체, 대량 처리 데이터는 SoA 또는 네이티브 pool에 보관한다.

```
struct ObjectHandle {
    uint32 index;
    uint32 generation;
};

struct ObjectSlot {
    Atomic<Object*> address;
    Atomic<uint32> generation;
    Atomic<uint32> state;
    TypeId type;
};

class Object {
public:
    ObjectHandle Handle() const noexcept;
    TypeId Type() const noexcept;
    void RequestDestroy();

protected:
    virtual void BeginDestroy();
    virtual bool IsReadyForFinishDestroy() const;
    virtual ~Object() = default;
};

class ObjectRegistry {
public:
    template <class T, class... Args>
    Result<RootRef<T>> NewObject(ObjectInit, Args&&...);

    template <class T>
    Result<Borrow<T>> Resolve(ObjectHandle, GcReadScope&, const ObjectAccessContext&);
};
```

기본안은 청크당 64K 슬롯이다. index의 상위 비트로 청크, 하위 비트로 offset을 계산한다. 최대 청크 수에 맞는 원자 포인터 디렉터리를 주소가 고정된 상태로 확보하고, 청크는 필요 시 commit한다. 슬롯 크기 32 bytes 가정 시 100만 슬롯은 약 32 MB이며 객체 본체·mark bitmap·정렬 비용은 별도이다. 실제 크기는 static_assert와 벤치마크로 확정한다.

ObjectHandle의 index 0 또는 별도 예약값은 null이다. generation wrap 직전 슬롯은 재사용을 중단한다. 청크와 free list를 공유하는 생산자는 shard 또는 잠금으로 시작한다. '슬롯 주소 안정'과 '객체의 유효 수명'은 다른 보장이다. 세대 번호만 확인한 뒤 raw pointer를 반환하는 것은 GC와의 동시 접근을 안전하게 하지 않는다.

5.2 생성·파괴 상태 전이

Free → Reserved → Constructing → Published → DestroyRequested → Retiring → Free를 기본 상태로 사용한다. 생성 중에는 private root를 유지하며 TypeInfo, 본체 생성, 초기 필드, handle을 완성한 뒤 release publication한다. 조회는 acquire로 상태와 주소를 관찰한다. 생성 실패 시 역순 정리하며 미완성 객체를 GC·에디터·네트워크에 공개하지 않는다.

NewObject가 반환하는 RootRef는 첫 소유자의 추적 가능한 필드에 연결될 때까지 수명을 유지한다. Outer는 이름·패키지 계층이며 GC 소유권을 자동 의미하지 않는다. Entity가 Component를 유지하려면 명시적 ObjectRef/GcArray edge가 있어야 한다. 패키지 로더, 월드, editor selection, native subsystem도 root token을 등록한다.

RequestDestroy는 논리적 사용 종료이며 즉시 delete가 아니다. owner Game lane에서 DestroyRequested 전이와 신규 사용·일반 task/activation admission 차단을 먼저 선형화한다. 그 뒤 기존 소유자의 짧은 teardown 전용 권한으로 EndPlay/Deactivate 등 필요한 종료 hook을 수행한다. 일반 Resolve/TryPin은 계속 실패하며 hook은 no-yield/no-safepoint이며 재진입 RequestDestroy는

idempotent다. 다른 lane의 요청은 Game lane commit 전까지 취소 요청 상태이며 즉시 파괴 완료로 보고하지 않는다.

BeginDestroy는 정확히 1회 실행하여 추적 edge를 정리하고 render/physics/audio의 독립 자원 token 해제를 요청한다. GC가 발견한 unreachable graph에도 적용되므로 managed 이웃의 생존·호출에 의존하지 않는다. cleanup의 필드 변경 자체도 기존 Borrow/GC-aware reader가 끝난 owner-lane 경계에서 수행한다. 후속 비동기 cleanup은 POD·backend token만 사용하며 Object graph를 역참조하지 않는다.

최종 회수 조건은 cleanup ready, 기존 strong root/edge 도달성 없음, reader epoch 종료, 관련 작업·모든 backend fence 완료의 논리곱이다. 기존 RootRef와 도달 가능한 ObjectRef는 논리적으로 종료된 객체의 저장소를 보존한다. 남은 field는 계속 trace하며 암묵적으로 child를 먼저 회수하지 않는다. 소멸자에서 async 작업 생성·객체 부활을 금지하고 사용자 직접 delete API를 제공하지 않는다.

5.3 클래스 기본값·서브오브젝트·컨텍스트

ClassDefaults는 TypeId별 불변 기본값과 default subobject graph를 보관하는 archetype이다. native 생성·script field 기본값 → class archetype → prefab/variant → instance override → 명시 spawn parameter 순서를 고정한다. default subobject는 stable SubobjectGuid를 가지며 생성 시 반드시 인스턴스별 복제·내부 참조 remap을 수행한다. 가변 component를 모든 instance가 공유하지 않는다. 기본값의 변경 전파도 prefab과 같은 old/new/local 비교를 사용한다. Object 기본값·참조 갱신 개념은 공식 자료로 대조하였다. S26

ClassDefaults와 TypeInfo는 module revision lease와 함께 유지한다. 기본값은 asset·불변 value만 참조하고 특정 PIE World의 Entity를 강하게 유지하지 않는다. editor 재구성 hook은 side effect가 제한된 C++/자체 언어 함수이며 preview world에서 실행한다. 게임 spawn·IO·RPC를 수행하는 범용 construction graph는 제공하지 않는다.

ObjectHandle은 프로세스 전역 slot identity다. World 전용 API는 별도 WorldId+WorldEpoch를 요구하여 editor/PIE/여행 전후 객체의 혼용을 거부한다. 명시 RequestDestroy는 강한 참조가 있어도 논리적 유효성을 종료할 수 있다. ObjectRef/RootRef는 메모리 보존과 논리적 생존을 구분하고 Resolve/TryPin은 DestroyRequested 이후 실패한다. 이미 빌린 pointer도 safepoint·yield·frame·비동기 전달을 넘지 못한다. 기존 RootRef 복사/이동은 보존 lease의 전달이며 신규 논리 사용이나 weak 승격이 아니다. 약한 참조 자동 null의 구현은 handle 검사로 대체하며 Unreal의 포인터 갱신 방식을 그대로 가정하지 않는다.

생성자는 backend 등록이나 최종 설정에 의존하는 파생 cache 계산을 수행하지 않는다. 값 복사·참조 remap 뒤 Initialize에서 파생 상태를 계산하고 Register에서 backend에 공개한다. pre-commit Initialize는 Result와 가역 journal을 사용하며 실패 시 미공개 graph를 rollback한다. graph Commit 이후 Register/Activate 실패는 신규 admission 차단·RequestDestroy·teardown/retire로 처리한다. 공개된 객체를 미공개 rollback 경로로 직접 해제하지 않는다.

복제·Undo 복구·prefab instantiate는 AllocateGraph → ApplyDefaults → Deserialize/Remap → Validate → pre-commit Initialize → Commit → Register → BeginPlay의 공통 instancing API를 사용한다. GUID/Outer·editor-only·transient field의 복사 정책을 명시하고 Commit 전 실패만 미공개 graph 전체를 역순 정리한다. Commit 성공 이후에는 context를 소비한 것으로 보고 정상 파괴·회수 경로를 사용한다.

5.4 ConstructionContext와 비공개 graph

C++ constructor는 bounded no-safepoint/no-yield 구간이다. GC 재진입·사용자 callback·IO·backend 등록을 금지하고 allocation pressure는 진입 전 처리하거나 사전 예약·명시 실패로 처리한다. 부분 생성 객체에 TypeInfo visitor를 호출하지 않는다.

ConstructionContext는 완전히 생성된 graph 구성원과 외부 dependency를 별도 private root 목록으로 유지한다. 긴 ApplyDefaults/Deserialize/Fixup은 이 context로 보호된 단계 경계에서 safepoint를 허용한다. 내부 identity lookup은 context 전용이며 public Resolve는 graph commit 전 실패한다. 초기 참조를 옮길 때부터 barrier를 적용하고 outgoing trace 완료 조건을 지킨다. 실패 시 context root를 유지한 채 구성 완료 역순으로 정리하며 미공개 graph를 외부에 노출하지 않는다.

CF06–CF08에서 ctor의 GC 요청 defer/진단, 단계 사이 강제 GC, allocation/remap 실패를 주입한다. 미구성 필드 visitor 호출·child 오수거·실패 graph 공개가 0건이어야 한다. destroy-request age·retained bytes·root chain을 진단하고 장기 RootRef로 유지된 종료 객체는 leak gate로 추적한다. timeout만으로 강제 free하지 않는다.

6 Nonmoving 병렬·점진 GC

6.1 수거 방식과 참조 규약

주소 비이동 mark-and-sweep, safepoint 구간의 병렬 marking, 여러 프레임에 분산한 점진 marking을 기본 구현으로 채택한다. 병렬은 여러 GC worker의 동시 처리를 의미하며, 초기 단계에서는 게임의 필드 변경과 GC의 필드 스캔을 동시에 허용하지 않는다. 완전 concurrent marker는 별도의 P3 연구로 분리한다.

참조 타입	도달성·역할	보장
`ObjectRef<T>`	추적된 Object 필드의 강한 edge	setter·복사·이동이 barrier 통과. 소유자가 도달 가능할 때만 대상 유지
`GcArray<ObjectRef<T>>`	강한 edge 집합	삽입·교체·bulk copy·serializer fixup 모두 barrier 경유
`RootRef<T>` / RootToken	native stack·subsystem·비동기 작업의 root	중앙 root registry 등록, 명시적 scope 종료
`WeakObjectRef<T>`	생존을 보장하지 않는 handle	안전 구간의 Pin에서 세대·상태 재검사 후 RootRef 반환
`SoftObjectRef<T>`	AssetId 기반 지연 참조	로딩을 자동 보장하지 않음. LoadTicket으로 명시적 요청
`Borrow<T>` / raw pointer	짧은 동기 접근	GcReadScope 또는 해당 lane 보장 안에서만 유효

표 15

ObjectRef를 일반 stack 변수나 비추적 native Array에 복사하는 것만으로 root가 등록되지 않는다. meta compiler는 추적되지 않는 `Object*` 필드와 `Array<ObjectRef<T>>` 같은 누락 위험 패턴을 오류로 판정한다. 사용자 정의 컨테이너는 TraceReferences와 write barrier 계약을 구현해야 한다.

Epic의 증분 GC 문서는 참조 변경 barrier가 누락되면 조기 수거가 가능하다고 설명한다. 이 계획은 같은 정확성 문제를 방지하는 **자체 collector 제안**이며 UE의 내부 구현을 그대로 재현한다는 의미는 아니다. S1

6.2 GC cycle

RootSnapshot: mutator를 safepoint에 정지시키고 새 epoch와 mark queue를 초기화한 뒤 root registry, published package, task root, editor root를 스캔한다.
ParallelMarkSlice: 모든 Object mutator가 정지한 구간에서 worker가 청크·회색 객체를 분담한다. epoch bitmap의 test-and-set으로 중복 enqueue를 억제한다. 큰 참조 배열은 256개 등 bounded batch로 분할한다.
MutatorResume: worker의 참조 스캔 완료를 barrier로 확인한 뒤 게임 작업을 재개한다. 현재 cycle에서 새 객체는 생존 표시하고 추적 작업에 등록한다.
InsertionBarrier: marking 중 강한 참조 또는 root에 새 대상이 연결되면 대상 handle을 shade queue에 추가한다. 초기안은 소유자의 색과 무관하게 새 대상을 shade하여 보수적으로 보호한다. write는 mutator lane에서만 수행한다.
FinalRemark: safepoint에서 worker·thread-local queue를 flush하고 신규 root·신규 객체·barrier queue를 재스캔한다. 작업이 남으면 다음 slice로 이월하고 sweep을 시작하지 않는다.
WeakProcess / Retire: mark closure를 확인한 safepoint에서 수거 cutoff epoch를 publish한다. 해당 epoch에 미표시된 기존 객체는 실제 slot 전이 전에도 신규 Resolve·pin/root·강한 edge 연결을 즉시 거부한다. 전체 슬롯을 이 정지 구간에서 모두 순회하지 않고 이후 chunk별로 Retiring 전이·sweep한다. 검사·edge/root 등록·cutoff publish는 같은 GC gate로 선형화한다.

BudgetedDestroy / Sweep: reader가 종료된 Game lane 경계에서 BeginDestroy를 예산 내 실행한다. 5.2절의 도달성·cleanup·epoch·backend 조건을 모두 확인한 뒤 메모리와 슬롯을 반환한다.

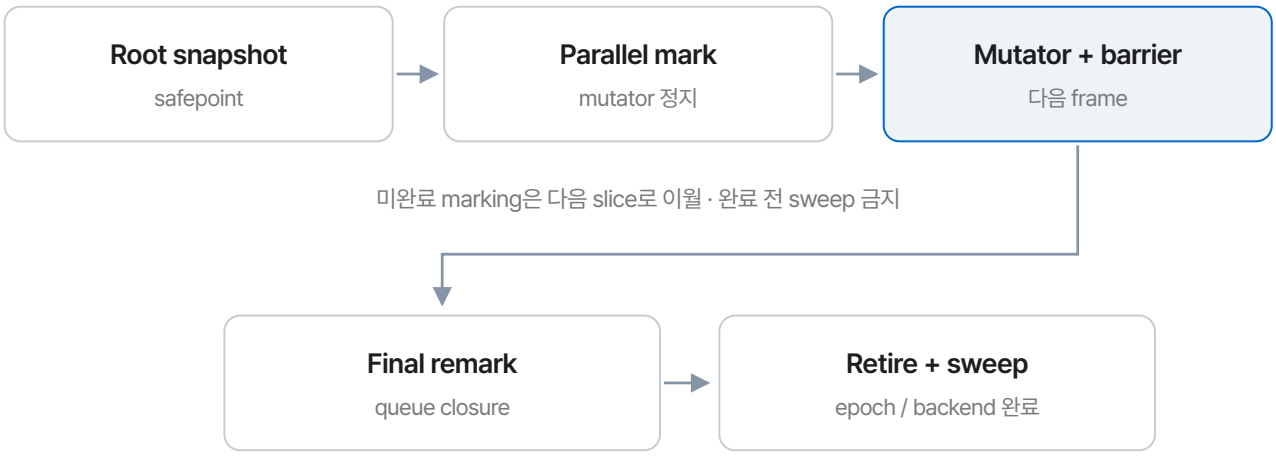


그림 2 · 점진 GC의 안전 구간과 수명 단계 (개념도)

삭제된 edge를 즉시 해석할 필요가 없는 보수적 insertion-barrier 방식을 택한다. 그 결과 해당 cycle에 일시적으로 도달하였던 객체가 다음 cycle까지 잔존할 수 있다. 원시 포인터 저장, barrier 없는 bulk memcpy, GcArray의 내부 buffer 직접 수정은 금지한다. serializer와 undo/redo도 동일 setter 경로를 사용한다.

6.3 동시성의 경계와 예산

병렬 GC worker가 멈춘 Object 필드를 스캔하는 동안에는 physics, render, audio가 자기 snapshot만 처리할 수 있다. Object field를 읽는 작업은 GC-aware lane으로 등록하고 safepoint 전에 완료 또는 협조 정지한다. GcReadScope는 회수 지연 장치이며 root나 필드 lock이 아니다. 최초안의 Borrow는 safepoint 전에 반드시 종료하며 필요한 수명은 RootRef로 유지하고 재개 후 다시 Resolve한다. Borrow를 가진 채 협조 정지하지 않는다. 긴 scope와 무한 작업은 watchdog으로 검출한다.

정지 목표는 mark slice 0.5 ms p99, 전체 safepoint 1.0 ms p99로 제안한다. 이는 **soft budget·미측정 목표**이다. root scan, 큰 객체 visitor, 최종 closure, 소멸 callback이 상한을 초과할 수 있으므로 phase별 최대값과 대기시간도 공개한다. callback에 IO·전역 lock 대기·무제한 하위 작업을 허용하지 않는다. backlog가 증가하면 스트리밍 admission 제한, 일시적 GC 예산 증가, 명시적 loading screen full cycle 순으로 대응한다.

정확성 검증은 정지형 full collector를 oracle로 사용한다. 고정 그래프와 무작위 mutation에서 live set의 누락 0건을 확인한다. 약한 참조 pin/retire 경합, 생성 중 GC, 취소된 로드, 컨테이너 재할당, 세대 wrap, 모듈 종료를 강제한다. 보수적으로 잔존한 객체는 다음 두 번의 무변경 full cycle 이후 해제되는지 검사한다.

6.4 Root·VM·sweep의 종료 조건

RootRef 생성·이동·해제는 thread-safe root registry와 barrier publication을 통과한다. 일반 worker는 Object field mutator가 아니지만 task root의 변경은 기록한다. VM은 safepoint에서 frame/global/coroutine root map을 고정하고 reference slot의 mutation을 정지한다. root provider·visitor는 작은 batch로 재개할 수 있게 구성하며 원자 cutoff 전 마지막 flush와 closure 확인을 수행한다.

새 객체는 현재 cycle의 생존 표시와 outgoing-reference tracing을 함께 수행한다. 초기 field가 이미 black이라는 이유로 child scan을 생략하지 않는다. sweep 도중 생성한 객체는 born epoch로 수거 후보에서 제외한다. barrier queue overflow 시 spill/admission 제한으로 대응하며 handle을 버리지 않는다. 지속적인 mutation으로 cycle이 완료되지 않으면 수거 예산·allocation admission을 조정하고 명시적 full-cycle 구간으로 전환한다.

0.5/1.0 ms는 모든 root 수·callback에 대한 hard deadline이 아니다. root 수, safepoint 협조 지연, remark 종료 비용, live edge 수를 workload에 포함하여 측정한다. 기준 workload에서 실패하면 성능 gate 미통과로 기록하고 root/callback/VM stack budget을 조정한다. 오수거를 허용하여 시간을 충족하지 않는다.

6.5 접근 선형화와 epoch 재사용

IsLogicallyAvailable은 generation·TypeInfo IsA·slot 상태·born/mark/cutoff와 필요한 WorldId/Epoch를 함께 검사한다. Resolve, Weak pin, raw handle에서 만드는 RootRef, ObjectRef의 신규 연결, serializer/Undo/VM store, backend 결과 commit에 같은 판정을 적용한다. 검사와 등록 사이 cutoff가 끼어들지 못하도록 동일 gate를 사용한다. 실패 가능한 신규 setter는 Result를 반환하고 이전 값을 보존한다. 생성·복제의 generated setter도 ConstructionContext 또는 이미 검증된 소유권 경로를 요구한다.

기존 강한 lease의 복사/이동과 신규 handle 승격은 서로 다른 연산이다. 전자는 DestroyRequested 저장소의 보존만 전달하고 일반 메서드 호출을 허용하지 않는다. cutoff로 수거가 확정된 handle을 기존 lease로 위장하여 연결할 수 없다. 같은 owner의 GcArray 재할당·compaction에서 기존 edge를 옮기는 연산은 owner identity를 유지하는 noexcept 보존 이전이다. 다른 owner로 연결하는 공개 copy-assignment와 구분하고 신규 필드 갱신은 owner-aware TrySet으로 제한한다. null edge clear는 cleanup 권한으로 허용한다. cutoff publish 직후·slot 전이 직전 pause fixture에서 모든 acquisition의 거부를 검사하고, live Borrow를 가진 safepoint 진입은 debug 오류로 검출한다.

초기 collector는 active mark/cutoff 분류 cycle을 1개만 허용한다. 모든 cutoff 대상 slot의 판정·Retiring 분류가 끝나기 전에는 bitmap/epoch를 재사용하거나 다음 mark를 시작하지 않는다. 이미 분류된 retirement backlog는 generation이 포함된 불변 record로 관리하여 후속 mark와 공존한다. retire record의 storage/metadata/backend hold는 GC root가 아니며 RootRef self-root로 구현하지 않는다. born epoch·64-bit epoch wrap은 정지·drain 후 재설정하는 정책으로 검증한다.

chunk마다 sweep을 정지하고 allocation·root 변경·후속 cycle 요청·지연 fence를 조합한다. 이전 판정이 새 mark에 덮이거나 retire backlog가 자기 도달성을 유지하지 않아야 한다. safepoint에서 worker를 반환하거나 GC 작업에 참여시켜 정지한 mutator가 worker 전체를 점유하지 않게 한다. 1-worker/2-worker에서도 GC 진행·종료를 검증한다.

7 Clang 메타 컴파일러와 직렬화

7.1 생성 파이프라인

UHT와 유사한 선언형 annotation을 사용하되 C++를 정규식으로 파싱하지 않는다. Host 도구 MetaCompiler가 compile_commands.json의 target·define·include 옵션으로 Clang AST를 구성한다. LibTooling의 AST 접근과 compilation database를 사용한다. S2, S3

```
ROCKS_REFLECT(TypeGuid = "4c76...")

class Character : public Entity {
    ROCKS_GENERATED(Character)
    ROCKS_FIELD(FieldId = 17, Edit, Save, Replicated)
    ObjectRef<MaterialAsset> material_;
    ROCKS_FIELD(FieldId = 18, Edit, Save)
    float speed_ = 6.0f;
    ROCKS_METHOD(Server, Reliable, RpcId = 5)
    void RequestInteract(NetObjectId target);
};
```

입력: target별 compile database, 모듈 manifest, annotation된 헤더, 스키마 이력. **출력:** .generated.h/.generated.cpp, TypeInfo 테이블, GC visitor, serialization field table, RPC validation stub, replication layout, editor JSON schema, 진단 파일.

첫 실행의 generated include 문제는 MetaParse 모드로 해결한다. generated header 자리에 비어 있는 유효한 stub을 생성하고, ROCKS_GENERATED는 AST 수집에 필요한 friend·declaration만 제공한다. 메타 수집 후 원자적으로 실제 파일을 교체하고 C++를 컴파일한다. 생성 결과가 바뀌지 않으면 파일 timestamp를 유지한다. 교차 target마다 ABI 관련 결과를 분리하며 파일 잠금으로 중복 생성을 차단한다.

TypeId·FieldId·RpcId는 선언 이름 hash만으로 결정하지 않는다. 명시 GUID 또는 source control에 저장되는 schema manifest로 안정화한다. 삭제한 필드 번호는 tombstone으로 예약한다. 충돌·중복 ID·상속 field 충돌은 빌드 오류다. 상속은 Object 계열 단일 주상속, reflected field는 지원 타입으로 제한한다. 초기에는 임의 template class, union, bitfield, 다중 Object 상속을 지원하지 않는다.

7.2 런타임 메타와 데이터 진화

RegisterObjectType은 TypeSchema와 중첩 field/name/attribute 배열·문자열·속성 payload를 검증하여 registration 소유의 불변 snapshot으로 깊은 복사한다. 전달한 const 참조·Span·scratch arena를 함수 반환 후 보관하지 않는다. R1 C07은 고정 Guid/POD 배열과 Core의 소유 byte buffer로 필요한 subset을 복사하여 C13 String/Name 도입을 기다리지 않는다. 검증·복사 실패는 등록 전부를 rollback하고 완성된 revision만 원자 게시한다. generated static table도 첫 구현은 같은 복사 경로를 사용한다. 코드 ModuleLease는 metadata 저장소 소유권의 대체물이 아니다. Object·visitor·Archive reader·비동기 schema 소비자는 해당 registration revision lease를 유지하고, 마지막 소비자와 destructor 종료 전 그 revision의 ops/code/data를 회수하지 않는다. 초기 구현은 active TypeId의 비호환 재등록과 제자리 table 교체를 거부하며 unregister는 소비자가 남으면 Busy를 반환한다. native layout 변경은 process restart 기본값을 유지한다.

```
struct FieldInfo {
    FieldId id;
    TypeId valueType;
    FieldFlags flags;
    ReadFieldFn read;
    WriteFieldFn write;
    TraceFieldFn trace;
};

struct TypeInfo {
    TypeId id;
    uint32 schemaVersion;
    Span<const FieldInfo> fields;
    ConstructFn construct;
    DestroyFn destroy;
    MigrateFn migrate;
};
```

비표준 layout Object에 offsetof를 무조건 적용하지 않는다. 생성된 typed accessor와 friend 함수를 사용한다. offset 최적화는 지원 ABI에서 검증된 경우에만 내부적으로 적용한다. 메타 정보는 native vtable 주소나 웹에 전달할 raw pointer를 포함하지 않는다.

Archive는 FieldId, wire type, payload length 기반으로 구성하며 unknown field를 건너뛴다. 패키지 내 객체 참조는 export/import ID, 외부 에셋은 AssetId, 네트워크 객체는 NetObjectId로 저장한다. ObjectHandle은 프로세스 로컬이므로 파일·네트워크에 기록하지 않는다. container 길이·재귀 깊이·압축 해제 크기·문자열 길이를 제한한다. 저장 중에는 snapshot 또는 편집 transaction boundary를 고정한다.

스키마 변경은 rename alias, 기본값, 명시 migration 함수, 삭제 tombstone으로 처리한다. 최소 3개 과거 schema fixture를 CI에서 읽고 다시 저장한다. 메타 compiler 버전, target ABI, compile flag, include closure hash를 cache key에 포함한다. editor와 cooked shipping 스키마는 같은 원본 ID를 유지하되 editor-only 필드를 제거한다.

초기 무재시작 갱신은 자체 언어의 호환 함수·shader·asset으로 한정한다. native C++는 함수 본문만 변경하여도 안전한 code patch·return address·function pointer·metadata versioning이 없으면 프로세스를 재시작한다. layout 변경 native DLL의 즉시 unload는 금지한다. 별도 native live-code 실험은 M7 이후 opt-in 기능이며 기본 반복 개발 SLA에 그 성공을 가정하지 않는다.

8 Task Graph와 스레드 실행 규약

8.1 Scheduler 계약

Task는 선행 작업, 실행 lane, 우선순위, 취소 token, 결과 저장소를 갖는 DAG 노드다. 초기 worker 수는 논리 코어 수에서 Game·Render·IO 여유를 차감하여 결정하며 고정 수치를 가정하지 않는다. 짧은 작업을 batching하고 기본 10–100 µs 수준의 유용한 작업량을 목표로 조정한다. 실제 임계값은 계측으로 확정한다.

```
struct TaskDesc {
    TaskLane lane;
    Priority priority;
    CancelToken cancel;
    Function<void(TaskContext&)> body;
};

Result<TaskHandle> Submit(TaskDesc&&, Span<const TaskHandle> prerequisites);
Result<TaskHandle> Then(TaskHandle, TaskDesc&&);
void Cancel(CancelSource&);
```

Submit은 shutdown/admission·capacity·잘못된 dependency를 명시 오류로 반환한다. 승인된 task만 handle을 반환하며 callable·capture·dependency 목록을 task-owned 저장소로 복사 또는 이동한다. 호출자의 Span·StringView를 나중까지 참조하지 않는다. Object capture는 RootRef 또는 게임 lane에서 검증할 weak handle로 표현한다. completion publication은 release, continuation 관찰은 acquire로 동기화한다. 참조 count가 0이라는 이유만으로 아직 queue에 있는 task node를 재사용하지 않는다.

실행 payload와 완료 관찰 record의 수명을 분리한다. body 반환 또는 queued 실행권 취소가 선형화되고 실행 lease가 0이 된 뒤, callable/capture와 불필요한 prerequisite hold를 지정 cleanup lane에서 정확히 1회 해제한다. callable destructor 반환까지 code ModuleLease를 유지하며, 그 cleanup 완료 후에만 Succeeded/Failed/Cancelled를 release 게시한다. 취소 요청 자체는 terminal이 아니다. TaskHandle이 보존하는 완료 record에는 상태·소유 진단 결과만 남고 실행 capture는 남지 않는다. queue tombstone과 observer가 남은 node의 재사용은 계속 금지한다. IO/GPU/독립 callback으로 이관한 lease는 각 owner가 별도로 drain한다. 현재 void task의 terminal에 명시적으로 이관되지 않은 숨은 RootRef를 보존하지 않는다. RunOnTerminal continuation은 자신의 별도 payload·수명으로 실행하며, 종료 시 terminal publication에 필요한 cleanup lane을 먼저 중지하지 않는다.

Submit admission은 callable을 이동하기 전에 실행 node의 내장 cleanup link/state 또는 해당 lane의 cleanup slot까지 확보한다. 실패 시 caller의 TaskDesc/capture 소유권을 유지하며 유효 handle을 반환하지 않는다. 승인 후 cleanup 전달은 기존 node를 intrusive teardown queue로 이관하여 추가 할당·일반 Submit·별도 task node·사용자 queue 여유를 요구하지 않는다. 따라서 포화나 일반 admission 종료로 cleanup enqueue가 실패하지 않는다. capture destructor는 scheduler/queue/registry gate를 모두 해제한 뒤 지정 lane에서 실행한다. destructor는 noexcept·비차단 계약을 따르며 비동기 후처리가 필요한 자원은 미리 backend lease로 이관한다. shutdown은 이 독립 cleanup queue와 실행 중 destructor까지 drain한 후 lane을 종료한다. CT08은 일반 node pool/queue를 포화시키고 외부 TaskHandle을 유지한 상태에서도 cleanup과 terminal이 진행되는지 확인한다.

Lane	소유 데이터	대기·전달 규약
Game	Object 가변 상태, World, 편집 transaction	프레임 경계에서 결과 commit. 디스크·GPU 동기 wait 금지

Lane	소유 데이터	대기·전달 규약
Worker	분할된 POD, immutable asset·pose	자신의 output 범위만 쓰기. continuation 중심 실행
Render	RenderScene, GPU submission	Game 주소 없음. immutable packet과 resource handle 수신
Physics	해당 tick의 body state	command 적용 → step → snapshot publication
IO	read request와 completion	실제 IO는 OS async 경로. worker를 파일 대기로 점유하지 않음
AudioRT	고정 voice·DSP 상태	동적 할당·lock·IO·일반 scheduler wait 금지
GC	safepoint의 관리 객체 graph	모든 mutator 정지 확인 후 parallel visitor 실행

표 16

대기는 worker help-execute 또는 continuation으로 처리한다. Game/Render의 상호 동기 wait와 lock 보유 상태의 task wait는 금지한다. lane별 lock rank를 정하고 debug에서 검사한다. work-stealing은 한 worker 소유 deque와 steal 경합으로 구현하되, 초기 정확성 버전과 결과·수명·stress를 대조한다. native Fiber/C++ coroutine은 측정 후 추가하며, M5의 script async는 VM 소유 resume frame으로 먼저 구현한다. 두 기능의 도입 시점을 혼동하지 않는다.

취소는 cooperative이며 이미 수행한 side effect를 자동 rollback하지 않는다. task는 cancel 확인 후 자신의 임시 메모리를 정리하고 terminal 상태를 정확히 한 번 publish한다. Shutdown은 일반 Submit을 차단하고 teardown capability를 가진 정리 작업만 완료 drain까지 허용한다. scheduler·Render/Audio/Physics 소유자는 그 뒤에 종료한다. Module unload는 해당 모듈 코드를 실행하는 task, callback, timer가 0개인 상태에서만 허용한다.

prerequisite 실패·취소는 기본적으로 dependent body를 실행하지 않고 terminal 결과로 전파한다. cleanup continuation은 RunOnTerminal 정책을 명시하여 선행 작업의 성공·실패를 처리한다. teardown capability는 임의 게임 task 제출 권한이 아니며 owner와 작업 종류를 제한한다. CF05에 admission 거부, 이미 완료된 prerequisite, 취소 전파, cleanup continuation, 다중 terminal 경쟁을 포함한다.

8.2 한 프레임의 데이터 전달

입력 수집 → 네트워크 수신 적용 → 게임/AI 의도 → 애니메이션·kinematic 목표 → fixed physics → 결과·이벤트 반영 → transform hierarchy 확정 → render/audio/VFX snapshot 추출 순서를 기본으로 한다. 별도 frame에서 병렬 실행하는 경우에도 tick 번호와 snapshot version으로 동일 선행 관계를 표현한다.

Render는 2–3개 in-flight frame slot을 사용하되 slot 재사용은 해당 slot의 모든 queue fence 완료로 판정한다. 단순히 '3프레임 경과'로 회수하지 않는다. 게임이 과도하게 선행하면 bounded queue가 backpressure를 제공한다. streaming, shader compile, GC, AI는 별도 budget과 priority aging을 사용하여 한 종류가 worker 전체를 점유하지 못하게 한다.

8.3 가변·고정 tick과 시간 도메인

Rocks는 가변 tick·고정 tick·time dilation·pause/resume·single-step을 기본 기능으로 제공한다. ClockService와 StepPlanner는 Core의 플랫폼 독립 값 계산이며 World/PIE/physics는 상위에서 조립한다. MonotonicClock은 실제 시간, FrameClock은 frame delta, GameClock은 pause/dilation 적용 시간, FixedClock은 정수 tick index와 고정 step, UIClock·network timeout·작업 deadline은 unscaled 시간을 기본으로 한다. 각 timer·animation·sequence·delay는 사용할 clock을 명시한다.

가변 Update는 frame당 한 번 게임 delta를 받는다. 고정 Update는 scaled accumulator에 realDelta×gameScale을 더하고 고정 h씩 소비한다. time dilation은 기본안에서 h 자체를 바꾸지 않고 고정 step의 실행 빈도를 조절한다. 같은 값을 delta와 accumulator에 중복 적용하지 않는다. 렌더 보간은 이전/현재 고정 snapshot과 accumulator/h를 사용하고 tick 경계를 넘는 mutable pointer를 보관하지 않는다. fixed/variable 두 group 등록은 명시 선택이며 동일 gameplay 동작이 두 번 실행되지 않게 한다.

scale은 finite·0 이상으로 제한하고 0은 game pause다. 역재생은 음수 delta가 아닌 sequence/replay seek 경로를 사용한다. scale 변경은 tick 경계에서 적용하고 replay/network에는 clock epoch·변경 tick·값을 기록한다. object/local scale은 animation·선택 gameplay clock에 우선 적용하며 공유 physics solver의 body마다 다른 step을 임의 적용하지 않는다. solver·network clock과 다른 의미는 별도 기능으로 검증한다.

긴 frame에는 max frame delta·max substeps·누적 debt 한도를 선언한다. local interactive 모드는 명시 정책에 따라 표시 frame을 늦추거나 잔여 debt를 제한하고 버린 simulation 시간을 report한다. authoritative server는 처리하지 않은 tick을 처리된 것으로 표시하지 않으며 overload/backpressure·진척 저하를 기록한다. pause 중에도 UI/입력 pump·IO·trace·network timeout은 동작한다. resume 시 과거 입력 edge를 재발사하지 않는다. 고정 단일 step은 pause 상태에서 정확히 h 한 번을 실행하며 가변 step은 명시된 debug delta를 사용한다.

```
struct TimeStepPolicy {
    double fixedStepSeconds;
    double maxFrameDeltaSeconds;
    uint32 maxSubsteps;
    TimeDebtPolicy debtPolicy;
};

struct TickContext {
    uint64 tickIndex;
    double deltaSeconds;
    double unscaledDeltaSeconds;
    ClockId clock;
    uint64 clockEpoch;
};

Result<StepPlan> PlanSteps(ClockState&, const TimeStepPolicy&, double realDeltaSeconds);
```

simulation clock state는 owner lane에서 commit한다. worker는 immutable TickContext를 받으며 완료 시 owner가 tick/World epoch를 검증한다. pause·dilation이 timeout·task cancellation을 막지 않는다. simulation 모드에서는 wall clock을 기다리지 않고 고정 tick을 가능한 속도로 진행할 수 있다.

8.4 안정성을 우선한 병렬 tick

TickGraph는 phase·명시 prerequisite·read/write set·소유 context로 DAG를 구성한다. 초기 Object field mutator는 Game lane 하나이며 native SoA·animation pose·culling·physics/nav query·stream decode·render packet 생성 등 독립 계산을 worker로 분배한다. 결과는 command buffer/immutable snapshot으로 반환하고 정해진 commit phase에서 적용한다. 일반 worker가 Object의 mutable field를 임의 변경하지 않는다.

physics 내부 병렬화와 외부 gameplay mutation을 구분하고, read phase가 끝나기 전에 구조를 변경하지 않는다. job 취소·지연 결과는 tick/context generation을 검사한다. 장치 API 호출은 backend thread/context 제약을 지키며 D3D11/OpenGL의 제한을 이유로 scene 준비·애니메이션까지 단일 thread로 고정하지 않는다. workers 수와 lane을 줄여도 진행 가능한 순차 기준 실행기를 유지한다.

결정성이 필요한 작업은 task 완료 순서 대신 stable entity/task key로 merge하고 random stream을 tick/entity에 연결한다. 부동소수점 합산 순서·physics backend 차이의 결정성 범위를 명시한다. lock-free·공유 mutable 구조로 전환하기 전에 sequential oracle·race stress·소유권·취소·GC safepoint 검증을 통과한다.

ID	시간·병렬 실행 검증
TK01	30/60/144 FPS 입력에서 고정 tick 수·변위·보간, variable delta 합산 기준 일치
TK02	scale 0/0.1/1/2·pause/resume·변경 tick·single-step·NaN/음수 거부
TK03	hitch·clock discontinuity·max substeps·debt 제한·server overload의 진척/손실 표시
TK04	UI/IO/timeout의 unscaled 동작과 game pause 분리, old input edge 재발사 0
TK05	0/1/N worker·순서 교란·취소·World 교체·GC 경계에서 동일 의미 결과·stale commit 0
TK06	reader 종료/command commit·DAG cycle·read/write 충돌·shutdown 경합과 trace 확인

표 17

Core C02/C15에서 clock/step 값을, C17/C26에서 Task·replay fixture를 검증한다. 실제 World/physics/PIE 연결은 M3/M4 필수다. 멀티스레딩의 처리량 향상은 실측하고 정확성·진척·누수 기준을 완화하지 않는다.

9 에셋 파이프라인·쿠킹·DDC·비동기 로딩

9.1 식별자와 파생 데이터

AssetId는 위치와 독립된 128-bit GUID로 부여한다. source asset, importer setting, stable type schema, platform profile, toolchain version의 관계를 AssetRegistry에 기록한다. 파일 rename·move는 ID를 보존하고, 복제는 새 ID를 부여한다. package manifest에는 hard dependency와 soft reference를 구분한다.

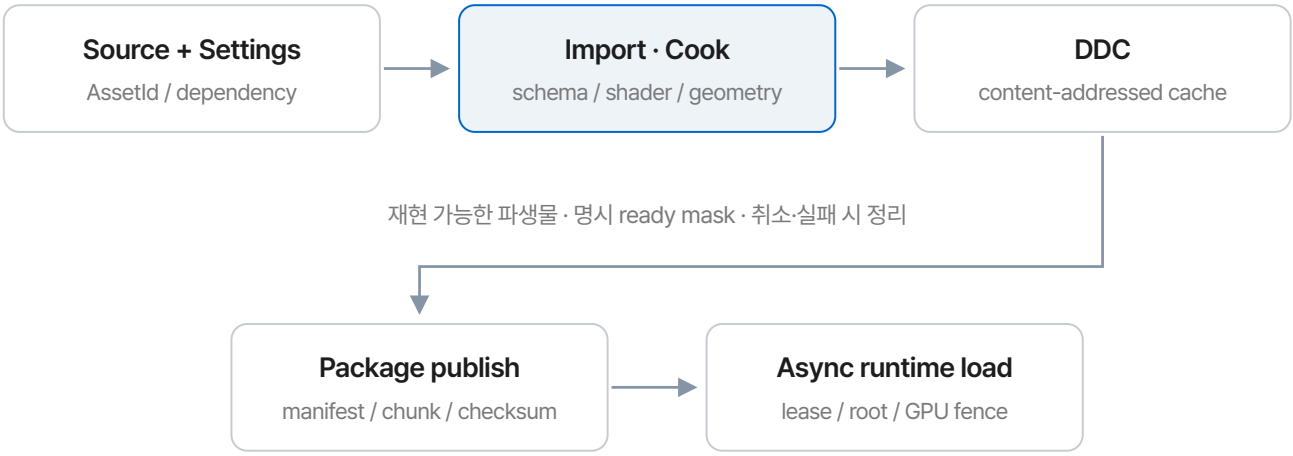


그림 3 · 에셋 파생 데이터와 runtime 로딩 (개념도)

DDC key는 source bytes hash + importer/version + canonical setting + dependency content hashes + target/feature profile + compiler/schema version으로 구성한다. 키만 있으면 재현 가능한 결과를 얻도록 경로·timestamp·locale을 제거한다. 순서 의존 출력은 canonical 정렬하고 난수 seed를 기록한다. 로컬 cache를 먼저 조회하고 공유 cache는 checksum을 검증한 뒤 사용한다. DDC는 삭제·재생성 가능한 파생 cache이며 원본 에셋 저장소가 아니다.

runtime reference graph와 cook build-dependency graph를 분리한다. 단순 AssetId 참조는 상대 cooked payload의 완료를 자동 선행 조건으로 만들지 않는다. 실제 산출물 의존이 순환하면 지원 가능한 SCC를 하나의 cook 단위로 축약하고, 정렬된 구성원 source/settings/schema·내부 edge ID·외부 component의 확정 digest로 key를 계산한다. 구성원 output은 component key+local output ID에서 파생하며 서로의 cooked hash를 재귀 정의하지 않는다. 지원하지 않는 순환 변환은 cycle 경로와 함께 거부한다. A↔B·3-node cycle 및 외부 변경 fixture에서 clean/incremental/local/remote 결과를 대조한다.

Importer는 glTF·텍스처·WAV/압축 오디오를 우선하고, FBX와 고급 DCC 연동은 plugin으로 확장한다. 임포트는 별도 worker process에서 수행하여 실패와 긴 컴파일을 격리한다. StaticMesh cook은 conventional LOD·meshlet·VirtualGeometry page·collision·nav source·GI proxy를 독립 산출물로 생성한다. 하나의 렌더링 메시를 모든 시스템의 직접 입력으로 재사용하지 않는다.

9.2 로딩 상태와 취소

Requested → ResolveManifest → IO → Verify/Decompress → DeserializeNative → ConstructObjects → Fixup → PostLoad → Upload/BackendReady → Ready로 구성한다. deserialize·압축 해제는 worker에서, Object 생성·추적 edge 연결·gameplay PostLoad는 Game lane에서 수행하는 기본안이다. GPU upload는 Render lane으로 전달한다.

```
struct AssetRequest {
    AssetId id;
    LoadPriority priority;
    ReadyMask required;
    CancelToken cancel;
};

Result<LoadTicket> RequestAsset(AssetRequest);
Result<void> OnReady(LoadTicket, TaskLane, Function<void(Result<AssetLease>)>&& callback);
```

RequestAsset 실패에는 유효 ticket이나 caller 관심이 없다. OnReady는 LoadTicket당 callback 1개를 허용하며 중복 등록은 AlreadyRegistered다. 성공 전에 callback record·지정 lane의 terminal 전달 slot을 예약하고, 실패 시 callback은 미등록·미실행이며 Function 소유권은 caller에 유지한다. 성공 후에는 loader가 callback/capture/code lease를 호출과 capture cleanup 종료까지 소유한다. callback 전달은 추가 일반 Submit·allocation·일반 queue 여유에 의존하지 않으며, LoadTicket의 반환·장기 보관은 accepted callback의 유실이나 완료 payload의 장기 보존을 유발하지 않는다.

scope/package gate 안에서는 delivery admission·결과 record·callback execution retention만 확정한다. 모든 gate를 해제한 뒤 지정 lane에서 사용자 callback과 capture destructor를 실행한다. 사용자 callback 안에서 같은 package를 요청하거나 lease를 복사할 수 있으나, 자기 scope Stop은 비동기 요청만 허용하며 자기 callback의 종료를 동기 wait하지 않는다. revoke는 lock 밖에서 accepted callback과 cleanup을 비동기 drain한다. revoke가 delivery admission보다 먼저이면 ScopeClosed terminal을 1회 전달한다. delivery admission이 먼저이면 callback 반환까지 execution retention으로 보호하지만, 그 뒤 revoke가 발생하면 새 asset 접근과 gameplay commit은 scope를 다시 검사하여 거부한다. callback 반환·capture cleanup 뒤 execution retention을 정확히 1회 해제하며, gate를 보유한 채 사용자 코드를 호출하는 구현은 금지한다.

LoadTicket은 요청 관심을 나타내며 AssetLease는 실제 resident 수명을 유지한다. 같은 package의 동시 요청은 하나의 로드 작업에 합류한다. 한 caller의 취소가 다른 caller의 공유 요청까지 취소하지 않는다. callback은 caller별 정확히 한 번 terminal 결과를 수신한다. Ready는 CPU 객체·render·physics 등 요청한 mask를 충족한 상태로 정의한다. 임시 fallback texture의 표시를 완전한 ready로 보고하지 않는다.

순환 참조 package는 dependency graph의 강연결 성분으로 묶어 allocate-all → deserialize-all → fixup-all → publish 순서로 처리한다. 다른 package의 PostLoad를 동기 wait하지 않는다. 로드 중 root set을 유지하며 취소·오류 시 생성된 객체와 backend 자원을 각자의 안전 시점에 정리한다. rename redirect는 기간과 만료 정책을 가진다.

LoadTicket의 request interest와 AssetLease의 residency retention을 별도 계수로 관리한다. 성공 lease는 package revision의 resident record·필요 관리 graph root·선언된 ReadyMask와 최소 residency를 release 또는 명시 scope revoke까지 보존한다. 추가 texture mip/page pin은 별도 계약이다. 정상 unload는 request interest와 발급·진행 중 사용의 residency retention이 모두 0일 때만 Resident→Closing으로 전이한다. lease 발급·승격·복사 admission과 이 전이는 같은 package gate에서 선형화한다. 발급이 먼저이면 unload가 대기하고 Closing이 먼저이면 새 발급은 실패한다. ticket의 취소·반환은 이미 성공 발급된 lease를 묵시 취소하지 않는다. 공유 asset graph의 논리 파괴 권한은 loader의 이 admission 계약을 거치며 일반 사용자에게 임의 파괴 권한을 주지 않는다.

PIE/World 종료의 명시 scope revoke는 정상 cache eviction과 분리한다. revoke는 해당 scope/generation의 신규 접근·발급·결과 commit을 차단하며 일반 접근에 ScopeClosed를 반환한다. 같은 immutable revision을 사용하는 다른 scope의 lease는 유지한다. 이미 허용된 짧은 접근·callback·추출 snapshot·IO/upload/recording은 이관된 storage/backend lease로 실제 완료까지 보호한다. revoked scope의 retention은 마지막 허용 사용 종료 후 정확히 1회 반환하고, 외부에 남은 revoked token은 상태 조회만 허용하는 tombstone으로 전환한다. token의 뒤늦은 파괴가 retention을 중복 반환하지 않는다. revoke와 성공 completion은 scope admission 결정 후 package gate 순서로 직렬화하며 반대 lock 순서를 금지한다. 취소/revoke 자체를 IO/GPU 완료로 간주하지 않는다. package retire는 다른 사용자와 모든 retention 종료를 다시 확인한다.

Closing 이후 subsystem detach → package RootRef 해제 → 논리 수거/retire → backend 완료 → 물리 회수 순서를 따른다. AssetId가 존재한다는 사실과 resident asset이 존재한다는 사실을 혼동하지 않는다. texture mip, geometry page, audio chunk는 Object 수명과 별도의 streaming residency로 제어한다.

NL03/NL04·PI05의 asset 통합 부속 사례는 성공 직후 마지막 ticket 반환과 lease 단독 다음 프레임 사용, interest 0과 lease 발급 경합, 공유 A/B 중 A revoke와 B 사용, revoke와 completion/GC/late Submit 경합을 검사한다. 정상 lease의 조기 무효화·타 scope 취소·stale commit·완료 전 회수·retention 이중 반환은 0건이어야 한다. 마지막 lease와 작업 종료 뒤 full GC 2회 및 backend drain에서 package graph/root/retention이 기준선으로 돌아와야 하며, 보관 중인 revoked token도 검사한다. C24 fake owner 시험과 M3 이후 실제 loader/RHI 시험 결과를 분리한다. callback 내부 같은 package 재요청·lease 복사·자기 Stop, 등록 OOM/queue 포화, delivery admission 전후 revoke를 추가하여 교착·accepted callback 유실·terminal 중복·capture 잔존이 0건인지 검사한다.

9.3 패키지와 패치

Package는 header/version, export/import table, dependency manifest, chunk table, codec, checksum, bulk payload를 갖는다. mmap 가능한 불변 데이터와 고정·변환이 필요한 데이터를 분리한다. atomic rename으로 cook output을 publish하고 중단된 임시 파일은 유효 산출물로 인식하지 않는다. compression·IO block 크기는 SSD 접근 패턴과 임계 메모리로 측정한다.

배포 manifest는 콘텐츠 hash와 명시 버전을 사용한다. 패치는 변경 chunk만 교체하고 구 manifest로 rollback할 수 있어야 한다. 잘린 파일, checksum 오류, 해제 크기 폭증, dependency 누락을 오류 주입으로 검증한다. 최종 패키지는 editor, importer, LLVM, 웹 개발 서버, 원본 source asset 없이 실행할 수 있어야 한다.

9.4 Virtual Asset·원본 bulk·재임포트

원본 metadata와 대형 bulk payload를 분리하여 작업자가 필요한 payload만 가져오는 Virtual Asset 흐름을 추가한다. 원본 bulk store는 authoritative·내구성 보관소이며 재생성 가능한 DDC와 분리한다. 원본 payload가 DDC에만 존재하는 구성은 허용하지 않는다. cook은 오프라인 가용성·누락 payload·hash를 사전 검증한다. 작업 흐름의 참고 범위는 Unreal Virtual Assets다. S30

자동 재임포트는 source watcher event를 debounce하고 source revision·import setting·local override를 비교한다. 임포트 오류 시 이전 정상 asset을 유지한다. metadata-only 조회·thumbnail·dependency audit은 bulk 전체 로드를 요구하지 않는다. branch 전환·asset move·source control lock·대형 binary diff를 editor에 연결한다.

ReadyMask는 target capability의 부분집합이어야 한다. DedicatedServer cook은 Gameplay/Collision/Nav bundle만 포함하고 GPU 준비를 요구하는 요청은 시작 전에 UnsupportedCapability로 실패한다. OnReady는 Game/Worker/Render completion lane만 허용한다. AudioRT·GC lane에서 일반 callback을 실행하지 않고 해당 subsystem의 bounded command로 변환한다.

9.5 필수 Texture Streaming

Rocks는 **일반 mip streaming을 M3/M4 필수 기능**으로 구현한다. M3에 mip cook·비동기 IO·상주 교체, M4에 다중 view·예산·취소·회귀를 완료한다. VirtualTexture의 tile/page streaming은 후속이다. Core/Object는 `rocks::AssetId` ·lease·취소·세대 계약과 CPU fake만 구현하며 실제 GPU는 RHI 착수 이후 연결한다.

cooker는 dimension·format·색공간·mip 수·크기/offset/hash·압축 단위·resident tail과 색/normal/mask 필터·alpha coverage를 기록한다. UV density·material UV scale·bounds·view 거리/투영·render resolution으로 desired mip을 산정한다. 동적 UV는 명시 scale/bias와 보수적 상한을 사용한다. editor/game/capture/split-screen 수요는 priority·hysteresis로 조정한다. 공개 개요를 참고한 자체 설계다. S65

GPU 상주·pin 예약분, IO/decode staging, upload bandwidth·동시 요청, 구/신 allocation의 **temporary peak**를 각각 제한한다. sampler LOD clamp를 메모리 회수로 집계하지 않는다. reduced-mip allocation 교체 또는 backend residency를 사용하고 minLOD/SRV·descriptor snapshot을 frame 경계에서 갱신한다. IO 완료만으로 GPU resident를 선언하지 않는다.

요청은 asset revision·device generation·request generation으로 식별한다. 취소 뒤 늦은 IO/decode/upload는 현재 요청과 대조하고 callback·staging은 실제 완료까지 보존한다. 제출된 upload는 취소만으로 반환하지 않는다. 신규 recording lease 차단·기존 lease 종료·마지막 copy/graphics/compute fence 완료 뒤 구 texture·descriptor·upload 범위를 회수한다.

활성 texture의 최소 tail은 pin한다. 실패 시 tail 또는 typed builtin fallback을 표시하되 TargetReady와 구분하고 오류를 보존한다. pin은 owner·목표 mip·기한·budget admission을 요구하며 예산 초과는 실패/명시 품질 조정으로 처리한다. 시네마틱/offscreen은 shot look-ahead로 요구 mip의 GPU 준비를 기다린다. timeout을 품질 충족으로 보고하지 않고 job에 실제 residency를 기록한다.

디버그는 desired/budgeted/resident mip·UV 오류·IO/upload 대기·evict/pin 사유·peak·fallback을 표시한다.

ID	수용 시험·판정
TX01	cook/hash·색/normal/alpha 기준 일치, 손상 mip 실패
TX02	다중 view·이동·해상도·UV 변경의 desired mip 기준 일치
TX03	pin/요청 포화 시 상주·temporary peak 준수
TX04	IO/decode/upload 취소·revision/device 교체의 stale publish 0
TX05	늦은 submit·eviction·작은 ring에서 lease/fence 전 재사용 0
TX06	cold/누락/폭주 시 fallback 유지, TargetReady 오표시 0
TX07	Deferred/Forward·upscale·컷씬/offscreen 준비 gate 회귀
TX08	이동/언로드 누수 0, mip 진동·IO·CPU/GPU 비용 계측

표 18

Texture binding snapshot은 logicalExtent·logicalMipCount·residentBaseMip·residentMipCount와 descriptor를 같은 generation으로 게시한다. 일반 dimension query는 원본 논리 크기를 반환하고 explicit LOD는 논리 resident 구간 clamp 후 물리 base를 변환한다. implicit derivative sampling도 같은 이용 가능한 mip의 의미를 보존한다. integer Load/texel 접근은 논리 좌표를 변환하거나 full-resident 전용으로 선언하고 진단한다. 모든 sampling에 일률 bias를 적용하지 않는다. TX02/TX05/API02는 mip별 색·비정방형 texture·명시 LOD·dimension 기반 UV/WPO의 승격/축소 및 metadata/descriptor 동시 게시를 검사한다.

예산 포화에서도 축소가 진행되도록 일반 admission이 사용하지 않는 전환 여유분을 둔다. 여유분도 사용할 수 없으면 이미 resident인 typed builtin으로 교체하고 구 recording/fence 종료 후 backing을 해제하여 tail을 재생성한다. pinned/cinematic 요구의 품질 저하·실패를 노출하고 무한 retry하지 않는다. TX03은 cap 100%·budget 강제 감소·기존 recording 유지·allocation 실패에서 상한 준수와 fence 해소 후 eviction 진척/명시 종료를 검사한다.

9.6 공용 내장 리소스와 초기화

Builtin Resource Pack을 배포한다. `rocks::BuiltinId` 는 stable 의미/schema, `rocks::TextureHandle` 은 device/slot generation이다. GPU handle은 직렬화하지 않는다.

white/black/transparent·flat-normal·checker/error texture, error material, 기본 cube/plane mesh·기본 mesh 렌더링용 fallback vertex/pixel shader 또는 backend 동등 단계·UI font, zero/identity buffer를 포함한다. white/black의 alpha와 transparent의 zero alpha, normal encoding별 중립값을 정의한다. identity buffer는 matrix/skin palette의 element 의미·stride·count를 명시하며 zero buffer로 대체하지 않는다.

각 ID는 2D/cube/array/3D dimension·extent·format·sample type·색공간·view 용도를 가진다. 2D white를 cube/uint/depth slot에 대입하지 않는다. typed variant를 등록하고 미지원은 오류로 반환한다. color와 data/normal의 sRGB를 구분한다. error material/fallback shader는 두 경로의 기본 mesh profile을 검증하고 고급 domain 미지원은 공개한다.

초기화는 manifest 검증 → DeviceCaps → 최소 fallback 생성/upload → fence 확인 → BuiltinReady → 일반 asset/preview 활성화 순서다. 최소 fallback은 일반 material compiler·asset streaming에 의존하지 않는다. 동시 최초 요청은 device generation별 1회 생성에 합류한다. 생성 실패는 Ready로 publish하지 않고 target 초기화를 실패시킨다. 이후 load 실패는 typed fallback과 오류를 함께 보고한다.

내장 GPU 자원은 device 수명 동안 pin하여 streaming 예산에 별도 예약한다. shader/mesh/font atlas의 residency·성장 상한도 계상한다. project unload로 해제하지 않으며 shutdown은 recording lease·GPU fence 후 정리한다. DeviceLost는 구 handle을 무효화하고 복구 device에 재생성한다. 불확실한 완료로 정상 pool에 반환하지 않는다.

manifest는 ID/schema·target/profile·hash·cooked shader variant·출처/라이선스/고지를 포함하며 font 재배포 조건도 검토한다. CPU-only/서버는 필요한 CPU data만 포함하고 GPU·font renderer를 링크하지 않는다. GPU 요청은 UnsupportedCapability이며 가짜 handle을 만들지 않는다. GPU offscreen은 실제 BuiltinReady를 검사한다. DDC·네트워크·프로젝트 없는 설치에서도 최소 자원을 제공한다.

ID	수용 시험·판정
BR01	빈 프로젝트·DDC 없음·오프라인에서 pack만으로 초기화
BR02	dimension/format/색공간/stride mismatch 거부, 기준 영상/값 일치
BR03	최초 요청 경합/실패: generation별 생성 1회, 실패 Ready 0
BR04	unload·늦은 submit·device loss에서 stale handle·조기 free 0
BR05	target별 cook closure/hash/고지 검증, CPU-only GPU 의존 0

표 19

10 Scene Graph·World·World Partition

10.1 계층과 대량 처리 데이터

World는 Entity/Component의 생명주기와 subsystem을 소유한다. Scene Graph는 parent·children·local/world transform·dirty version만 관리하는 경량 native 구조이며 Entity와 일대일 대응을 강제하지 않는다. render/physics/audio용 handle은 각 시스템의 슬롯을 가리킨다. handle 종류를 암묵 변환하지 않는다.

```
struct SceneNodeHandle {
    uint32 index;
    uint32 generation;
};

struct TransformCommand {
    SceneNodeHandle node;
    Transform local;
    uint64 version;
};

class World {
public:
    Result<RootRef<Entity>> Spawn(TypeId, SpawnDesc);
    void EnqueueDestroy(ObjectHandle);
    void Tick(FixedTickContext&);
    WorldSnapshot ExtractSnapshot(FrameId);
};
```

reparent는 cycle 검사·world transform 유지 옵션·하위 dirty 전파를 transaction으로 처리하며 2.3절의 affine/TRS 표현 가능성 검사 실패 시 계층 변경을 rollback한다. parent가 변경된 level을 topological order로 계산하고 같은 depth의 독립 subtree를 병렬 처리한다. physics-driven transform과 animation root motion이 같은 tick에 서로 덮어쓰지 않도록 authority를 별도 필드로 유지한다. 프레임 도중 구조 변경은 command buffer에 기록하여 commit 경계에 적용한다.

논리 Entity와 시스템별 SoA 데이터를 병행한다. ECS로 전체 Object 모델을 즉시 대체하지 않는다. 다량 NPC·foliage·particle에 필요한 chunk iteration은 독립 데이터 모듈로 구현한다. save game은 transient runtime handle을 저장하지 않고 stable actor ID·asset ID·게임 schema를 사용한다.

10.2 셀 스트리밍과 큰 월드

World Partition은 actor descriptor, grid cell, data layer, streaming source, HLOD, persistent actor로 구성한다. 초기 cell edge 256 m를 제안하되 실제 geometry·collision·AI·IO 비용으로 조정한다. actor는 stable GUID와 canonical owning cell을 갖고, 여러 cell에 걸친 bounds는 검색 인덱스에 등록한다. 큰 landmark와 전역 시스템은 별도 persistent layer를 사용한다.

셀 상태는 Unloaded → Requested → CPUReady → SystemsReady → Activated → Deactivating → Unloaded이다. activation은 render·collision·navigation의 필수 준비 mask를 만족한 tick 경계에서 수행한다. collision이 준비되지 않은 공간으로 캐릭터가 진입하면 이동 제한·선행 로드·loading gate 중 프로젝트가 선택한 정책을 적용한다. 화면에 보인다는 이유만으로 gameplay ready를 선언하지 않는다.

셀 간 영속 관계는 SoftEntityId로 유지하고 resident 시점에 weak resolve한다. Object의 강한 상호 참조가 전체 월드를 상주시킬 수 있으므로 cell boundary를 넘는 hard edge는 cooker가 진단한다. 꼭 필요한 의존성은 함께 활성화할 묶음으로 명시하고 최대 묶음 크기를 검사한다.

우선순위는 거리·이동 속도·시야·게임 중요도·네트워크 relevance를 반영한다. unload hysteresis와 최소 residence time으로 진동을 방지한다. CPU/GPU/physics/nav residency를 각각 계측하고 통합 상한으로 admission을 제어한다. teleport는 destination prefetch와 commit gate를 사용한다. 카메라 상대 render origin, 물리 지역 원점, audio listener-relative 위치, network quantization origin에 같은 world epoch를 기록한다.

HLOD·terrain tile·foliage instance·nav tile·GI proxy는 동일 cell manifest에 연결하되 서로 다른 streaming level을 허용한다. HLOD 재생성과 cell recook 범위는 DDC dependency graph로 추적한다. editor의 one-actor-per-file 저장, data layer preview, cell lock·dirty 상태는 P2에서 추가한다.

10.3 맵 이동·저장·런타임 공간 서비스

World travel은 PrepareDestination → Prefetch → QuiesceOldWorld → TransferState → ActivateNewWorld → RetireOldWorld 순서다. seamless travel에서는 connection·account·PlayerState 중 명시된 항목만 유지하고 WorldId/Epoch를 변경한다. persistent service가 이전 World의 Entity/Component를 강하게 보관하지 않게 검사한다. 일반 cell unload와 whole-world travel을 서로 다른 operation으로 정의한다. S29

SaveGame은 map manifest·stable actor/prefab IDs·spawn/despawn tombstone·script schema·game state를 snapshot으로 저장한다. 읽을 때 전부 instantiate → remap → state apply → register 순서를 수행한다. async save·incremental save·save slot·version migration·충돌 복구를 지원하며 network replay와 파일 포맷을 분리한다.

공통 spatial query는 overlap/trigger volume·zone·team/tag query·significance를 제공한다. render BVH·physics broadphase·nav index를 하나의 구조로 강제하지 않고 각 결과를 WorldId·tick/version으로 정합한다. multi-camera·spectator·split-screen의 streaming source와 listener는 LocalPlayer별로 관리한다.

AAA01/AAA02의 지속 월드 상태·PCG provenance와 AAA08의 이동 환경 snapshot은 이 World/partition 규약을 확장한다. 구체 revision·실패·하네스 기준은 33.6절을 적용한다.

11 렌더러 기반과 Deferred·Forward 정식 지원

11.1 지원 계약

Deferred와 Forward+를 모두 불투명·반투명·skeletal mesh·조명·그림자를 지원하는 정식 렌더링 경로로 제공한다. Deferred의 반투명 pass만 Forward로 처리하는 것은 이 요구의 완료가 아니다. 프로젝트 기본값과 개별 ViewDesc에서 경로를 선택한다. 같은 World/session을 보는 복수 view는 서로 다른 경로로 동일 RenderScene snapshot을 사용할 수 있다. EditorWorld와 각 PIE는 별도 논리 RenderScene 또는 SessionId/WorldEpoch별 가시성·가변 proxy 격리를 갖는다.

```
enum class RenderPath : uint8 {
    Deferred,
    ForwardPlus
};
enum class UpscaleMode : uint8 {
    Native,
    Spatial,
    Temporal,
    FSR,
    DLSS,
    XeSS
};

struct ViewDesc {
    ViewId id;
    RenderPath path;
    Extent2D renderSize;
    Extent2D outputSize;
    uint32 sampleCount;
    UpscaleMode upscale;
    CameraSnapshot camera;
};
```

항목	Deferred	Forward+
불투명 shading	depth·GBuffer 후 조명 평가	depth·light cluster 후 material draw에서 조명 평가
머티리얼	공통 Material IR에서 GBuffer 출력 생성	같은 IR에서 direct lighting·surface 평가 생성
광원	clustered/tiled light list를 deferred pass에서 소비	같은 light list를 opaque/transparent shader에서 소비
그림자·IBL·fog·decal	공통 shadow/IBL/fog. GBuffer decal 지원	공통 shadow/IBL/fog. mesh decal 또는 pre-shading decal
AA	TAA·temporal upscaling, 필요 시 spatial AA	TAA·temporal upscaling 및 MSAA 1/2/4x. 초기 MSAA 기준 4x
screen-space 효과	GBuffer normal/roughness 활용	필요한 depth/normal/roughness 보조 출력 선언
반투명	공유 Forward transparent pass	동일 transparent pass
VirtualGeometry	visibility 기반 shading 또는 fallback mesh	단계별 direct shade 통합, 미지원 조합은 cooked conventional LOD
경로 전환	View별 PSO·resource layout·history 재구성	동일. 기존 history 무조건 재사용 금지

표 20

양 경로에서 필수 지원하는 공통 표면 모델은 opaque, masked, unlit, metallic/roughness PBR, normal map, emissive, two-sided, skinning, instancing이다. clear-coat·skin/SSS·cloth·eye·foliage·thin transmission은 M6–M8의 고급 공통 재질 범위다. strand hair·임의 layered closure는 P3이며 지원 profile과 별도 gate를 갖는다. 기능별 지원표를 editor와 cooker가 조회한다. 지원하지 않는 기능은 경고와 명시적인 fallback 또는 cook 오류로 처리하며, 검은 표면으로 처리하지 않는다.

Forward는 MSAA를 사용하지 않아도 완전한 경로여야 한다. MSAA는 비용 선택지이고 temporal upscaling의 필수 조건이 아니다. temporal upscaling과 MSAA를 동시에 선택하면 색·깊이·velocity·coverage를 단일 sample 입력으로 변환하는 검증된 resolve pass를 사용한다. 깊이와 motion vector를 단순 평균하지 않고 전경 sample 선택과 coverage 규칙을 적용한다. 기본 성능 프로파일은 Forward+ 1x + temporal, 별도 품질 프로파일은 Forward+ 4x MSAA + native/spatial로 제안한다.

11.2 RHI와 GPU 자원 수명

RHI는 resource handle, device memory, descriptor, command list, queue, swapchain, fence, timestamp, capabilities를 제공한다. 버퍼·텍스처·pipeline creation과 submission 실패를 명시적으로 반환한다. graphics/compute/copy queue는 각각 fence를 가지며 제출 순서와 queue wait를 상위 RenderGraph가 결정한다.

```
struct GpuFencePoint {
    DeviceId device;
    uint64 deviceEpoch;
    QueueId queue;
    uint64 value;
};

struct SubmissionResult {
    GpuFencePoint completion;
};

class RenderDevice {
public:
    virtual Result<BufferHandle> CreateBuffer(BufferDesc) = 0;
    virtual Result<TextureHandle> CreateTexture(TextureDesc) = 0;
    virtual Result<SubmissionResult> Submit(const SubmissionDesc&) = 0;
    virtual void Retire(ResourceHandle, Span<const GpuFencePoint> lastUse) = 0;
};
```

graph import/command recording 전에 RenderResourceLease를 원자적으로 획득한다. recording packet은 resource·descriptor allocation·upload range·PSO·vendor context·command allocator의 사용 집합을 소유한다. Retire는 신규 lease를 차단하되 이미 승인된 recording을 무효화하지 않는다. Submit 성공은 packet lease를 submission record와 fence retirement로 이관하며 호출자의 임시 Span을 보관하지 않는다. Retire도 fence 목록을 소유 저장소로 복사하되 호출자 목록은 그 시점의 하한이며 RHI 내부 last-use 집계가 최종 기준이다. Retire 이전에 승인된 recording의 늦은 Submit은 새 queue fence 등록과 recording lease 이관을 같은 동기화 경계에서 수행한다. fence 등록 전에 마지막 recording lease가 0이 되는 순간을 허용하지 않는다.

graph compile/Close 실패나 미제출 취소는 recording worker가 종료된 뒤 lease를 해제한다. Submit 실패는 미제출·부분 제출·device lost를 구분하며 이미 제출되었거나 제출 여부가 불확실한 자원은 RHI 내부 recovery record에 보존한다. 오류 반환만으로 정상 pool에 재사용하지 않는다. 자원 재사용은 신규 사용 차단, CPU recording lease 0, 모든 제출 fence 완료를 함께 요구한다. D3D12 command list가 전달 객체의 참조를 유지하지 않는 제약과 별도로 엔진이 이 수명을 보장해야 한다. S53

record 전/중/후 pause와 asset unload·viewport close·shader reload·submit 실패·device loss를 조합한다. descriptor/upload ring을 축소하여 과거 fence가 완료되어도 recording lease가 있으면 재사용되지 않는지 검증한다.

ResourceHandle은 slot+generation으로 구성한다. Descriptor index, upload memory, PSO와 resource는 마지막으로 사용한 **모든 queue의 fence**가 완료된 후 회수한다. Object GC 완료는 GPU 완료를 뜻하지 않는다. device loss는 진단 정보 저장·종료 또는 제한된 재생성 경로로 처리하고 무한 재시도하지 않는다. 일반 texture mip streaming은 M3/M4 필수다. sparse/virtual paging 최적화는 후속으로 분리하고 고정 budget·실제 backing memory 회수·명시 eviction을 먼저 검증한다.

GPU resource·descriptor·packet·vendor context는 소유 DeviceId/DeviceEpoch로 scope한다. 작은 handle에 모든 필드를 중복 저장할 필요는 없지만 registry domain으로 같은 식별을 검증해야 한다. fence의 의미는 device incarnation·queue·value의 튜플이다. lease 획득·graph import·Submit/Retire·비동기 완료가 domain을 검사하고 구 장치의 불확실한 완료는 구 recovery domain에서만 처리한다. 새 장치의 fence/pool을 구 장치 회수에 사용하지 않는다. API04/BR04는 두 장치와 재생성 전후 같은 slot/queue/value 및 지연 callback을 주입하여 잘못된 resolve·조기 회수·BuiltinReady 덮어쓰기 0건을 검사한다.

11.3 Render Graph

pass는 read/write, subresource range, access, queue, attachment load/store를 선언한다. graph compiler는 DAG 검증, 사용되지 않는 pass 제거, lifetime 분석, barrier·queue wait, transient heap alias, command recording plan을 생성한다. swapchain·history·asset texture는 external import/export 상태를 명시한다. side effect가 있는 readback·present pass는 제거 대상이 아니다.

처음에는 단일 graphics queue와 alias 비활성 모드로 기준 영상을 확보한다. 이후 copy queue → compute overlap → alias 순서로 최적화하고 각 단계의 영상·barrier validation 결과를 대조한다. D3D12의 access/layout/synchronization 의미를 구분하고 지원 시 enhanced barriers backend를 사용한다. S4

GPU profiling은 pass 이름, draw/dispatch 수, transient peak, queue overlap을 기록한다. shader에서 bindless로 간접 접근하는 자원도 pass resource 선언에 포함한다. graph lifetime만으로 범위를 추론할 수 없는 vendor upscaler와 외부 pass는 명시적 resource·state 계약으로 통합한다.

11.4 공통 frame graph

공통 전반부: scene upload·skinning → frustum/거리 판정 → 이전 HZB 기반 provisional reject 분리 → 첫 depth/visibility → 현재 HZB → provisional reject 재검사·새 가시 대상의 같은 frame depth/visibility 복구 → light cluster·표면 shading. 이전 HZB에서 탈락한 집합을 최종 폐기하지 않는다. 복구 결과를 현재 frame의 color·depth와 후속 화면 효과에 포함하며 필요 시 HZB를 갱신한다. shadow는 각 light view의 별도 가시성 계약을 사용한다.

재검증을 생략하는 단순 profile은 이전 HZB만으로 불확실한 대상을 제거하지 않고 보수적으로 그린다. 느린 camera strafe·이동/삭제 occluder도 검증하며 cut·teleport만 예외 처리하여 안전성을 가정하지 않는다. 새 출현·cut에서는 occlusion history를 무효화한다. 이 recovery 구조는 자체 설계안이며 UE 내부 구현의 동등 재현을 뜻하지 않는다. S57

Deferred: GBuffer/visibility → decal·AO·독립 world-space GI 준비 → direct+IBL+base GI의 opaque lighting → 현재 lit color 기반 SSR/SSGI trace → denoise·별도 indirect/specular 항 합성. **Forward+:** 선택 depth/normal/roughness/velocity·AO·독립 GI cache 준비 → forward opaque lighting → 현재 lit color 기반 SSR/SSGI trace → denoise·추가 indirect/specular 항 합성. 현재 scene color를 요구하는 SSR/SSGI를 opaque shading 이전에 실행하지 않는다. 이전 frame color를 쓰는 알고리즘은 그 의존성·reset·latency를 명시한다.

screen-space 합성은 원본 scene radiance와 별도 contribution buffer를 사용하여 같은 항을 이중 가산하지 않는다. 투명 표면은 별도 forward/resolve 계약을 사용한다. 이후 fog/atmosphere·transparency 순서는 효과별 volume/depth 규칙으로 결정하고, velocity 보완 → AA/upscale → output post → HUD → present로 연결한다. current color를 다시 자기 조명 입력으로 사용하는 순환 graph는 compiler가 거부한다.

혼합 경로와 다중 view에서는 shadow는 light/caster revision·projection·coverage·filter key가 동일한 view family에서만 공유하고 scene data는 공통으로 사용한다. 반면 temporal history와 view-sized render target은 ViewId·경로·크기·sample count·jitter sequence별로 분리한다. 경로 전환은 frame boundary에서 수행하고 PSO 준비가 안 된 경우 이전 경로를 유지하거나 명시적으로 대기 화면을 표시한다.

RenderInstanceId·scene snapshot·shadow/GI cache·temporal view key는 SessionId/WorldEpoch 또는 동등한 전역 고유 domain을 포함한다. ViewId 재사용에는 generation과 history reset이 필요하다. 불변 mesh/texture·pipeline cache·GPU arena의 물리 공유는 가변 scene 공유와 구분한다. PI03/PI05는 EditorWorld와 PIE 2개에 같은 asset·다른 transform/material/light를 배치하고 한 session만 이동/seek/stop하여 타 영상·history·lease가 영향을 받지 않는지 검증한다.

11.5 최초 그래픽 API 지원 범위

최초 정식 renderer 지원 범위는 **D3D11·D3D12·Vulkan·Metal·OpenGL의 5개 backend**다. Deferred/Forward는 이 API와 독립된 shading 경로이며 모든 backend의 기본 profile에서 두 경로를 검증한다. 구현을 순차적으로 착수할 수는 있으나 Vulkan/Metal/OpenGL을 선택 후속으로 분류하여 최초 지원 완료를 선언하지 않는다. 플랫폼·SDK·실장비 미확보 항목은 NOT RUN/미완료로 남기고 일정·인력을 재산정한다.

backend	첫 target·shader 경로 제안	실행·동기화 계약
D3D11	Windows, FL11_0/SM5 baseline; HLSL→FXC/D3DCompile→DXBC	context는 동시에 한 thread만 사용; 초기 immediate context owner, deferred recording은 선택 최적화

backend	첫 target·shader 경로 제안	실행·동기화 계약
D3D12	Windows; HLSL→DXC→DXIL, 지원 SM/caps 고정	command list·descriptor·queue fence·explicit state
Vulkan	Windows/Linux, Vulkan 1.2급 capability 후보; HLSL→DXC SPIR-V	binding/layout·queue family·barrier·timeline 지원 여부를 명시
Metal	macOS 우선; typed IR→SPIR-V/MSL 변환 →Apple offline library	Objective-C++ private adapter·command buffer completion·storage/resource usage
OpenGL	Windows/Linux desktop core 4.5급 후보; IR→SPIR-V→GLSL	context owner lane·GL sync·state cache·필요 memory barrier; SPIR-V 직접 입력은 필수 가정하지 않음

표 21

버전·feature level 수치는 초기 profile 제안이며 실제 최소 OS/GPU/SDK와 format/limit conformance를 고정한 뒤 승인한다. macOS Metal용 Core/PlatformBase·window surface adapter와 toolchain worker도 최초 backend 범위에 포함한다. macOS는 전체 Studio·build/meta/cook·PIE·test를 포함한 정식 개발 환경이다. Metal runtime만 성공한 상태를 macOS 개발 지원 완료로 판정하지 않는다. D3D11 SM5 target과 context thread 계약, DXC·SPIRV-Cross·Apple precompile 도구를 공식 자료로 확인한다. 변환기가 존재한다는 이유만으로 모든 shader 의미의 동등성을 가정하지 않는다. S69, S70, S71, S72, S73

기본 공통 profile은 conventional mesh, opaque/masked PBR, depth/shadow/velocity, texture streaming, 기본 post/upscale 입력과 built-in resource를 포함한다. bindless·mesh shader·ray tracing·sparse residency·async compute·timeline은 capability 확장이다. 하위 backend가 고급 경로를 지원하지 않으면 기존 mesh/descriptor·일반 shadow·명시 GI 대체 profile을 사용한다. 모든 기능을 최저 사양으로 제한하거나 동일 고급 기능 지원으로 표시하지 않는다.

RenderGraph는 추상 access·dependency·resource lifetime을 생성하고 backend lowering이 실제 barrier/queue/state를 구성한다. D3D11/OpenGL은 graphics owner lane의 명령 stream으로 직렬 lowering할 수 있으며 존재하지 않는 copy/compute queue 병렬성을 보고하지 않는다. completion token은 backend의 실제 query/sync/command completion에 연결한다. fence 수치를 임의 증가시켜 GPU 완료로 간주하지 않는다. logical PSO는 D3D11/GL에서 검증된 state bundle로 표현할 수 있다.

clip depth·Y 방향·winding·UV origin·matrix packing·binding reflection·sRGB·format·sample count·depth resolve를 backend conformance로 고정한다. 엔진의 reversed-Z/[0,1] 규약을 유지하되 GL context capability와 shader 변환에서 필요한 변환을 한 번만 적용한다. shader key는 API·target language/profile·compiler/translator 버전·layout 규약까지 포함한다. D3D11용 DXBC에 DXIL을 그대로 대입하지 않으며 Metal/GL artifact도 독립 cook 산출물로 보관한다.

ID	최초 renderer 완료 기준
API01	5개 backend의 device·offscreen·지원 platform presentation·resize·device failure 결과
API02	동일 material/mesh의 Deferred/Forward 기준 영상, depth/normal/motion/좌표·색공간 일치
API03	shader compile/reflection·binding layout·format/limit 검증, unsupported 명시 오류
API04	recording/cancel/late submit·resource 교체·streaming·builtin·GPU completion 수명
API05	CPU-only 금지 dependency 검사와 각 실제 GPU suite·debug validation 결과 분리

ID	최초 renderer 완료 기준
API06	graph serial/async lowering의 결과 일치·실제 queue 표시 ·frame/VRAM/compile 비용 계측

표 22

M3/M4는 이 5-backend 기본 Gate를 포함하도록 재산정한다. 기존 단일 D3D12 중심 7–12개월 구간을 그대로 확정 납기로 약속하지 않는다. Core 첫 작업에는 RHI를 연결하지 않고 profile·opaque resource/completion 계약만 준비한다.

12 업스케일링·AA·동적 해상도

12.1 공통 인터페이스와 지원 단계

업스케일링은 Deferred와 Forward 양쪽의 필수 기능이다. native 출력, 자체 spatial baseline, 자체 temporal upscaling/TAAU, 외부 FSR·DLSS·XeSS adapter를 같은 인터페이스로 연결한다. 외부 SDK는 해당 장치·OS·드라이버·SDK 조건을 충족할 때 노출하며, 선택 기능의 실행 가능 여부와 fallback을 UI에 명시한다. 모든 vendor 경로가 모든 GPU에서 실행된다고 가정하지 않는다.

```
struct UpscaleInputs {
    GraphTexture colorHdr, depth, motion;
    GraphTexture exposure, reactiveMask;
    GraphTexture transparencyMask;
    Extent2D renderSize, outputSize;
    Float2 jitterPixels;
    float deltaSeconds;
    bool resetHistory;
};

class Upscaler {
public:
    virtual UpscalerCaps QueryCaps(const DeviceCaps&) const = 0;
    virtual Result<UpscalePlan> Configure(UpscaleConfig) = 0;
    virtual GraphTexture AddPasses(RenderGraph&, const UpscaleInputs&) = 0;
    virtual void RetireContext(Span<const GpuFencePoint>) = 0;
};
```

모든 필드를 모든 SDK가 요구하는 것은 아니다. adapter는 필수 입력·허용 render size·HDR convention·지원 reactive mask·내부 sharpen 여부를 선언한다. 공통 입력의 존재가 SDK 간 의미의 동일성을 보장하지 않으므로 motion 방향·단위·jitter 포함 여부·depth reverse·exposure 방식을 adapter에서 변환하고 자동 영상 테스트로 확인한다.

단계	제공 기능	완료 조건
U0 · M3	native, bilinear/개선 spatial, 선택 sharpening	모든 출력 크기와 viewport resize, 두 경로 정상 표시
U1 · M4	motion vector, jitter, TAA/TAAU baseline	camera·rigid·skinned motion, history reset, disocclusion 검증
U2 · M5	dynamic resolution, FSR adapter	render/output 독립 크기, budget 안정화, 두 경로 회귀 통과

단계	제공 기능	완료 조건
U3 · M6 이후	DLSS·XeSS adapter	지원 장치별 실행·SDK validation·fallback·패키징 검증
U4 · P3	frame generation 등 선택 기능	별도 latency·UI·frame pacing 계약과 실제 표시 검증

표 23

공식 SDK 입력 계약은 각 adapter의 기준으로 사용한다. S19, S20, S21

SDK는 자동으로 '최신'에 추종하지 않고 Foundation 이후 각 adapter 착수 시 검증 commit·binary hash·지원 장치를 manifest에 고정한다. 특정 SDK가 제공하는 spatial·temporal·frame generation 기능을 동일 기능으로 취급하지 않는다. Frame generation은 시뮬레이션 tick 또는 입력 반응 속도를 증가시키지 않으므로 본 계획의 기본 60Hz 성능 판정에 생성 frame을 포함하지 않는다.

12.2 시간 재구성과 수명 규칙

카메라의 현재·이전 unjittered matrix, 현재·이전 object transform, skeletal pose, morph와 vertex deformation을 저장한다. motion은 engine 공통 convention으로 '현재 sample에서 이전 frame 위치로 향하는 output-normalized vector'를 제안하고 adapter에서 pixel/NDC로 변환한다. 현재·이전 jitter는 별도 입력으로 관리하여 중복 보정하지 않는다. teleport, spawn, camera cut, world origin 변경, RenderPath 변경, upscaler 교체, output resize는 history reset 또는 SDK가 검증한 resize 절차를 수행한다.

disocclusion, animated foliage, particle, emissive, 반투명, thin geometry의 ghosting을 별도 측정한다. reactive/transparency mask는 지원 adapter에 전달하고, 지원하지 않으면 보수적 history weight를 적용한다. HUD·text·cursor는 업스케일 후 출력 해상도로 합성한다. world-space UI는 depth·motion을 제공하거나 의도한 투명 경로를 사용한다. sharpening은 SDK와 후처리에서 중복 적용하지 않는다.

기본 temporal upscaling은 선형 HDR color, depth, motion, exposure가 유효한 단계에서 실행하고 tone mapping·출력 UI는 그 뒤에 수행한다. DOF·motion blur·bloom·film grain의 위치는 효과별 render/output resolution 계약을 정한다. vendor 요구가 다른 경우 전용 graph 구성으로 처리하고 디버그 화면에서 실제 입력을 확인할 수 있어야 한다.

dynamic resolution은 지연된 GPU timestamp의 이동 평균, hysteresis, 변화율 제한으로 render size를 제어한다. UI와 출력 해상도는 고정한다. quantization·최소/최대 비율은 adapter 권고 범위와 alignment를 만족해야 한다. CPU 병목일 때 render scale을 계속 낮추지 않으며 resize 시 임시 resource 급증과 history 메모리 상한을 검사한다.

12.3 검증 기준

고정 카메라·수평 회전·고속 이동·skinning·반투명·입자·가는 철망·반복 무늬·야간 emissive·resize를 포함한 시퀀스를 사용한다. Deferred/Forward × native/spatial/temporal/지원 vendor × 1x/Forward 4x MSAA의 지원 조합을 manifest로 정의한다. 비지원 조합은 설정 단계에서 명시적으로 거부한다.

native 고해상도 기준 영상과 비교하여 SSIM/이미지 차이, edge flicker, trail 지속 시간, disocclusion 회복, 모션 벡터 오류를 기록한다. 하나의 SSIM 값으로 품질을 승인하지 않는다. target GPU별 upscaler 시간, 전체 GPU 시간, peak VRAM, frame latency를 측정한다. 초기 목표는 정적 장면 SSIM 0.97 이상, camera cut 후 과거 history 잔상 0 frame이며 다른 동작 시퀀스의 허용치는 M4에서 기준 영상을 검토하여 고정한다. 수치는 모두 미측정 제안이다.

13 GPU Scene·Static Mesh·VirtualGeometry

13.1 GPU-driven 기준 경로

StaticMeshAsset은 positions, packed normals/tangents, UV, material slots, section, bounds, conventional LOD, meshlet, collision, GI proxy를 보관한다. import 때 퇴화 삼각형·잘못된 winding·nonfinite 값·UV 범위를 진단한다. meshoptimizer는 cache·fetch 최적화와 meshlet cook의 후보로 어댑터에 통합한다. S5

GPU Scene은 RenderInstanceId별 transform, previous transform, mesh/material handle, bounds, flags를 저장한다. 변경 범위만 upload하고 frame overlap 중 같은 buffer 영역을 덮어쓰지 않는다. material table·descriptor table도 version과 deferred retirement를 적용한다. GPU에는 Object 주소나 ObjectHandle을 직접 전달하지 않는다.

culling은 frustum → distance/LOD → HZB → meshlet/cluster → indirect argument 생성 순으로 발전한다. 초기에는 indexed indirect draw를 사용하고, caps가 확인된 장치에 Mesh Shader backend를 추가한다. ExecuteIndirect는 GPU가 작성한 명령 buffer를 실행할 수 있으므로 CPU 개별 제출 감소의 기반으로 사용한다. S6

indirect buffer overflow와 page request overflow는 counter와 제한을 가진다. overflow 시 객체를 임의로 소실시키지 않고 추가 batch·보수적 fallback·다음 frame 재요청을 적용한다. 비컬링 기준 depth/ID 영상에서 얻은 가시 집합 V_ref가 제출 집합 V_submit의 부분집합인지 확인한다. 가시 물체를 occluded로 잘못 제거하는 false positive는 금지하고, 가려진 물체를 추가로 그리는 false negative는 성능 비용으로 계측한다. CPU reference의 보수적 가시 집합과 완전 일치를 강제하지 않는다. overflow 재요청은 현재 frame의 parent/fallback 표시를 유지한 상태에서만 다음 frame으로 이월한다.

13.2 Nanite 유사 가상화 지오메트리

공개 Nanite 발표의 cluster hierarchy·streaming·rasterization 개념을 참고하되 자체 구현의 품질·지원 범위를 독립 검증한다. S7

오프라인: mesh partition → cluster boundary 보존 simplification → error metric → parent/child hierarchy 또는 DAG → quantization/compression → page packing → 상위 fallback 저장. **런타임:** projected error 기반 cut 선택 → resident page 확인 → missing request 집계 → 안전한 parent fallback → hardware/compute raster → visibility/material resolve.

meshlet 도입만으로 가상화 지오메트리가 완성되었다고 판정하지 않는다. 누락 page에서도 watertight한 인접 경계, 보수적 bounds, LOD crack 방지, parent-child 동시 표시 방지, page table update atomicity, 압축 해제 비용, material binning까지 완료해야 한다. root와 최소 fallback은 항상 resident로 유지한다. draw에 사용한 page는 fence 완료 전 eviction하지 않는다.

첫 지원은 rigid opaque/masked StaticMesh이며 skinned mesh·복잡한 displacement·반투명은 conventional 경로로 유지한다. Deferred visibility resolve를 먼저 완료하더라도 Forward에서 동일 asset의 conventional LOD가 반드시 동작해야 한다. Forward용 VirtualGeometry는 material evaluation을 shared IR에서 생성하고 직접 shading 또는 resolve 비용을 별도 검증한 후 지원표를 확대한다. 기능 제한과 fallback을 editor asset inspector에 표시한다.

14 조명·DynamicGI·재질·후처리

14.1 Material과 shader

Material Graph는 typed IR → 공통 shader 생성 → backend별 FXC/DXC/변환기/Apple compiler → artifact/reflection → pipeline의 단계로 컴파일한다. 11.5절의 DXBC·DXIL·SPIR-V·MSL·GLSL 경로를 구분한다. common surface function을 중심으로 Deferred, Forward, depth, shadow, velocity, visibility resolve variant를 생성한다. attribute·texture binding·shading model의 의미를 두 경로에서 공유한다.

Permutation은 static switch 목록과 기능 profile로 제한한다. platform×path×sample count×material×skinning 조합의 증가량을 cooker가 보고한다. runtime parameter는 불필요한 shader variant를 생성하지 않는다. PSO cache key에는 shader hash·root signature·render format·sample count·device/driver 호환 정보를 포함하고 stale cache를 폐기한다. shader hot reload 실패 시 이전 검증 버전을 유지한다.

14.2 조명 확장

기본은 directional/point/spot, clustered light list, cascaded/atlas shadow, IBL·reflection probe, SSAO, emissive, exposure·tone mapping이다. 이후 contact shadow·SSR·decals·sky/atmosphere·volumetric fog를 추가한다. Forward는 screen-space 효과에 필요한 최소 surface 출력만 선택 생성하고 full GBuffer를 의무화하지 않는다.

가상 그림자는 page table, physical tile pool, demand feedback, caster/receiver invalidation, cache age를 별도 모듈로 구현한다. 전통 shadow map fallback을 유지한다. VirtualTexture도 page 요청·upload·anisotropic border·mip fallback·fence retirement를 포함한다. geometry·shadow·texture page pool은 공통 budget arbiter와 각기 다른 eviction 정책을 사용한다.

M6–M8 고급 직접 조명 범위에는 Rect/area emitter, source radius/length/width/height, Kelvin 색온도, IES profile 임포트, light-function/cookie를 포함한다. 광량은 directional의 lux, local light의 lumen/candela 등 타입별 단위·환산·IES 정규화 규약을 고정한다. analytic 근사와 면광원 적분, shadow의 실제 source-size 반영 여부를 구분하고 Deferred/Forward·baked/probe/ray·capture별 지원표를 공개한다. 공개 Rect Light·IES 자료는 제작 기능의 비교 기준이다. S58, S59

고정 물리 카메라에서 source 크기·거리·광량·IES 방향을 바꾸어 조도·specular highlight·soft shadow를 검증한다. unsupported 조합은 명시 근사/fallback으로 보고하며 point/spot만으로 고급 직접 조명 완료를 판정하지 않는다.

14.3 Lumen 유사 DynamicGI

Lumen의 공개 문서는 screen trace, software/hardware ray tracing, surface cache와 시간 분산 갱신의 결합을 설명한다. 아래 구성은 이를 참고한 자체 단계안이다. S8

G0: baked/probe IBL + SSAO + reflection probe. 두 렌더 경로의 안정적인 기준 조명.
G1: screen-space GI/reflection과 temporal denoiser, 화면 밖 probe fallback. screen trace만으로 완전한 GI를 선언하지 않음.
G2: diffuse probe/radiance cache, world-space coverage와 update budget, moving light invalidation. 첫 DynamicGI 실용 단계.
G3: static SDF/clipmap 또는 하드웨어 BVH trace, surface card/cache, hit lighting의 통합. 두 backend 중 하나부터 실증.
G4: multi-bounce 근사, specular roughness별 trace, 대규모 월드 streaming, dynamic/skinned representation 확장.
SDF·surface cards·BVH용 mesh는 에셋 cook에 연결하고 버전·checksum을 유지한다. ray tracing 표현과 raster 표현의 오차를 debug view로 확인한다. thin wall light leak, emissive noise, disocclusion, cache coverage hole, streaming 경계의 잔상, material view dependence를 검증한다. 실내·실외·이동광원·mirror·skinned character에 대한 지원 수준을 별도로 기록한다.

Deferred와 Forward 모두 공통 GI 결과를 조명 입력으로 소비한다. 필요한 depth/normal은 Forward auxiliary pass에서 생성한다. 선택 backend/profile의 hardware ray tracing capability가 없으면 software/probe 경로로 낮추며 경로별 품질·시간을 따로 측정한다. 최소 GI가 없는 경우에도 직접 조명·shadow·IBL이 정상 표시되는지 검증한다.

후처리는 linear HDR, exposure, tone mapping, output color space를 명시한다. bloom, DOF, motion blur, color grading, film grain, sharpen은 해상도·AA·upscale 위치를 소유한 pass로 구현한다. HDR display·wide gamut은 26절의 정식 출력 profile로 개발한다. stereo/VR은 별도 P3 profile이며 Forward 지원만으로 VR 지원을 완료하였다고 판정하지 않는다.

14.4 고급 재질·투명·Ray Scene

고급 공통 Material IR은 clear coat·anisotropy·subsurface/profile skin·cloth·eye·two-sided foliage·thin translucent·water를 명시 shading model로 정의한다. detail normal·UV transform·vertex color·material function/layer instance·parameter collection·per-instance data·WPO와 이전 WPO를 지원한다. 모델별 Deferred encoding과 Forward evaluation을 모두 제공하며 표

현 한계는 capability로 노출한다. Unreal 재질 분류를 비교 기준으로 사용한다. S32

Substrate 유사 임의 closure/layer 체계는 기존 PBR에 자동 포함되지 않는다. 재질 compiler·GBuffer 저장·ray evaluation·shader permutation 비용을 별도 연구 gate에서 검증한다. S33

Transparency는 blend mode·sort key·depth write·two-sided·refraction·separate translucency·velocity/DOF 정책을 포함한다. planar reflection·scene capture·render-to-texture·orthographic view와 recursion depth 제한을 M6–M8에 추가한다. OIT는 선택 후속 알고리즘이며 기본 sorted transparency의 한계를 숨기지 않는다. baked lightmap·irradiance probe·reflection capture는 DynamicGI 대체 profile로 제공한다.

RayScene은 BLAS/TLAS의 geometry revision·transform·material table·ray mask·skinned update·scratch lifetime을 소유한다. raster virtual geometry와 ray/collision proxy 오차를 별도 측정한다. GI·reflection·shadow ray budget과 denoiser 입력을 공유 계약으로 정의하며 GPU fence 전 acceleration structure를 재사용하지 않는다. 작은 reference path tracer는 M9의 조명/재질 검증 도구와 offline 선택 profile로 계획한다. production 영화 renderer와의 완전 동등성을 가정하지 않는다. S35

14.5 필수 노드 기반 머티리얼 에디터

머티리얼 에디터는 선택 도구가 아니라 반드시 구현할 제품 기능이다. Unreal Material Editor의 그래프 기반 제작·컴파일·미리보기·인스턴스 편집 과정을 기준으로 자체 에디터를 구현한다. Blueprint와 범용 gameplay visual scripting의 제외 정책은 그대로 유지한다. 머티리얼 그래프는 GPU 셰이더를 생성하는 선언형 도구이며 게임 객체 생성·RPC·게임 이벤트·임의 gameplay 실행 흐름을 포함하지 않는다. 커스텀 게임플레이 언어와 C++는 typed parameter API로 재질을 제어한다. S60

Core/Object가 첫 작업이라는 순서는 유지한다. M3에서 최소 제작 경로를 제공하고 M4에서 필수 기본 에디터 Gate를 통과한다. 고급 domain·layer·shading 기능은 해당 렌더러 지원 단계에 맞춰 확장한다. 당장 미지원인 node/target은 명시 상태로 둘 수 있으나, '머티리얼을 코드로만 작성 가능'하거나 'compiler만 존재함'을 에디터 완료로 판정하지 않는다.

기능군	필수 제작 기능	단계·검증
그래프 편집	typed pin 연결·연결 해제, drag/pan/zoom, 다중 선택·복사/붙여넣기·삭제, undo/redo, node 검색·palette, comment·reroute·정렬	M3 최소 편집, M4 정상 제작 Gate
기본 표현식	scalar/vector/color·texture parameter, sample/sampler, UV/transform, math/lerp/clamp, normal·vertex color·time, static switch	M3 PBR 기본 세트, M4 확장·타입 오류 진단
Material Output	BaseColor·Metallic·Roughness·Normal·Emissive·OpacityMask 등 domain별 typed output, blend·two-sided·shading model	M3 opaque/masked/unlit, 이후 지원표 확장
실시간 확인	구·평면·cube·선택 mesh 미리보기, 조명/환경·camera, Deferred/Forward 전환, node preview·realtime on/off	M3 전체 재질 preview, M4 node preview·비교
인스턴스 편집	parent 상속, parameter 그룹·override/reset, texture/color picker, static variant, runtime dynamic parameter	M4 기본 instance·상속, M5 확장
재사용	typed Material Function·입출력, 함수 탐색·의존 추적·변경 전파, Material Attributes 집합	M4 함수, M5/M6 layer/blend instance
진단·비용	node/pin 위치 오류, generated HLSL, target/path별 compile 결과·instruction/texture/sampler 비용·variant 수	M3 오류 표시, M4 상세 통계·compile queue

기능군	필수 제작 기능	단계·검증
제작 통합	asset browser·mesh slot drag/drop, 저장·재열기·source control diff, PIE preview, cooker/DDC·headless compiler	M3 왕복, M4 반복 제작·자동 회귀

표 24

Unreal의 문서는 live update·node preview·platform stats·generated shader code와 인스턴스 override를 제작 기능으로 구분한다. 자체 구현도 각각의 완료 기준을 둔다. 통계는 compiler가 제공하는 정적 지표와 실제 GPU 측정을 분리하며 instruction 수만으로 frame 성능을 보장하지 않는다. S61, S62

14.6 모듈·파일·확장 인터페이스

모듈·예정 파일	의존·책임	금지 의존·수명
MaterialSchema: Public/Material/GraphSchema.h, NodeDescriptor.h, ParameterSchema.h	Core·MetaSchema·Serialization; stable NodeId/PinId/ParameterId·schema version·domain	RHI·World·웹·Object 포인터 없음
MaterialCompiler: GraphValidator.cpp, MaterialIR.cpp, HlslEmitter.cpp, SourceMap.cpp	Host Core·MaterialSchema·ShaderWorker; typed DAG 검증·IR·HLSL·diagnostic	EditorWeb 없이 headless 실행; host DXC/STL은 Runtime에 유입 금지
MaterialRuntime: MaterialHandle.h, MaterialInstance.h, MaterialBinding.cpp	Core·RenderData·RHI 계약; cooked shader set·parameter layout·frame snapshot	graph UI·Clang·DXC·원본 node 구현 없음
MaterialAssetBridge: MaterialAsset.cpp	AssetRuntime·ObjectRuntime·MaterialSchema/Runtime; 원본·의존 asset·cooked artifact 연결	Renderer가 Object graph를 순회하지 않음
MaterialCook: MaterialCook.cpp	Host-only target; Asset 도구·MaterialSchema·MaterialCompiler·cooked format 계약	Shipping 링크 제외; Compiler/DXC 의존은 이 target에서만 사용
EditorMaterial: GraphCommands.cpp, PreviewSession.cpp, CompileCoordinator.cpp	EditorBackend transaction·asset bridge·compiler service·renderer preview	async callback에 raw Object/WebView 주소 캡처 금지
Editor/Web/Material: MaterialGraph.tsx, InstancePanel.tsx, DiagnosticsPanel.tsx	versioned RPC schema; 그래프·속성·인스턴스·오류 UI	브라우저에서 별도 shader 의미/validator 구현 금지

표 25

경로는 모듈 내부 Public/Private 상대 경로 제안이다. MaterialSchema/MaterialRuntime은 Runtime STL 비사용 정책을 따른다. 그래프 canvas widget은 필요하면 다른 전문 도구에서 재사용할 수 있으나 node 의미·VM·execution flow를 통합한 범용 Blueprint 시스템으로 확대하지 않는다. EditorWeb와 native viewport는 동일한 preview session/route epoch를 사용한다.

```
struct MaterialNodeDescriptor {
    NodeTypeId type;
    uint32 schemaVersion;
    Span<const MaterialPinDesc> pins;
    MaterialNodeCapabilities capabilities;
};

struct MaterialCompileRequest {
    MaterialGraphSnapshot source;
    MaterialCompileProfile profile;
    MaterialSemanticHash expectedHash;
};

class MaterialCompilerService {
public:
    Result<CompileTicket> RequestCompile(MaterialCompileRequest&&);
    Result<void> Cancel(CompileTicket);
};

Result<NodeRegistration> RegisterMaterialNode(const MaterialNodeDescriptor&, NodeLoweringOps,
                                              ModuleLease);
```

예시는 signature 초안이다. registration은 schema·pin type·domain·stage·backend 지원·parameter migration·lowering ops를 검증한다. snapshot은 compiler가 소유하는 immutable native 데이터이며 Task에 Object 주소를 전달하지 않는다. descriptor의 배열은 등록 시 소유 복사하고 lowering function pointer는 module lease로 보호한다. unregister는 신규 compile admission 차단·진행 작업 drain 뒤 수행한다. 새 node는 plugin 등록과 backend conformance로 추가하며 Core 파일 수정을 요구하지 않는다.

NodeId/PinId/ParameterId는 표시 이름과 분리한다. graph schema는 layout revision과 semantic revision을 구분하고 node 이동·comment 변경이 shader compile을 유발하지 않게 한다. unknown node는 editor에서 payload를 보존하는 placeholder로 열되, 출력에 도달하는 미지원 node는 target cook 오류로 처리한다. 함수·layer 참조의 순환과 shader dataflow cycle을 진단한다. 구조 변경·parameter rename·pin 제거에는 schema migration과 인스턴스 override 보존 정책이 필요하다.

14.7 셰이더 의미·인스턴스·레이어

하나의 typed Material IR에서 Deferred/Forward 및 depth/shadow/velocity variant를 생성한다. IR은 value type·shader stage·coordinate space·texture 색공간·derivative 요구·resource binding·domain을 기록한다. vertex 단계의 implicit derivative sample, 잘못된 normal 공간, 지원하지 않는 scene texture 접근은 위치가 있는 오류로 반환한다. WPO는 current/previous 평가 계약을 가지며 upscaler motion과 shadow/velocity pass에 동일 revision을 사용한다.

shader에서 scene color/GBuffer를 읽는 기능은 target/path/domain capability를 검사한다. Forward에서 없는 GBuffer를 읽는 node를 정상 지원으로 표시하지 않는다. PostProcess domain은 26절의 HDR/노출·tone-map 전후 위치와 read/write 자원 계약을 따른다. UI·decal·particle·landscape·volume domain은 같은 편집기에서 지원표에 따라 활성화하며, 해당 renderer가 준비되기 전에는 Disabled/Unsupported로 표시한다.

scalar/vector/texture 같은 dynamic parameter 수정은 이미 호환되는 binding layout 내에서 shader 재컴파일 없이 frame parameter snapshot을 갱신한다. static switch·blend/shading model·layer topology·layout 변경은 compile/cook 대상이다. material instance parent chain은 cycle을 거부하고 override는 MaterialAssetId·ParameterId·layer instance 경로·function call-site NodeId 경로로 식별한다. shared/global parameter는 별도 명시 namespace로 공유하고 기본 함수 호출의 독립 override와 구분한다. 재질 함수의 같은 parameter 이름이나 같은 layer asset의 반복 사용이 서로의 override를 덮어쓰지 않게 한다. runtime에서는 cook된 variant만 선택하고 shipping에 원본 material graph/HLSL 도구를 포함하지 않는다. OpenGL의 target GLSL compile/link와 driver pipeline compilation은 플랫폼 실행 단계이며 유효하지 않은 program binary cache는 검증된 target source로 복구한다.

Material Function과 Material Attributes 기반 layer/blend 편집은 필수 확장 범위다. M5/M6에 layer stack·순서·mask/blend·instance override를 추가한다. Unreal의 layer 편집은 인스턴스에서 layer를 교환·재정렬하는 제작 과정의 참고다. 이는 14.4절 Substrate 유사 임의 closure 체계의 연구 완료를 의미하지 않는다. 고급 closure가 미완성이어도 기본 PBR·함수·레이어 에디터는 완료할 수 있다. S63

Custom HLSL expression은 선언된 input/output·include allowlist·resource budget·지원 target을 가진 선택 고급 node로 둔다. 게임플레이 커스텀 언어 실행 node로 취급하지 않는다. 초기에 미지원이어도 기본 에디터 Gate를 막지 않으며 임의 shader text가 graph validator·resource binding 검사를 우회하도록 구현하지 않는다.

14.8 비동기 컴파일·미리보기·반복 개발

편집 명령은 expected document revision을 포함한 transaction으로 적용한다. source asset 저장과 shader 성공 여부는 분리한다. 오류가 있는 graph를 draft로 저장·재열기할 수 있지만 cooker는 현재 source semantic hash의 필수 variant가 모두 검증된 경우에만 publish한다. 이전 성공 shader로 잘못된 현재 graph를 조용히 출하하지 않는다.

CompileCoordinator는 연속 편집을 debounce/coalesce하고 foreground material과 선택 node preview를 우선 처리한다. node preview는 요청한 node/해상도/target 수를 제한하며 모든 node를 매번 컴파일하지 않는다. material·function dependency hash, node plugin/compiler/translator/target toolchain 버전, include closure, static parameter, platform/path/domain/quality/profile이 DDC key에 포함된다. graph 위치·zoom·selection 같은 UI metadata는 semantic key에서 제외한다. remote compile에는 동일 toolchain과 immutable 입력을 사용하고 cache corruption·worker 실패는 검증 후 local retry/terminal error로 처리한다.

결과 수용 조건은 session epoch·현재 semantic hash·target profile·compile generation의 일치다. undo/redo·빠른 edit·preview 종료 이후 늦은 결과는 폐기한다. 취소는 즉시 worker 소멸을 뜻하지 않으므로 source snapshot·module lease·출력 buffer를 terminal까지 유지한다. node 이동으로 document revision만 바뀐 경우에는 같은 semantic 결과를 사용할 수 있다.

compile 실패 시 preview는 이전 검증 shader를 유지하되 '현재 graph와 다른 마지막 성공 버전' 상태와 오류 위치를 표시한다. 성공 shader set·parameter layout·resource binding은 호환성을 검사하여 frame 경계에서 한 묶음으로 교체한다. parameter 갱신과 texture 교체도 frame별 immutable snapshot을 사용한다. 구 PSO·shader·texture·descriptor는 recording lease와 실제 마지막 GPU fence가 모두 종료된 뒤 회수한다. node/graph를 닫았다는 이유로 진행 중 GPU 자원을 해제하지 않는다.

성능 검증은 dynamic parameter-only 변경의 compile job 0개, graph UI layout-only 편집의 semantic hash 불변, 변경된 function의 역의존 closure만 재컴파일하는지를 우선 판정한다. warm/cold compile p50/p95, worker queue, DDC hit, foreground preview 도달 시간, VRAM을 계측한 뒤 시간 목표를 확정한다. 웹 편집·CPU headless compiler와 GPU offscreen preview의 검증 결과를 구분한다.

14.9 필수 마일스톤과 수용 시험

단계	필수 결과	완료 조건
MAT0 · M3	MaterialSchema·typed validator·IR·headless ShaderWorker, web graph 기본 편집	텍스처와 parameter를 연결한 PBR graph 저장→compile→재열기
MAT1 · M3/M4	native live preview·asset 적용·Deferred/Forward·undo/redo·오류 표시	동일 graph를 두 렌더 경로의 mesh에 적용, 실패 후 이전 preview 유지와 상태 표시
MAT2 · M4	instance editor·functions·node preview·검색·comment/reroute·HLSL/통계·기본 cook	아래 MT01–MT10의 기본 profile 통과; 기본 에디터 필수 Gate
MAT3 · M5/M6	layer/blend instance·의존 diff·UI/PostProcess 및 준비된 domain	고급 재질 범위별 migration·cook·회귀 통과

단계	필수 결과	완료 조건
MAT4 · M6–M8	skin/cloth/eye/water 등 모델·VT/virtual geometry/RT 표현 확장	target별 지원표·fallback·shader cost·upscale/velocity 검증

표 26

ID	수용 시험	판정 기준
MT01	graph 저장/재열기·복사·undo/redo·node/pin migration	연결·ID·parameter override 보존, 파손 시 명시 오류
MT02	타입·stage·domain 오류, cycle·누락 function·unknown node	원인 node/pin 위치 일치, 잘못된 required variant publish 0건
MT03	같은 PBR graph의 Deferred/Forward·depth/shadow/velocity	profile별 reference 비교·coverage 일치, 경로별 제한 표시
MT04	instance parameter/texture 변경·static switch 변경	dynamic parameter/graph UI layout-only compile 0개, static 변경의 필요한 variant 생성
MT05	빠른 edit·undo·cancel·session close·compile 결과 역순 도착	현재 hash/profile만 적용, stale publish·UAF 0건
MT06	function/layer 반복 사용·parent chain·dependency 변경	순환 거부, instance별 override 분리, 영향 closure만 invalidation
MT07	compiler 실패·DDC 손상·worker 종료·cook 누락 variant	이전 preview 상태 표시·재시도 한도·현재 source cook 실패
MT08	웹 editor 없는 CPU headless compile/cook·GPU offscreen 비교	동일 toolchain/profile의 canonical artifact hash, GPU 실행은 별도 지원 장비에서 확인
MT09	shader/texture 교체 중 late GPU submit·preview 반복 종료	recording/fence 전 free 0건, 완료 후 resource/lease 회수
MT10	plugin node 추가·비활성 target·저장 충돌·draft recovery	Core 변경 없이 확장, unsupported 구분, source 보존

표 27

MAT2의 MT06은 Material Function·기본 instance에 적용하고 layer fixture는 MAT3에 추가한다. MT03/08/09는 선언된 GPU profile에서 실행하며 mock 결과를 실제 렌더 검증으로 합산하지 않는다. 전체 UE 기능의 즉시 동등 구현을 요구하지 않되 기본 머티리얼 에디터의 완료를 선택 항목으로 낮추지 않는다.

14.10 필수 라이트맵 베이킹과 범용 프로브 조명

라이트맵 베이킹과 이동 객체용 프로브 조명은 범용 제품의 필수 기능이다. 고급 DynamicGI의 대체 경로이자 낮은 GPU 비용의 정식 품질 profile로 제공한다. M3에 bake/runtime artifact 계약, M4에 CPU 기준 baker·lightmap 적용·기본 probe 샘플링, M5/M6에 adaptive volume·scenario blend·규모/제작 도구를 확장한다. GPU baker는 성능 선택이며 CPU baker의 실행·headless 재현성을 선행한다.

BakeScene은 geometry revision·light/material/sky 값·UV/chart·transform·sampling seed·settings의 immutable snapshot이다. lightmap UV 생성/검사·overlap/padding·texel density·atlas packing·방향성/비방향성 출력·dilation·denoise·압축을 제공한다. progressive preview와 최종 품질을 구분하고 재베이크 영향 dependency·중단/재개·분산 job·결과 checksum·atomic publish를 지원한다. bake 결과는 DDC cache와 별도 versioned lighting asset/manifest로 저장하며 원본 scene 변경으로 stale가 되면 명시 표시한다.

M3에 BakeMaterialProfile을 Material IR의 CPU 소비 target으로 정의한다. M4 기본 PBR·opacity/mask·emissive·normal/UV·static switch·material function 부분집합은 CPU evaluator 또는 명시 surface 표현으로 변환한다. texture의 offline sampling·색공간·footprint와 instance/time 입력을 BakeScene revision에 고정한다. derivative·view-dependent·Custom HLSL 등은 정확/근사/proxy/미지원 상태를 선언하고 승인된 bake proxy가 없으면 node 위치를 포함한 오류로 종료한다. 묵시 불투명/평균색 대입으로 성공시키지 않는다. CPU-only bake는 GPU raster/readback에 의존하지 않는다. LT01/LT06에 UV 변환·masked/emissive texture·함수·static switch를 추가하여 GPU 없는 전체 bake와 stale 결과 차단을 검증한다. M8 RayMaterialProfile이 이 초기 CPU 계약을 대체하지 않는다.

Static/Baked light의 direct+indirect, Mixed light의 baked indirect/선택 shadow mask+runtime direct, Realtime light의 runtime contribution을 구분한다. 동일 광원의 같은 항을 lightmap·probe·DynamicGI에서 이중 가산하지 않는다. shadow-mask channel 한도·overflow fallback·movable receiver/caster 정책을 profile에 정의한다. 정적으로 baked된 mesh를 이동/변형하면 invalidation 또는 probe fallback을 적용하며 이전 lightmap을 정상 상태로 유지하지 않는다. reflection capture는 diffuse probe와 별도 specular 데이터다.

프로브의 '동적' 의미는 세 수준으로 구분한다. 첫 필수 단계는 bake된 irradiance를 움직이는 entity가 위치/normal에 따라 보간하여 사용하는 기능이다. 다음은 시간대·조명 상태별 baked scenario를 런타임에서 선택/혼합하는 기능이다. light/geometry 변경에 맞춰 irradiance 자체를 새로 계산하는 runtime probe GI는 별도 update budget·invalidation·history 계약의 고급 확장이다. Unity APV 문서의 baked lighting을 이용한 runtime 변화와 실제 GI 재계산을 혼동하지 않는다. S74

ProbeVolume은 자동/수동 배치·adaptive density·validity·relocation·leak reduction·baked coefficients·cell streaming을 포함한다. 초기 SH L2는 후보이며 계수·좌표·정규화·압축 오차를 고정한다. 큰 skinned mesh는 단일 anchor와 다중 sampling/volume profile을 구분한다. 실내/외 경계·얇은 벽·움직이는 캐릭터·문 열림 상태·World cell 경계에서 누광·색 불연속을 시각화한다. scenario blend는 양쪽 revision/공간 배치의 호환성과 peak memory를 확인하고, 불일치 시 명시 전환/재베이크를 요구한다.

bake editor는 lightmap density/UV/chart·probe validity·샘플링 위치·direct/indirect 분리·stale 상태·진행/잔여 작업을 표시한다. headless `rocks bake lighting` 은 같은 validator·scene snapshot·seed·작업 manifest를 사용한다. output은 5개 RHI backend의 material binding으로 연결하고 lightmap mip·probe cell을 streaming budget에 포함한다. bake worker는 scene 객체 주소가 아니라 immutable 값과 resource lease만 보유한다. 예산 부족·취소·worker crash 뒤 부분 결과를 성공 asset으로 게시하지 않는다.

ID	필수 검증
LT01	UV overlap/invalid chart·padding·texel density·alpha geometry·normal·energy/색공간 기준
LT02	direct/indirect/mixed shadow 항 분리, Deferred/Forward 및 5개 backend의 결과 비교
LT03	이동 entity·대형 mesh·얇은 벽·probe validity·cell 경계의 보간·누광 기준
LT04	scenario 변경/혼합·낮/밤·memory peak·미지원 runtime 갱신의 명시 상태
LT05	geometry/light/material 변경·재베이크·DDC 손상·중단/재개·stale publish 0
LT06	editor/headless 동일 입력 결과·취소/실패/반복 bake의 lease/메모리 누수 0

표 28

RT/cache 재질의 근사·동적 입력 갱신은 AAA03, 대기·reflection·probe의 일관된 환경 revision은 AAA09 계약을 함께 적용한다(33.6절).

LT01–03·LT05–06의 static/basic probe·CPU bake 사례는 M4 필수다. LT04는 M4에 미지원 runtime 갱신의 명시 상태를 검사하고 scenario 선택/혼합·adaptive volume 사례를 M5/M6의 필수 owner gate로 남긴다. report는 required_cases와 deferred_cases/owner_gate를 분리하며 후속 사례를 먼저 PASS로 합산하지 않는다.

15 Skeletal Mesh와 Animation

SkeletonAsset은 stable bone ID, parent index, bind/inverse-bind pose, retarget profile을 보관한다. SkeletalMesh는 section별 bone palette·influence 제한·LOD·morph target을 보관하고 같은 skeleton의 공유를 허용한다. import 시 누락 bone·비정규 weight·좌표/scale·음수 determinant를 검사한다. cook 시 skin weight precision과 bone pruning을 LOD별로 결정한다.

AnimationClip은 position·rotation·scale track, curve, notify, root motion을 포함한다. ACL을 압축·복호화 후보로 사용하며 CPU 작업 메모리와 시간·오차를 adapter에서 계측한다. S9

```
struct PoseBuffer {
    Span<Transform> localPose;
    PoseVersion version;
};

struct AnimEvalInput {
    SkeletonHandle skeleton;
    ClipHandle clip;
    float timeSeconds;
    AnimParameterSnapshot parameters;
};

TaskHandle EvaluateAnimation(const AnimEvalInput&, OwnedPoseOutput,
                             Span<const TaskHandle> dependencies);
```

초기 순서는 clip sampling → blend tree → state machine → montage/additive → IK → retarget → animation graph compiler이다. graph는 cook 단계에서 cycle·타입·누락 pin을 검증하고 실행 bytecode 또는 task plan을 생성한다. worker는 immutable parameter snapshot과 자기 pose output만 처리한다. notify는 tick·sequence ID를 기록하여 게임 lane에서 정확히 한 번 전달한다.

root motion 추출 → character movement intent → collision constrained 이동 → 실제 이동 결과로 pose 보정 순서를 정한다. ragdoll은 physics result를 bone별 blend weight로 합성하고 transform authority를 명시한다. animation budget allocator는 거리·가시성·중요도별 update rate를 조절하되 gameplay critical notify·root motion은 생략하지 않는다.

GPU skinning은 vertex shader baseline 후 compute skin cache로 확장한다. current/previous skin pose·morph·deformation을 velocity pass에 공급하여 시간 업스케일링과 일치시킨다. CPU skinning 기준 결과와 tolerance를 비교하고 Deferred·Forward·shadow·picking에서 동일 pose를 검증한다. 초기 오류 목표는 cook된 clip의 발·손 등 end effector 최대 1 cm, 회전 최대 0.5°이며 콘텐츠별 승인값으로 변경한다. cloth·muscle·motion matching은 P3다.

15.1 동작 제작과 고급 캐릭터

socket/attachment, blend space, layered/additive blend, montage slot·sync group·marker, pose cache·animation sharing, notify state와 root-motion source를 지원한다. Motion Warping·stride/orientation warping·foot placement는 목표 transform·window·collision 결과를 결합한다. 시각 보정이 authoritative 이동을 무단 변경하지 않게 한다. 참고 개념을 자체 component·언어 API로 제공한다. S38

M6의 basic rig·morph/facial editor → M7의 retarget·full-body IK·동작 검색 prototype → M8/M9의 motion matching·cloth/groom profile 순서로 확장한다. pose search database는 feature schema·normalization·selection cost·DDC key를 갖는다. geometry cache/Alembic 계열과 USD scene import는 선택 importer를 통해 M7 이후 편입하고 지원 field·변환 손실을 보고한다.

16 충돌·물리·캐릭터 이동

16.1 통합 구조

물리 기본안은 Jolt adapter와 자체 PhysicsWorld·CharacterMovement 통합이다. Jolt는 멀티코어 rigid-body·collision 라이브러리이며 게임 객체와 별도 native 수명으로 관리한다. S10

Collision은 raycast·sweep·overlap·trigger·filter, Dynamics는 rigid body·constraint·sleep·CCD를 담당한다. query-only collider를 지원한다. Sphere·Capsule·Box·Convex·Compound·TriangleMesh·Heightfield를 우선하며 dynamic body는 convex/compound를 기본으로 한다. 고밀도 virtual geometry와 collision mesh·nav geometry는 별도 cook 산출물이다.

```
struct PhysicsBodyHandle {
    uint32 index;
    uint32 generation;
};

struct PhysicsCommand {
    TickId tick;
    PhysicsBodyHandle body;
    PhysicsOp op;
    PhysicsPayload payload;
};

class PhysicsWorld {
public:
    virtual void Enqueue(PhysicsCommand) = 0;
    virtual PhysicsSnapshot Step(FixedStepDesc) = 0;
    virtual Result<QueryHit> Sweep(const SweepQuery&, const QueryReadScope&) const = 0;
};
```

Step은 60Hz, 최대 substep 4를 초기 제안값으로 사용한다. 싱글플레이의 누적 지연 처리와 서버의 authoritative tick을 구분한다. 서버는 tick/input을 묵시적으로 건너뛰지 않고 admission·부하 제한·명시적 simulation-rate 정책으로 대응하며 변경을 protocol/replay에 기록한다. Jolt 내부 job adapter는 공통 Task Graph와 연결하되 같은 worker를 이중 점유하는 별도 무제한 pool을 생성하지 않는다. physics lock을 보유한 callback에서 게임 코드를 실행하지 않는다.

16.2 소유권과 event

대상	transform authority	전송 규약
Static / Kinematic	World / Gameplay / Animation	step 시작 전 command 적용
Dynamic / Ragdoll	Physics	step 완료 snapshot을 World에 반영

대상	transform authority	전송 규약
Character	CharacterMovement	capsule sweep·step·slide 결과를 이동 상태로 확정

표 29

접촉·trigger event는 tick, body handles, event type, pair ID를 값으로 복사한다. Game lane에서 generation과 owner weak handle을 재검증한 뒤 callback을 호출한다. callback 중 body 추가·삭제는 다음 command batch로 이월한다. body 삭제는 해당 step·query reader 완료 후 수행하며 삭제 이벤트 중복·역순 정책을 테스트한다.

CharacterMovement는 경사·계단·ground snap·jump·moving platform·crouch·sweep depenetration을 독립 상태 머신으로 구현한다. 네트워크 예측 상태는 입력 sequence, 위치·속도·mode, base platform ID를 포함한다. 일반 rigid-body 시뮬레이션의 완전한 cross-platform 결정성을 전제로 하지 않는다.

검증 장면은 stack stability, fast bullet CCD, sliding slope, moving platform, ragdoll transition, streaming collider 제거다. 고정 seed 반복성·penetration·에너지 이상 증가·이벤트 중복을 측정한다. 파괴·vehicle·cloth/soft body·rope·buoyancy·physics field는 기본 이동과 joint 안정화 이후 개별 모듈로 추가한다.

16.3 Query·필터·물리 재질

QueryReadScope는 해당 physics world가 step·구조 변경을 수행하지 않는 읽기 phase의 gate다. 외부 worker는 gate 안에서만 live broadphase를 조회하고 scope를 저장·yield하지 않는다. step과 겹쳐야 하는 async query는 step 경계에 batch 실행하여 값 결과를 전달한다. immutable QuerySnapshot이라는 이름만으로 vendor broadphase가 복제되어 있다고 가정하지 않는다. 독립 snapshot broadphase는 필요 시 별도 메모리·정합 비용을 검증한다.

collision channel·object type·query filter·trigger·ignore pair·physical material/surface·friction combine·complex/simple collision을 data schema로 제공한다. body mass/inertia·COM·constraint editor·physics asset·debug recorder를 포함한다. fracture·vehicle·cloth·rope·buoyancy는 adapter별 지원표와 sample을 갖고 rigid-body 지원으로 완료 처리하지 않는다.

17 네트워크·Replication·Prediction

17.1 프로토콜과 객체 ID

NetCore는 socket/transport·connection·packet IO·channel·암호화된 transport adapter를 담당하고, Replication은 schema·interest·snapshot·RPC를 담당한다. 자체 암호 알고리즘을 설계하지 않고 유지되는 보안 transport 구현을 사용한다. 초기 대상은 dedicated server와 16 client이며 listen server는 후속 지원이다.

```
struct NetObjectId {
    uint32 slot;
    uint32 generation;
};

struct ReplicationContext {
    ConnectionId connection;
    TickId serverTick;
    uint32 byteBudget;
    SchemaHash schema;
};

void BuildSnapshot(const ReplicationContext&, NetSnapshotWriter&);
void ApplyAuthoritativeSnapshot(const NetSnapshotView&);
```

session·world·spawn 순서와 NetObjectId를 관리하고 Object handle을 wire에 기록하지 않는다. spawn 이전 update, destroy 이후 지연 packet, reconnect의 오래된 ID를 검출한다. schema hash·protocol version·asset manifest를 handshake에서 확인한다. 필드 직렬화는 meta compiler가 생성하되 서버 권한, 범위, 길이, 호출 빈도를 runtime에서 검증한다.

delivery는 unreliable sequenced snapshot, 제한된 reliable control/RPC, 우선순위 channel로 구분한다. MTU·fragment 상한·재조립 timeout·신뢰 channel queue 한도를 설정한다. reliable을 '항상 빠른 전달'로 해석하지 않는다. snapshot은 ACK된 baseline에 대해서만 delta encoding하고, baseline 유실·재접속 시 full state로 복구한다.

17.2 게임 상태 동기화

interest는 World Partition cell, 거리, team/owner, gameplay tag, relevancy rule을 조합한다. bandwidth budget을 넘으면 중요도·변경 빈도에 따라 상태 갱신을 조절한다. dormancy·wake·entity migration을 명시한다. 에셋 미상주 시 NetObject와 render-ready 상태를 구분하고 spawn resource prefetch·placeholder 정책을 적용한다.

캐릭터는 client input sequence → local prediction → server authoritative simulation → ACK와 correction → 미확인 입력 replay로 구성한다. remote actor는 buffered interpolation을 사용한다. moving platform·root motion·teleport는 prediction state에 포함한다. 일반 physics body는 state replication과 correction을 기본으로 한다. lag compensation은 서버의 제한된 hitbox history로 구현하고 full World rollback을 초기 범위에서 제외한다.

Replay는 input, authoritative event, RNG seed, schema/asset manifest, periodic snapshot을 기록한다. 결정적인 게임 로직과 비결정적인 physics/render를 구분한다. 영상 재생과 네트워크 디버그를 위한 상태 replay를 제공하고 모든 플랫폼의 bitwise physics 재현을 약속하지 않는다.

시험 profile은 RTT 0/50/100/200 ms, jitter 0/20 ms, loss 0/1/5%, reorder·duplicate·disconnect다. 서버 60Hz tick, per-client bandwidth·RPC queue·correction 거리·stale packet을 계측한다. 강제 despawn/respawn과 cell 이동 중 ID alias·use-after-free·권한 위반 0건을 필수 기준으로 한다.

17.3 Subobject·소유권·여행·온라인 서비스

Entity의 replication root와 등록된 component/subobject 목록을 별도로 관리한다. stable subobject ID, dynamic 생성/삭제, owner-only/skip-owner 조건, OnRep 변경 event, dormancy·relevancy, 대형 collection delta를 지원한다. ObjectRef의 wire 표현은 replicated object면 NetObjectId, asset이면 AssetId이며 일반 local object는 직렬화 오류다. 소유자 spawn → subobject spawn → field fixup 순서를 보장한다. S28

connection ownership과 ControlledEntity possession은 GC 소유권·Scene parent와 독립이다. client RPC는 그 connection에 허용된 server endpoint만 호출한다. travel/reconnect handshake는 WorldEpoch·schema·asset revision을 다시 확인하고 이전 world의 RPC·prediction·reliable backlog를 폐기 또는 명시 migration한다. late join은 current snapshot을 받고 역사적 one-shot

event를 다시 실행하지 않는다.

OnlineServices는 Auth·Session/Lobby·Presence/Friends·Invite·Matchmaking·CloudSave·Achievement/Stats·voice/text chat의 provider adapter를 갖는다. M5에는 local/LAN/mock provider와 session join/reconnect를, M6–M7에는 선택 플랫폼 provider를 통합한다. commerce·platform entitlement·crossplay 운영은 별도 제품 통합 gate로 관리한다. transport replication이 서비스 backend까지 제공하는 것으로 표시하지 않는다. S36

18 오디오·DSP Graph

오디오는 게임의 AudioComponent 설정과 실시간 native mixer를 분리한다. miniaudio는 device IO·decode backend 후보로 사용하고 엔진의 voice·routing·DSP 정책은 독립 인터페이스에 정의한다. S11

```
struct AudioVoiceHandle {
    uint32 index;
    uint32 generation;
};

struct AudioCommand {
    uint64 sampleTime;
    AudioVoiceHandle voice;
    AudioOp op;
    AudioCommandData data;
};

Result<AudioCommandTicket> TryEnqueueAudio(AudioCommand);
void RenderAudioBlock(float* output, uint32 frames) noexcept;
```

48 kHz·256/512 frame block을 기준 후보로 사용하며 device가 지원하는 실제 값으로 구성한다. voice pool·DSP state·command ring을 미리 할당한다. AudioRT callback에서 Object·파일 IO·일반 task wait·heap allocation·mutex·직접 로그 출력을 금지한다. trace는 미리 확보한 ring에 숫자 event만 기록한다.

Audio graph는 worker에서 검증·생성한 불변 plan을 block boundary에 교체한다. 이전 graph와 clip chunk는 audio epoch ACK 후 반환한다. 명령은 bounded MPSC 또는 producer별 SPSC로 전달한다. 일반 명령은 포화 시 Backpressure를 반환하며 낮은 우선순위 parameter는 정의된 latest-value 규칙으로 병합한다. 수락되지 않은 명령을 성공으로 간주하지 않는다. listener·emitter 위치는 snapshot으로 전달한다.

stop/destroy는 voice slot 수에 비례한 사전 할당 mailbox에 generation 포함 idempotent 상태로 기록한다. stop-all/device-stop은 별도 epoch 요청을 사용한다. graph 교체는 미확인 교체 수를 제한하며 ACK 이전 graph/clip을 재사용하지 않는다. AudioRT는 대기·동적 spill을 수행하지 않고 producer가 실패를 처리하거나 admission을 줄인다. 모든 critical 이벤트의 무제한 수용을 보장하지 않는다.

queue 용량 1/2, consumer 지연, stop storm, graph 교체, device loss, voice slot 재사용을 조합하여 수락/거절/병합 의미·ACK·메모리 상한을 검사한다. 이전 generation의 stop이 새 voice를 중단하거나 cleanup ACK 전에 데이터를 반환하면 실패다.

기본은 voice priority·virtualization·bus mixing·resample·gain ramp·pan·distance attenuation·occlusion·reverb send·compressor/limiter이다. HRTF·ambisonics·convolution reverb·audio authoring graph는 P2/P3다. 긴 음원은 worker decode와 chunk ring을 사용하며 underrun을 계측한다. 장치 변경·sample-rate 변경·pause·suspend를 명시적으로 처리한다.

1시간 stress에서 underrun 0건을 목표로 하되 장치·block·OS 부하를 함께 기록한다. callback p99.9는 block 시간의 50% 이하를 초기 목표로 한다. clipping·NaN·갑작스러운 gain discontinuity·삭제된 emitter의 잔존 voice를 자동 검출한다.

예측 가능한 재생 준비 요청과 sample deadline 실패 결과는 AAA10(33.6절)을 적용한다. AudioRT는 준비를 기다리지 않고 승인된 무음/이전 결과/fallback 정책을 사용한다.

19 Navigation과 AI

Recast cook과 Detour query를 adapter로 사용한다. nav build 입력은 렌더링 LOD가 아닌 명시 nav geometry다. agent radius·height·slope·step profile마다 tile key를 분리한다. tiled streaming, off-mesh link, area cost, dynamic obstacle 갱신을 단계적으로 지원한다. S12

```
struct NavRequest {
    AgentProfileId profile;
    WorldPosition start, goal;
    NavWorldVersion version;
    CancelToken cancel;
};

PathTicket FindPathAsync(NavRequest);
void CommitNavTile(TileCoord, OwnedNavTile, NavWorldVersion);
```

query는 versioned nav view와 전용 query context를 사용한다. Detour의 node pool·open list·sliced state는 worker/request별 독점 대여하며 sliced query의 모든 단계에 같은 context와 view lease를 유지한다. 초기 기준 구현은 tile mutation과 query 사이 read/write phase gate를 사용한다. 긴 sliced query는 node/time budget을 제한하고 tile commit 전에 완료·취소한다. phase gate를 yield 너머 무기한 보유하지 않는다.

M7 snapshot 확장은 새 navmesh와 연결에 영향받는 tile/link 저장소를 별도로 구축한 뒤 publish한다. 이전 view의 구조는 reader가 완료될 때까지 수정하지 않는다. 같은 Detour tile pointer의 얕은 복사를 immutable snapshot으로 인정하지 않는다. 공식 구현에는 query 작업 메모리와 기존 인접 tile link를 수정하는 연결 과정이 존재하므로 epoch에 의한 회수 지연만으로 data race가 해결되지 않는다. S55, S56

병렬/sliced query·취소·obstacle·인접 tile 교체/unload를 직렬 oracle·TSan으로 검사하고 node/open-list 한도 초과를 명시 결과로 보고한다. path 결과 적용 시 version과 agent generation을 재검증한다. 미상주 tile을 '경로 없음'과 구분하여 NeedsStreaming·Partial·Blocked·Complete 상태를 제공한다. 동적 nav 변경과 physics collision 변경의 적용 tick을 연결한다.

AI는 perception → blackboard snapshot → behavior/state/utility decision → movement·ability command로 구성한다. 초기에는 Behavior Tree와 Blackboard, 시각·청각 perception, async path following을 구현하고 State Tree·EQS·utility AI를 확장한다. AI worker는 Object를 직접 수정하지 않는다. 늦게 완료된 결과는 entity generation·decision version이 다르면 폐기한다.

AI LOD는 거리·화면 가시성·교전 중요도별로 60/20/5/1Hz 등 제안 rate를 배정한다. 모든 NPC가 같은 frame에 갱신되지 않게 분산한다. gameplay critical event는 낮은 주기와 별도 interrupt로 처리한다. path 재요청·perception raycast는 budget을 제한한다. nav debug view, BT 실행 trace, perception 로그, path cancellation 이유를 editor에 표시한다.

20 오브젝트 태깅·컴포넌트·프리팹·게임플레이

20.1 태깅과 검색

Object Tag와 Gameplay Tag는 편집 분류·검색과 게임 규칙을 각각 명확한 정책으로 지원한다. 공통 기반은 등록형 계층 태그 사전이다. 예를 들어 Character.Enemy.Boss가 Character.Enemy와 Character의 하위 태그인지 조회한다. editor 전용 label과 런타임 규칙 tag를 구분하고 표시 이름과 stable TagGuid를 분리한다.

```
struct TagId {
    uint32 value;
};

struct TagGuid {
    uint64 high, low;
};

class TagSet {
public:
    bool HasExact(TagId) const;
    bool Has(TagId parentOrExact) const;
    bool Matches(const TagQuery&) const;
};
```

TagId는 실행 중 사전의 compact ID이며 영속성·네트워크는 TagGuid 또는 handshake로 합의한 dictionary index를 사용한다. query는 All/Any/None·계층 포함·정확 일치를 조합하여 cook 가능한 expression으로 생성한다. 태그 사전 변경은 version을 증가시키고 rename redirect·삭제 진단을 제공한다. 임의 문자열을 frame마다 intern하지 않는다.

World는 tag→actor/component inverted index를 유지하여 FindByTag를 지원한다. tag 변경은 Game lane transaction에서 원본과 인덱스를 함께 갱신하고 event를 발행한다. query result는 snapshot handle 목록이며 후속 사용 시 generation을 재검증한다. component tag의 Entity 상속·합성은 자동 암묵 적용하지 않고 query option으로 지정한다. replication은 변경 delta와 권한을 명시하며 editor-only tag는 cook에서 제거한다.

검증은 계층 경계(Enemy와 EnemyBoss의 문자열 혼동 방지), rename, duplicate, unloaded cell descriptor 검색, replication dictionary mismatch, prefab override에서 수행한다. UI의 tag picker·검색 filter·bulk edit와 게임의 damage·ability·AI filter가 동일 query 평가기를 사용한다.

20.2 컴포넌트 시스템

Component는 Entity 소유의 Object이며 TransformComponent만 SceneNodeHandle을 갖는다. render·audio·physics·network component의 backend 본체는 독립 native handle이다. Entity는 `GcArray<ObjectRef<Component>>` 로 소유하고 Component의 owner 역참조는 weak를 기본으로 한다. data-only component도 지원하여 모든 component가 transform을 갖지 않게 한다.

Construct → ApplyDefaults/Fixup → Attach → Initialize → Register → Activate/BeginPlay → Tick → Deactivate/EndPlay → Unregister → Retire의 순서를 정의한다. BeginPlay/EndPlay는 world session과 별도 event로 기록하여 PIE 재시작·셀 unload·prefab 재적용 중 중복 실행을 방지한다. disabled와 unregistered, editor-only, pending destruction은 서로 다른 상태다.

```
class Component : public Object {
public:
    WeakObjectRef<Entity> Owner() const;
    virtual Result<void> OnInitialize(ComponentContext&);
    virtual Result<void> OnRegister(ComponentContext&);
    virtual void OnActivate();
    virtual void OnDeactivate();
    virtual void OnUnregister();
};

struct TickSpec {
    TickGroup group;
    float intervalSeconds;
    Span<const ComponentHandle> prerequisites;
};
```

tick group은 PrePhysics, PhysicsSync, PostPhysics, AnimationFinalize, LateUpdate를 기본으로 하고 prerequisite DAG의 cycle을 오류로 검출한다. 독립 component의 병렬 tick은 read/write access 선언이 검증된 경우만 허용한다. Object 가변 상태를 다루는 기본 tick은 Game lane에서 수행한다. tick 중 component 추가·삭제·reparent는 deferred command로 commit한다.

singleton-per-entity·required component·mutually exclusive 규칙을 type metadata로 표현한다. editor가 자동 추가하는 종속 component는 transaction에 포함한다. cook은 순환 종속·누락 필수 component를 검사한다. constructor에서 backend world에 즉시 등록하지 않으며 Register가 실패하면 backend의 부분 등록을 해제하고, 이미 Commit된 객체는 5.3절의 RequestDestroy·정상 retire 경로로 정리한다. 미공개 rollback과 구분한다. editor에는 component tree, active state, tick 의존성, native handle 상태를 표시한다.

20.3 프리팹·중첩·변형·인스턴스 오버라이드

PrefabAsset은 actor/component graph의 local stable NodeGuid, type, field value, parent 관계, 내부 참조를 보관한다. instantiate 시 NodeGuid→새 ObjectHandle/EntityGuid mapping을 먼저 생성하고 내부 참조를 remap한다. 외부 asset은 AssetId, 외부 world object는 SoftEntityId로 유지한다. 원본 Object 주소나 Registry index를 저장하지 않는다.

```
struct PrefabInstanceDesc {
    AssetId prefab;
    Transform root;
    PrefabOverrideSet overrides;
};

Result<PrefabInstanceHandle> InstantiatePrefab(World&, const PrefabInstanceDesc&);
Result<void> RebasePrefabInstance(PrefabInstanceHandle, PrefabRevision newBase);
```

override는 NodeGuid+FieldId+요소 stable key를 주소로 하는 patch다. field 수정, component/child 추가·삭제, reparent, tag 변경을 지원한다. Array index만으로 override를 식별하면 원본 삽입 시 다른 요소를 수정할 수 있으므로 editor 노출 collection에는 stable ElementGuid를 부여하거나 keyed collection을 요구한다.

variant는 base prefab과 patch를 저장한다. nested prefab은 참조 AssetId와 별도 instance namespace를 갖고 reference cycle을 cooker가 거부한다. 초기 중첩 depth 상한 16을 제안한다. 원본 변경 전파는 old base/new base/local override의 3-way 비교로 수행한다. override가 없는 필드는 갱신하고, local override는 보존하며, 삭제된 node·변경된 field type·동시 구조 수정은 conflict로 표시한다.

editor는 Apply to source, Revert override, Unpack instance, Make variant, Compare revision을 제공한다. Apply는 source asset 편집 transaction과 그 영향 목록을 생성하며 동일 instance를 무제한 연쇄 재적용하지 않는다. instance root와 child의 영속 ID를 유지하여 save game·외부 actor reference·network spawn이 흔들리지 않게 한다. 네트워크는 prefab ID·revision·spawn parameter와 authoritative override를 검증한 뒤 생성한다.

PIE 시작은 편집 graph를 별도 World로 복제하고 prefab/actor ID remap을 유지한다. PIE 중 원본 수정은 명시적인 live patch 범위에서만 반영한다. structural change가 있으면 해당 instance 또는 PIE 재시작으로 처리한다. 변경 propagation·GC·Undo/Redo·save/load·cell unload·replication을 함께 수행하는 통합 테스트를 M5의 필수 gate로 지정한다.

20.4 게임 프레임워크·입력·스크립팅

GameSession, WorldSession, MatchRules, MatchState, PlayerState, LocalPlayer, PlayerAgent, ControlledEntity/Character, CameraDirector, ScopedService, GameplayEvent, SaveGame을 제공한다. 서버 권한 규칙과 로컬 presentation을 분리하고 특정 Unreal 클래스 계층과 동일한 이름을 갖는 것만으로 기능이 완성되었다고 판정하지 않는다. Ability는 tag 기반 조건·cooldown·cost·effect·prediction key를 중심으로 P2에서 확장한다.

Input은 action/context, key/gamepad/touch, rebinding, dead zone, chord, haptics, focus를 지원한다. editor shortcut·viewport camera·PIE game input의 우선순위를 명시한다. 입력 snapshot은 fixed tick에 timestamp를 포함하며 replay·prediction에서 재사용한다. text 입력과 key binding을 구분하여 IME를 지원한다.

게임플레이 스크립팅은 자체 커스텀 언어를 정식 수단으로 채택한다. Blueprint 및 Blueprint 유사 범용 visual scripting은 지원 범위에서 제외한다. 언어·컴파일러·VM·디버거와 Object·GC·RPC·프리팹 연동은 23절의 필수 구현 대상으로 정의한다. 머티리얼·애니메이션·VFX 전용 그래프와 컷씬 타임라인은 선언형 제작 도구이며 게임플레이 스크립팅 언어를 대체하지 않는다.

20.5 권한별 프레임워크와 프리팹 식별자

MatchRules는 서버 전용 규칙·spawn/respawn·match lifecycle, MatchState는 모든 참여자에게 복제할 경기 상태, PlayerState는 possession·재접속을 넘는 플레이어 상태다. PlayerAgent는 해당 connection의 입력·명령 경계이며 LocalPlayer는 로컬 viewport·device·focus를 소유한다. CameraDirector는 gameplay·spectator·cutscene control layer를 합성한다. pause·time dilation·spectator·split-screen을 clock/context별로 정의한다. S27

nested prefab의 주소는 PrefabInstanceGuid + 중첩 instance 경로 + NodeGuid + FieldId + ElementGuid다. 같은 NodeGuid를 가진 원본이 여러 번 중첩되어도 충돌하지 않는다. instance EntityGuid는 저장 시 보존하고 새 복제 시 재발급한다. runtime component 추가/제거·class default 갱신·script reload·network subobject 생성은 같은 instancing registry를 사용한다.

Behavior/State tree의 조건·action, ability task·UI binding·sequence event는 자체 언어의 검증된 함수와 typed data를 사용한다. 범용 event/action node로 게임플레이를 작성하는 우회 Blueprint 체계는 도입하지 않는다.

21 VFX·게임 UI·환경 시스템

VFX는 System/Emitter/Module/Parameter로 구성하고 CPU 입자 baseline 이후 GPU compute simulation·indirect draw로 확장한다. emitter spawn/death/event와 particle pool은 bounded counter를 사용한다. GPU collision은 depth/SDF 기반 시각 효과이며 authoritative gameplay hit 판정으로 사용하지 않는다. emitter ID와 RNG seed로 재현 범위를 정하고 network에는 개별 입자 대신 effect event와 parameter를 전달한다.

VFX Graph는 typed module IR에서 CPU bytecode·GPU shader를 생성한다. parameter snapshot과 per-emitter state를 분리하고 async compute 결과의 queue dependency를 RenderGraph에 선언한다. particle motion vector·reactive mask·bounds를 제공하여 Deferred·Forward·temporal upscaling에서 ghosting·잘림을 검증한다. simulation budget, scalability LOD, warmup, pooling, preview scrubber를 포함한다.

게임 UI는 웹 에디터 UI와 독립된 runtime UI다. retained widget tree, layout·style·input routing, focus·accessibility, text shaping, glyph atlas, image·vector primitive, clip·batch를 제공한다. 한국어 IME·Unicode·fallback font·localization·DPI scale을 지원한다. HarfBuzz/FreeType 계열은 adapter 후보로 검토하고 engine API에 해당 라이브러리 타입을 노출하지 않는다.

UI render는 출력 해상도에서 업스케일 이후 실행한다. widget Object와 render draw list를 분리하고 atlas page는 GPU fence 후 회수한다. large list virtualization·dirty layout·draw invalidation을 계측한다. native HUD·world-space UI·editor overlay의 hit test 경로를 구분한다.

환경 모듈은 terrain heightfield/tile LOD, foliage instancing·wind, sky·sun/time-of-day, fog를 P2로 구성한다. terrain render/collision/nav 변경은 하나의 편집 revision으로 cook dependency를 갱신한다. M7에는 water surface·underwater 원신호와 기본 volumetric fog를, M8에는 cloud·고급 대기/volume profile을 제공하여 26절의 합성이 입력 부재로 지연되지 않게 한다. groom·destruction·고급 fluid는 P3의 개별 budget·feature gate로 계획한다. 모든 후속 모듈은 태그 ·component·prefab·asset·profiling 계약을 사용한다.

21.1 전문 제작 데이터와 UI 편집

terrain은 non-destructive height/weight edit layer·spline 도로/하천·hole·brush·foliage paint·grass rule·runtime virtual texture와 bake를 단계별로 제공한다. spline은 별도 component·sample API·mesh deformation·collision recook을 갖는다. edit layer revision에 따라 collision/nav/HLOD/PCG 결과를 무효화한다. S31

geometry editor는 primitive/blockout·pivot/transform·boolean·mesh simplify·UV·collision 생성·vertex paint를 우선하며, Geometry Script는 자체 언어의 offline geometry API로 제공한다. dynamic mesh의 topology version과 GPU/collision cook lifetime을 구분한다. S39

WidgetAsset은 hierarchy·layout/style·animation·typed binding을 저장하고 전용 UI designer로 편집한다. 게임 동작은 script component/function으로 연결한다. property push·MVVM 계열 변경 통지, cached layout/draw·retainer render target·virtual list·font atlas eviction을 지원한다. 화면 HUD는 output 해상도에서, world-space widget은 3D scene의 depth/velocity·upscaler 정책 안에서 렌더링한다. S37

VFX는 ribbon/mesh/decal renderer·event·data interface·GPU readback latency·simulation cache를 명시한다. audio는 sound cue/variation·concurrency·dialogue/subtitle·submix/send·sample-clock scheduler·offline mixer를 추가한다. 유체·sparse volume cache·spatial audio의 지원은 별도 asset/decoder profile로 기록한다.

21.2 UMG 계열 인게임 UI의 정식 범위

인게임 UI runtime과 전용 UI Designer는 모두 P1 필수 기능이다. 엔진 배포물은 WebView 없이 HUD·menu·inventory·dialogue·settings UI를 실행한다. WidgetAsset에 hierarchy·layout·style·animation·binding을 저장하고, 동작은 C++ 또는 자체 언어의 WidgetController로 구현한다. Widget Blueprint·event graph·별도 visual scripting VM은 제공하지 않는다. 작업 흐름의 참고 자료는 UMG Designer다. S50

기본 widget는 Canvas/Overlay/Horizontal·Vertical Box/Grid/Wrap/Scroll/VirtualList, Text/RichText/Image/Button/Toggle/Slider/Progress/EditableText/Dropdown을 포함한다. slot별 anchor·alignment·padding·size rule·min/max·aspect·safe area·DPI·clipping·z-order를 정의한다. user widget 재사용·nested template·theme/style asset·named slot·default/instance override를 지원한다.

widget instance는 UI World/LocalPlayer scope의 owner를 갖고, dynamic 생성·attach·visibility·enabled·remove·destroy를 명확히 구분한다. controller delegate·animation·async asset lease는 owner 해제 시 취소한다. root widget이 사라져도 이벤트 연결 때문에 인게임 World 전체가 잔존하지 않게 검사한다.

21.3 전용 UI Designer와 preview

웹 UI Designer는 widget palette·hierarchy·drag/drop canvas·선택/정렬/분배·anchor/constraint gizmo·property Inspector·style/theme·binding panel·animation timeline·미리보기 장치를 제공한다. Undo/Redo·복제·reparent·다중 선택·copy/paste·asset reference·수정 충돌을 공통 Transaction에 연결한다.

미리보기는 editor DOM으로 비슷하게 재현하는 데서 종료하지 않고 실제 runtime layout/shaping/render 결과를 native preview에 표시한다. viewport 좌표·DPI 변환을 고정하여 drag/hit test가 동일 widget을 선택해야 한다. design-time sample ViewModel·상태·해상도·언어·gamepad focus를 바꿔 preview한다. 동작 이벤트는 binding된 자체 언어 source/함수로 이동한다.

UI animation은 opacity·색·render transform·선택 layout property를 keyframe·curve·clip으로 편집한다. gameplay clock·real clock·cutscene clock을 명시하고 pause/time dilation 영향을 선택한다. 공유 curve/evaluator 기반을 사용할 수 있으나 UI timeline의 local time과 world sequence의 timebase를 혼동하지 않는다.

21.4 Binding·입력·렌더링 규약

ViewModel은 reflected field·typed command·property-changed event를 제공한다. 매 frame 모든 getter를 호출하는 polling을 기본으로 하지 않는다. binding은 stable FieldId/type으로 cook 검증하며 null·collection diff·model 교체·구독 해제·thread dispatch를 처리한다. worker 결과는 Game/UI lane command로 적용한다. MVVM 계열 개념은 공식 자료와 대조하고 자체 언어에 연결한다. S51

입력은 hit-test grid·tunnel/bubble·mouse capture·keyboard/gamepad focus·tab/navigation·modal stack·drag/drop·tooltip·touch를 포함한다. LocalPlayer별 focus와 device routing을 분리하고 컷씬/PIE/editor focus를 control layer로 전환한다. EditableText는 IME composition·clipboard·grapheme 단위 cursor/selection·RTL/BiDi·password·validation을 지원한다. UTF-8 byte offset을 사용자 글자 index로 사용하지 않는다.

layout/paint invalidation·retainer target·virtual list·batch·clip·atlas budget을 profiler에서 표시한다. UI material은 별도 material domain과 입력 capability를 가진다. screen UI는 upscaling 이후 output 해상도에서, world-space UI는 scene geometry/texture와 hit test 정책으로 처리한다. HDR UI luminance는 OutputSurface 규약에 맞게 합성하고 TAA/DOF/blur에 HUD가 무단 포함되지 않게 한다.

21.5 UI milestone과 검증

Core Foundation 뒤 M3에 WidgetAsset·hierarchy·basic canvas·runtime preview를 구현한다. M4에 HUD/menu·기본 controls·input·자체 언어 callback·Undo/저장/cook을 완료한다. M5에 typed binding·virtual list·gamepad focus·UI animation, M6에 world widget·HDR·복잡한 localization/IME·전문 Designer를 완성한다.

필수 sample은 gameplay HUD, inventory virtual list, gamepad settings menu, dialogue/subtitle, editable chat input이다. editor preview와 packaged game의 layout·hit test·image를 같은 WidgetAsset로 비교한다. 500개 visible widget·100개/초 model update fixture에서 UI CPU p95 ≤1 ms·GPU p95 ≤0.75 ms를 초기 미측정 목표로 두고, virtual list는 전체 item 수보다 visible 수에 비례하는 비용을 검증한다. CPU-only layout/바인딩 검사와 offscreen GPU 픽셀 검사를 모두 제공한다.

22 웹 에디터와 네이티브 뷰포트

22.1 프로세스와 메시지 경계

Windows host는 WebView2, macOS host는 WKWebView 어댑터로 공통 TypeScript/React UI와 native RHI viewport를 배치한다. 웹 canvas에서 네이티브 화면을 매 프레임 video로 복사하는 구성을 기본으로 하지 않는다. 네이티브 viewport의 위치·크기·DPI·focus를 host가 조정한다. 투명 overlay와 창 겹침의 제약은 M3 prototype에서 검증한다. 두 host는 같은 editor

RPC·transaction·focus 계약을 구현하며 플랫폼별 child window/view·DPI·IME·접근성을 독립 검증한다.

Web UI ↔ Editor RPC ↔ EditorBackend ↔ World/Asset/Transaction 구조를 사용한다. JSON schema 기반 메시지로 시작하고 대형 thumbnail·trace는 blob/shared memory handle로 전달한다. 메시지에는 protocol version, request ID, document/world ID, expected revision, object stable ID가 있다. raw pointer·임의 native function name·임의 파일 경로 실행은 제공하지 않는다.

```
SetProperty(requestId, worldId, actorGuid, componentGuid,
            fieldId, value, expectedRevision, transactionId)
→ Success(newRevision) | Conflict(currentValue) | ValidationError
```

backend는 thread affinity·권한·field type·range·대상 세대·revision을 검증한다. mouse drag는 preview update를 병합하고 release 시 하나의 undo command로 commit한다. 구조 변경과 저장은 원자적인 transaction boundary를 갖는다. WebView content의 origin·navigation·host message를 검증하고 native host surface를 제한한다. S13

22.2 제작 기능과 transaction

M3는 Outliner, Inspector, Asset Browser, native viewport, transform gizmo, log, save/load, undo/redo를 제공한다. M4/M5는 component tree·tag picker·prefab hierarchy/override·play/stop·multi-object edit·asset dependency/ref finder를 완료한다. 머티리얼 에디터는 같은 command·selection·transaction 체계 위의 필수 전문 도구다. M3/M4에 14.5–14.9절 MAT0–MAT2를 완료하며 compiler만 구현한 상태를 완료로 인정하지 않는다. Animation, VFX, Audio, Terrain, Sequencer, profiler 전문 편집기도 이 편집 계약으로 확장한다.

Script Editor는 자체 언어의 문법 강조·자동 완성·정의 이동·진단·breakpoint·call stack·watch를 제공한다. 범용 node graph로 게임 로직을 작성하는 기능은 구현하지 않는다. 컷씬 편집기는 25절의 독립 핵심 제작 도구로 개발한다.

transaction은 객체 주소가 아닌 stable ID·FieldId·old/new value·asset revision을 기록한다. 생성·삭제·reparent·prefab structural override는 graph patch로 저장하고 undo 기간의 snapshot 메모리를 budget으로 제한한다. 삭제 객체를 undo용 root로 무제한 유지하지 않는다. 변환 drag·multi-edit·복합 component 추가는 atomic group으로 취급한다.

저장은 임시 파일→fsync/flush 정책→atomic replace→revision publication으로 수행한다. 외부 파일 변경·저장 충돌·프로세스 강제 종료를 감지하고 journal로 복구한다. source control 연동은 checkout/status/diff/asset lock을 adapter로 제공한다. multi-user editing은 단독 transaction·revision 충돌 해결이 안정화된 P3 이후 과제다.

PIE는 별도 World와 입력 context, 별도 network session으로 실행한다. Stop은 task·timer·delegate·audio·physics·render proxy를 정리하고 editor World만 유지한다. 1,000회 play/stop 이후 live object·resident memory·GPU resource count가 안정 구간으로 복귀해야 한다. asset/메타 layout 변경 시 안전하지 않은 live reload는 재시작을 요구한다.

22.3 Rocks 제작 템플릿과 사용자 카탈로그

Rocks는 반복적인 파일 생성·설정·콘텐츠 조립을 제작 템플릿으로 자동화한다. 이 절의 템플릿은 C++ template과 구분되는 선언·파일·에셋·설정 묶음이다. 생성 타입은 `rocks::Object`, `rocks::Entity`, `rocks::Component`처럼 명명하고 U/F/A/T/R 타입 접두사를 사용하지 않는다. 게임 동작은 C++와 커스텀 언어로 작성하며 Blueprint를 생성하지 않는다.

기본 카탈로그는 프로젝트, 런타임·도구 모듈, 테스트 fixture, 컴포넌트, prefab/variant, material, UI widget/화면, 컷씬/shot, input profile, import recipe, build/test/cook preset을 포함한다. 프로젝트에는 모듈·샘플·검증 명령을 함께 제공한다. Required/Optional capability를 선언하며 초기 미지원 플랫폼의 구현을 강제하지 않는다.

사용자는 선택한 파일·에셋·설정에서 템플릿을 저장하고 이름·설명·태그·미리보기·매개변수·의존 패키지·호환 엔진/스키마·버전을 등록한다. 유형·프로젝트·태그·capability 검색과 비호환 사유를 제공한다. 개인 로컬·프로젝트·팀 카탈로그를 지원하고 게시·업데이트는 명시 동작으로 수행한다. 출처·digest·이용 범위를 기록하며 오프라인에서도 고정 revision을 조회한다.

적용 방식	생성 결과와 갱신 계약
Copy	독립 소스/에셋을 생성한다. 출처만 보존하며 원본 변경을 자동 적용하지 않는다.
Linked	원본 revision과 override를 보존한다. 업데이트 제안·차이·충돌을 검토한 뒤 새 revision을 적용한다.
Preset	허용된 설정 필드에 값과 출처를 적용한다. 파일 복제나 실행 코드 삽입을 암묵 수행하지 않는다.

표 30

Linked는 기존 자산의 instance/override 의미를 재사용한다. 일반 소스에는 생성 이력과 명시 업데이트만 제공한다. 범용 객체 모델이나 모든 형식을 처리하는 동적 언어를 추가하지 않는다.

22.4 생성·갱신·원자적 적용 규약

manifest는 매개변수·기본값·조건·출력 경로·소유권·capability를 선언한다. plan에서 값 오류·이름 충돌·경로 이탈·중복 출력·누락 의존성을 검사하고 생성/변경/삭제 diff를 표시한다. 숨겨진 임의 shell hook은 금지한다. 추가 변환은 버전이 고정된 host transformer의 명시 입력·출력·실행 정책으로 제한한다.

새 에셋·객체에 stable ID를 부여하고 내부 참조는 clone-remap한다. 외부 참조는 선언에 따라 유지하거나 매개변수로 받는다. template-local ID→instance ID 대응표를 저장하여 업데이트 시 ID를 보존한다. 타입/필드 ID는 스키마 migration 규칙을 따른다.

새 reflected/script 선언을 생성하는 템플릿은 declaration-local ID마다 새로운 TypeId를 최초 plan에서 한 번 발급하고 journal에 보존한다. 동일 plan 재시도·업데이트·rename은 발급 ID를 유지한다. 기존 클래스를 참조하는 asset/prefab 복사는 원 TypeId를 유지한다. FieldId/RpcId는 declaring TypeId 내의 안정된 식별자이며 상속·외부 schema 참조와 새 내부 선언의 remap을 구분한다. TT03–05는 같은 template로 독립 타입 2개 생성·동시 meta compile, 재시도 ID 유지, 과거 asset 읽기를 검사한다.

업데이트는 이전 원본 base·사용자 결과 local·새 원본 incoming의 3-way 비교다. 자산은 필드·구조 ID로 병합하고 원본 삭제와 사용자 override의 충돌을 보고한다. 텍스트 충돌·binary 변경을 자동 덮어쓰지 않으며 해결 또는 항목별 보류 전 revision을 완료 처리하지 않는다.

사용자 코드와 재생성 코드를 경로·manifest로 구분한다. 생성기는 generated 파일만 교체하고 사용자 함수는 별도 구현 파일로 연결한다. 이전 hash와 현재 값이 다르면 diff로 사용자 수정을 보호한다. 무변경 출력은 rewrite하지 않아 compile/link·recook 전파를 방지한다.

적용은 격리 staging → validate → revision 재확인 → journal → commit 순서다. 여러 rename을 하나의 OS 원자 연산으로 가정하지 않는다. transaction·journal로 부분 노출을 통제하고 실패 시 이번 변경만 rollback한다. source-control 충돌·프로세스 중단·디스크 부족에도 복구 가능해야 한다. 작업 ID와 적용 이력으로 재시도 중복을 방지한다.

22.5 헤드리스·도입 순서·완료 기준

예정 CLI는 `rocks template plan`, `rocks template apply`, `rocks template validate`를 제공한다. plan은 고정 template revision·매개변수·대상 base revision으로 변경 manifest를 출력하고, apply는 동일 plan을 재검증한 뒤 적용한다. 대화형 응답이 없는 경우 충돌을 오류와 JSON 보고서로 반환한다. validate는 schema·참조·코드 생성 규약과 선택된 빌드/에셋 검사를 수행한다. CLI와 에디터는 같은 planner·validator·applier를 사용하며 headless에서 window·GPU·WebView를 요구하지 않는다.

첫 Core 작업 C00에는 Python host 기반 최소 scaffold만 둔다. Core target·테스트 runner·모듈 manifest의 반복 입력을 줄이는 제한된 생성기이며 Rocks 런타임·VM·웹 에디터에 의존하지 않는다. R0/R1에 전체 템플릿 서비스를 요구하지 않는다. Foundation 이후 M3에 project/module/test·카탈로그·CLI와 기본 material template, M4에 material 편집 통합

·component/prefab/UI/input/import·transaction, M5에 고급 material/cutscene/build preset·사용자 저장·Linked 업데이트를 완성한다. 기본 material template을 M5까지 유예하지 않는다. 전문 편집 기능의 실제 완성 시점은 해당 시스템의 gate를 따른다.

게임 배포물에는 생성·cook된 코드와 데이터만 포함한다. 카탈로그·planner·journal·Python·transformer를 런타임 필수 의존성으로 만들지 않는다. 템플릿은 제작 도구이며 런타임 기능 구현을 대신하지 않는다.

시험 ID	필수 완료 기준
TT01	새 checkout에서 C00 scaffold로 CPU-only Core/test 구성, 상위 시스템 의존 0건
TT02	필수 템플릿 유형별 유효/무효 매개변수·capability·독립 header/asset 검증
TT03	같은 plan 재시도 시 중복 ID/파일 0건, 무변경 출력 hash·timestamp 유지
TT04	nested clone·공유 child·순환·외부 참조의 remap 정합, 기존 ID 보존
TT05	3-way 변경/삭제/override 충돌 검출, 미해결 local 변경의 덮어쓰기 0건
TT06	각 commit 단계의 실패·중단 주입 후 rollback/복구, 무관한 사용자 파일 변경 0건
TT07	개인/프로젝트/팀 저장·검색·버전 고정·오프라인 조회 및 호환 오류 검증
TT08	에디터/CLI 동일 plan의 의미적 결과 일치, shipping에서 템플릿 도구 의존 0건

표 31

TT01은 C00이다. M3/M4는 TT02의 준비된 template 유형·TT03/04/06/08의 Copy/Preset·ID/remap·복구·CLI 사례를 요구한다. TT07의 배포 카탈로그 조회는 M3/M4, 사용자 저장/팀 버전 관리와 TT05의 Linked 3-way 갱신은 M5 필수다. 동일 시험 ID 안에서도 case별 required/deferred·owner gate를 기록한다. 계획 개수와 생성·갱신·복구 검증 완료 수를 분리한다.

22.6 필수 에디터 내 실행(PIE)

공유 물리 DeviceHub와 action 소비 scope를 분리한다. Host는 opaque InputConsumerScope를 session·route generation에 연결하고 종료 시 admission을 닫는다. InputCore에는 World/Widget 타입을 도입하지 않는다. AdvanceInput과 QueryAction/CopyActionEvents는 InputReadKey의 scope/generation·user·update domain·clock epoch·tick을 같은 history key로 사용한다. UIRealTime·gameplay fixed/frame·replay 영역의 같은 숫자 tick을 합치지 않는다. route 전환·재시작에는 새 scope generation 또는 clock epoch를 사용하고 기존 key를 재해석하지 않는다. 종료 scope에는 ScopeExpired, 보존 구간 밖에는 HistoryExpired를 반환하며 다른 scope의 최신 결과를 대체 반환하지 않는다. 종료 시 cancel batch의 소비·명시 폐기까지 기록한 뒤 scope를 회수하며, 지연 조회자가 소멸 scope의 terminal event를 받지 못하는 경우에도 expired 오류를 명시한다.

PIE는 Rocks의 필수 기능이다. Core/Object Foundation 이후 M3에 기본 실행, M4에 Play·Pause·Resume·SingleStep·Stop과 복수 session 격리를 완료한다.

EditorWorld는 편집 원본, PlaySession은 독립 실행 소유자다. Play는 고정 revision의 graph를 복제하여 `rocks::Object` · `rocks::Entity` · `rocks::Component` 의 runtime ID를 발급하고 참조를 remap한다. 원본 stable ID 대응표를 보관하며 가변 객체·script state는 공유하지 않는다. 불변 asset만 revision lease로 공유하고 material instance·animation state·streaming 사용권은 분리한다. 복제 실패는 미공개 graph 전체를 rollback한다.

각 session은 SessionId/WorldEpoch, VM globals·coroutine·root, clock·RNG, 입력 context·player routing, network endpoint·replication scope를 갖는다. physics·audio·timer·delegate·task·render proxy도 session owner로 추적한다. 전역 Registry에서도 타 session handle·callback을 거부하며 네트워크 연결은 명시 group만 허용한다. 2개 이상 session의 pause·stop·port·저장 경로를 격리한다.

Pause는 simulation tick만 정지하고 UI·진단·IO 완료·cleanup은 계속 처리한다. Resume는 벽시계 적체를 무제한 보충하지 않는다. SingleStep은 pause 중 설정된 simulation dt로 fixed tick을 정확히 1회 실행한다. dilation 0에서도 명시 step이 가능하며 accumulator의 추가 tick은 금지한다. variable/fixed update·time dilation은 session별 clock 계약을 따른다. focus·mouse capture는 session token으로 관리하고 전환·Stop·focus loss에서 해당 token만 반환한다.

live edit는 선택한 PlaySession의 transaction이다. Apply Simulation Changes는 선택한 허용 field의 diff·원본 revision·schema·ID를 검증한 명시 transaction이며 자동 병합하지 않는다. 충돌·취소·Undo를 지원하고 Stop은 원본을 저장하지 않는다. 임시 runtime 객체·권한 상태·VM stack은 반영에서 제외한다. native C++ 변경은 기본 재시작, 호환 asset/script는 안전 경계의 revision 교체, layout 변경은 session 재생성으로 처리한다.

Stop은 비동기 전이로 일반 작업 차단→취소·논리 종료→teardown drain→task/IO/coroutine·backend fence/ACK 완료→root/lease 해제·최종 GC→추가 cleanup drain→session 제거 순서다. cleanup의 scheduler·metadata·writer를 마지막 사용까지 유지한다. 소유 기록이 모두 닫혀야 Stopped로 전이하며 timeout/crash는 실패로 남긴다. 강제 free로 누수를 감추지 않는다. editor 소유·공유 cache의 budget과 기준선은 별도 기록한다.

시험	필수 판정
PI01	원본 불변·clone/remap·실패 rollback
PI02	pause/dilation 조합·SingleStep 정확히 1 tick
PI03	2개 이상 session의 VM·입력·network·clock·Stop 격리
PI04	명시 변경 반영·충돌·Undo·취소·자동 병합 금지
PI05	지연 IO/fence·재진입·취소·terminal 1회
PI06	1,000회 Play/Stop·실패 주입 후 session 객체/root/task/lease 0, 메모리·GPU 기준선 복귀

표 32

누수 없는 종료는 필수 기준이나 보편적 무오류의 증명은 아니다. 미종료 기록·증가 추세·미확인 cache는 성능보다 먼저 수정한다.

23 자체 게임플레이 언어·컴파일러·VM

23.1 언어 범위와 구현 단계

게임플레이는 C++와 자체 정적 타입 언어로 작성한다. Blueprint, 범용 visual scripting, Lua/C#를 기본 게임플레이 언어로 채택하지 않는다. 언어 이름과 문법은 Foundation Gate 이후 L0/M3에서 확정하며 아래 예시는 요구 기능을 나타내는 가상 문법이다.

```
component DoorLogic : ScriptComponent {
    @edit @save var openAngle: Float = 1.57;
    @edit @save var target: WeakRef<TransformComponent>;

    on Interact(actor: ObjectRef<Entity>) {
        if actor.tags.has(tag("Character.Player")) {
            if let scene = target.pin() {
                scene.setRotation(angle(openAngle));
            }
        }
    }
}
```

전체 언어 범위는 module/import, 명시 타입·지역 타입 추론, value struct·enum, 함수·interface, control flow, array/map, ObjectRef/WeakRef/AssetRef, component event, attribute, async/await, Result를 지원한다. 임의 포인터 연산·unchecked cast·native heap 접근·사용자 정의 allocator·동적 eval은 초기 언어에서 제외한다. generic은 기본 container에 필요한 범위부터 지원하고 C++ 전체 문법을 복제하지 않는다.

구현은 lexer/parser → AST → 이름/타입 검사 → typed IR → verified bytecode → interpreter VM → debugger 순서다. LLVM 기반 AOT 또는 JIT는 VM 성능 측정 이후 선택한다. 초기부터 복수 backend를 개발하지 않는다. C++ native binding은 meta compiler가 생성한 허용 API 테이블을 통해 호출한다.

단계	산출물	완료 기준
L0 · Foundation Gate 이후 M3	문법·타입·오류·모듈·수명·실행 모델 규격, conformance fixture	Core/Object 계약에 맞는 언어 규칙 확정
L1 · M3	compiler, verifier, interpreter, native call	조건문·함수·객체 필드·배열·오류 처리와 바인딩 테스트
L2 · M3/M4	ScriptComponent, event, prefab 기본값, editor 진단	상호작용·문·카메라·컷씬 trigger를 C++ 수정 없이 제작
L3 · M5	async, RPC, save schema, debugger, 안전한 reload	취소·GC·replication·중단점·migration 통합 검증
L4 · M6 이후	profiler·최적화 IR, 선택 AOT	hot function 비용을 근거로 backend 투자 결정

표 33

23.2 GC·동시성·오류 규약

script reference object는 ObjectRuntime의 TypeInfo와 GC 추적 체계에 편입한다. VM용 두 번째 독립 tracing heap을 초기안으로 도입하지 않는다. VM stack·global·suspended coroutine은 등록된 root provider가 active reference slot을 열거한다. verified bytecode의 root map 또는 보수적으로 전체 tagged reference slot을 정확히 스캔하는 기준 구현을 사용한다. native bridge는 ObjectHandle을 검증한 뒤 같은 Game lane에서만 객체 필드를 수정한다.

script field·container reference 변경은 ObjectRef와 같은 insertion barrier를 사용한다. await 이후에도 필요한 Object는 coroutine root로 유지하고 owner 파괴·World 종료 시 coroutine을 취소한다. native pointer·Borrow·Span을 yield 너머에 보관할 수 없도록 타입 검사한다. default 실행 lane은 Game이며 worker script는 immutable value 데이터와 thread-safe native API만 사용한다.

bytecode verifier는 type, branch target, stack depth, local initialization, native API index, reference store를 검사한다. VM은 instruction·recursion·memory budget과 cancellation poll을 제공한다. 무한 loop는 프레임 전체를 점유하지 않도록 interrupt하며 script error에는 source range·call stack·entity·World·tick을 기록한다. 엔진의 손상 없이 해당 script instance 또는 transaction을 종

료하는 정책을 정한다.

script reload는 함수 본문 교체부터 지원한다. 실행 중 frame은 이전 code revision을 유지하고 그 revision은 frame/coroutine이 종료될 때 반환한다. layout 변경은 stable FieldId migration과 기본값 적용 후 safe boundary에서 교체하며 불가능하면 해당 instance 또는 PIE를 재시작한다. 변경된 script가 오래된 serialized prefab·save·RPC와 호환되는지 cooker가 검사한다.

네트워크 RPC는 script annotation에서 static manifest를 생성한다. client가 임의 함수 이름으로 server 코드를 호출할 수 없게 한다. prediction·replay 대상 함수는 deterministic subset으로 제한하여 wall clock·unordered iteration·비고정 RNG·임의 IO를 금지한다. native physics를 호출한 전체 script가 자동으로 결정적이 되는 것은 아니다.

23.3 제작 경험과 검증

웹 Script Editor는 compiler와 동일 parser/type schema를 사용하는 language service를 연결한다. completion·rename·find references·hover·diagnostic·source map을 제공한다. debugger는 breakpoint·step·call stack·watch·native boundary 표시를 지원하고 멀티플레이에서 어느 process/world를 제어하는지 표시한다. 중단 시 physics/audio/Render와의 pause 정책을 명시한다.

conformance corpus는 정상 코드뿐 아니라 잘못된 타입·미초기화·순환 import·잘못된 bytecode·GC 경합·reload·취소를 포함한다. AST/bytecode fuzzer와 C++ oracle fixture를 사용한다. allocation·native call·GC barrier·task 대기 비용을 profiler에 표시한다. 라이브러리 API 문서와 script binding 문서는 같은 TypeInfo에서 생성한다.

23.4 ABI·오류·재구성의 경계

MetaCompiler는 native binding schema를 먼저 생성하고 ScriptCompiler는 이를 읽어 TypeId/FieldId/RpcId와 bytecode를 생성한다. 두 compiler는 같은 schema registry를 사용하고 서로의 generated source를 다시 입력하는 cycle을 만들지 않는다. ScriptRuntime은 opaque script class·field storage를 다루며 World 관련 native API는 WorldIntegration이 등록한다.

nullable reference는 pin/resolve 결과 확인 없이 호출할 수 없고 tag literal은 compile/cook 시 dictionary에 검증한다. cooperative VM 중단은 opcode·native call 경계에서만 수행한다. native call 안의 lock 보유 코드를 강제 선점하지 않으며 native API마다 lane·최대 작업·취소 가능 여부를 선언한다.

script error·task cancellation은 이미 외부에 적용한 damage/IO/RPC를 자동 rollback하지 않는다. 원자성이 필요한 편집·능력 판정은 staged command transaction을 commit 전에 검증한다. reload는 code뿐 아니라 TypeInfo·binding table·source map의 revision lease를 유지한다. layout migration 시 coroutine root map을 검증하며 불가능한 paused frame은 명시 취소 후 재시작한다.

24 Unreal 참고 체계의 추가 도입

24.1 채택 원칙과 우선순위

Unreal에서 참고할 대상은 작업 흐름·데이터 규약·검증 체계다. 이름이나 node UI를 일괄 복제하지 않는다. 특히 Blueprint 기반 확장 지점은 **C++ interface·자체 언어·typed data asset·prefab**으로 설계한다. 아래 항목은 기존 절의 기능을 실제 제작 체계로 연결하는 추가 범위다.

참고 체계	자체 엔진의 구현 범위	우선순위·완료 단계
Unreal Subsystem / World context	Engine·GameSession·World·LocalPlayer 단위 서비스, 명시 init dependency·teardown	P0/P1 · M2–M4
Gameplay Tags / GAS	attribute set, effect·stacking·duration, ability task, cue, prediction key	tag P1 M4, 능력 체계 P2 M6

참고 체계	자체 엔진의 구현 범위	우선순위·완료 단계
Asset Manager / Data Asset	primary asset 유형, named bundle, load lease, dependency audit, typed table·registry	P1 · M3–M5
Enhanced Input / Common UI 계열	action/context·rebinding·입력 장치 전환·focus·modal stack	P1 · M4–M5
Game Features / Plugins	feature lifecycle, component/input/data 등록 token, content mount	P2 · M6
Sequencer / cinematic tools	timeline·shot·camera·track·subsequence·runtime evaluator·render job	P1 핵심 M4, 완성 도구 M6
Control Rig / IK Retarget	rig data·constraint·IK·retarget profile, animation editor preview	P2 · M6–M7
StateTree / EQS / Smart Objects	상태 평가·공간 query·상호작용 슬롯 예약·AI budget	P2 · M6–M7
Mass 계열 대량 처리	NPC/traffic/foliage용 SoA chunk·processor, Entity presentation bridge	P2 · M7
Data Layers / HLOD / Level Instance	cell layer·prefab level instance·HLOD cook·편집 preview	P2 · M7
PCG 계열	seed·typed point/attribute 데이터·생성 규칙·incremental rebuild·bake	P2/P3 · M7–M8
MetaSounds / Quartz 계열	typed DSP graph, parameter automation, sample clock 예약	P2 · M6–M7
Device Profile / Scalability / CVar	품질 계층·caps·예산 profile, override 우선순위·재현 설정	P0부터, P1 완성 M5
Localization / Automation / Insights	text ID·번역 pipeline, headless commandlet, 통합 trace·회귀	P0부터 전 단계

표 34

24.2 게임플레이·콘텐츠 확장 계약

능력 체계는 attribute base/current value, modifier aggregation, effect priority·stack rule·duration, tag requirement, cooldown·cost, cue event, cancel reason을 정의한다. 여러 effect의 합산 순서와 snapshot/live capture 의미를 고정하고 저장·reconnect 복구를 검증한다. client-predicted ability는 prediction key로 임시 effect를 authoritative 결과와 병합·철회한다. 구현 방향의 참고 범위는 Unreal GAS의 능력·속성·효과 개념이며 script 실행은 자체 언어로 제공한다. S14

ScopedService은 scope별로 하나의 명시 owner를 갖고 constructor에서 service locator를 호출하지 않는다. init dependency DAG와 teardown 순서가 있다. world 종료 시 해당 scope의 task·delegate·timer·asset lease를 정리한다. timer는 world time/real time 구분·owner token·pause·취소를 지원한다. interface와 weak delegate는 Object 수명 확인 후 호출하고 dynamic multicast의 iteration 중 해제·재진입 정책을 정의한다.

PrimaryAsset는 예를 들어 CharacterDefinition·MapDefinition·CutsceneDefinition처럼 직접 로드할 진입 asset이다. bundle은 Gameplay, Visual, Audio, Cinematic 등 의미별 dependency 집합이다. Asset Manager는 bundle별 LoadTicket/AssetLease와 용량 audit을 제공한다. DataAsset와 DataTable은 reflected schema·stable row ID·validation rule을 사용하고 script class default·prefab default와 우선순위를 구분한다. Asset Manager 개념은 공개 문서에서 참고한다. S15

GameFeature는 Installed → Registered → Loaded → Active의 상태와 역방향 전이를 갖는다. component·input context·tag rule·UI·asset registry 등록마다 해제 token을 기록한다. feature 해제는 의존 feature와 진행 중 ability·cutscene·task를 정리한 뒤 수행한다. native module 코드가 참조되는 동안 동적 unload하지 않는다. 독립 기능 활성화라는 작업 흐름을 참고한다. S16

Config는 engine default → platform → project → device profile → user → session override의 우선순위를 사용한다. 런타임 변경 가능한 CVar와 재시작이 필요한 CVar를 구분하고 값의 출처를 UI에 표시한다. quality profile은 RenderPath, AA/MSAA, upscaler, GI, shadow·streaming budget을 함께 검증한다. 개별 GPU 기능이 없을 때 설정 조합 전체가 유효하게 낮아져야 한다.

24.3 전문 도구와 후속 범위

SmartObject는 interaction definition·slot·예약 token·접근 경로·중단 사유를 정의한다. slot owner가 unload되면 예약·AI task를 취소한다. EQS는 후보 생성·필터·점수·async query를 budget 안에서 수행한다. Mass형 데이터는 worker가 chunk를 처리하고 actor/component는 표현·interaction에 필요한 개체에만 연결한다.

PCG는 seed와 input revision에 대한 재현 가능한 point/attribute 결과를 생성한다. editor preview와 bake를 구분하고 World Partition cell 경계를 넘는 생성물에 stable ID를 부여한다. runtime PCG는 CPU·asset prefetch·replication 권한을 별도로 제한한다. graph 표현을 제공하더라도 범용 게임플레이 코드의 node 언어로 확대하지 않는다.

Control Rig는 본 constraint·IK·space switching과 animation key 생성을 위한 전용 rig 도구로 정의한다. 대화·컷씬은 animation clip, additive pose, facial morph curve, look-at constraint로 제작하고 script는 state/event 제어만 담당한다. M6의 basic take recorder는 transform·camera·event를 기록한다. M7 이후 Live Link형 mocap·full-body IK·motion matching의 개별 gate를 확장한다.

MetaSounds형 DSP graph도 block-size·sample rate·실시간 안전성·node state를 검증하는 오디오 전용 도구다. animation·material·VFX·DSP graph, PCG graph, 컷씬 timeline은 사용자가 명시한 Blueprint 제외 정책과 양립하는 전용 데이터 편집기다. 게임플레이 조건문·이벤트 조합은 자체 언어로 작성한다.

25 컷씬 제작기·런타임 시퀀스·영상 출력

25.1 제품 범위와 타임라인 데이터

컷씬 제작은 기본 게임플레이와 같은 P1 우선순위다. M4에서 카메라·샷·캐릭터 애니메이션·오디오·자막·이벤트가 있는 컷씬을 제작하고 게임 안에서 재생한다. M4에서 기본 nested sequence를 함께 제공하고 M6에서 고급 nesting·curve editor·take/영상 출력을 확장한다. 공개 Sequencer의 멀티트랙·비선형 편집 작업 흐름을 참고하되 event와 condition은 자체 언어의 검증된 함수를 호출한다. S17

SequenceAsset은 rational display rate, 정수 tick resolution, track/section/channel, keyframe·tangent·easing, binding table, subsequence, marker, pre-roll, event policy를 갖는다. 24, 25, 30, 60, 24000/1001 fps 등에서 시간을 float frame 누적으로 계산하지 않는다. curve는 scalar Hermite·step·linear, 회전 quaternion interpolation과 명시 Euler 편집 규약을 제공한다.

트랙	제작 기능	runtime 계약
Camera / Shot	컷·blend·camera rail·look-at·filmback·focal length·DOF·exposure	camera authority·cut flag·upscale history reset
Transform / Property	actor/component·light·material·visibility·tag parameter	reflected writable field·restore/keep 상태
Skeletal / Facial	clip·blend·root motion·IK weight·morph curve	pose sampling·notify 정책·physics authority

트랙	제작 기능	runtime 계약
Audio / Dialogue / Subtitle	waveform·cue·localization key·speaker·mix automation	sample clock 동기·streaming ready·언어별 길이
Spawn / Binding	임시 prefab 생성·기존 Entity 연결·binding override	stable ID·generation·GC root·종료 정리
VFX / Environment	emitter·light·weather·time-of-day	pre-roll·seed·scrub 가능한 상태
Event / Script	marker·게임 이벤트·커스텀 언어 함수	event ID·방향·seek/skip 정책·중복 방지
Subsequence / Shot	nested edit·offset·time scale·take	cycle 금지·시간 변환·충돌 우선순위

표 35

binding은 EntityGuid/ComponentGuid, 20.5절의 완전한 PrefabNodeAddress, 또는 명시 binding role을 사용한다. prefab 주소는 instance GUID·중첩 instance 경로·NodeGuid를 포함하며 NodeGuid 단독 저장을 금지한다. 상대 주소는 명시 binding root/role을 기준으로 해석하고 runtime 결과는 WorldId/Epoch·ObjectHandle generation을 검증한다. actor tag 자동 검색은 여러 후보가 있을 때 모호성을 오류로 표시한다. Spawnable은 sequence가 소유하는 임시 actor, Possessable은 기존 world actor에 대한 임시 제어다. 둘의 종료 수명과 상태 복원을 구분한다.

25.2 평가기와 게임플레이 통합

```
enum class SeekPolicy {
    SilentPreview,
    ReplayEvents,
    RuntimeSeek
};
enum class FinishPolicy {
    RestorePreAnimatedState,
    KeepFinalState
};

struct SequencePlayDesc {
    AssetId sequence;
    SequenceBindings bindings;
    FinishPolicy finish;
    TickId synchronizedStart;
};

SequenceHandle PlaySequence(World&, SequencePlayDesc);
void SeekSequence(SequenceHandle, SequenceTime, SeekPolicy);
void StopSequence(SequenceHandle, SequenceStopReason);
```

editor preview와 runtime은 같은 evaluator를 사용한다. seek 가능한 property·pose·camera는 시간의 함수로 평가하고, irreversible gameplay event는 분리한다. SilentPreview에서 스크러빙·뒤로 재생·undo 시 damage·save·RPC·asset deletion 같은 side effect를 실행하지 않는다. runtime event는 sequence instance path+event ID+loop iteration+방향을 ledger로 기록하여 같은 asset의 중첩 재생도 구분한다. loop 재실행과 seek replay는 명시 정책에 따른다.

pre-animated state는 대상 identity+FieldId/control channel별 첫 owner 진입 시 공통 base로 저장한다. owner 집합에 track priority·blend type·weight·stable instance ID를 적용하며 중간 owner 종료는 token만 제거하고 남은 owner를 재평가한다. 개별 sequence가 시작 때 읽은 값을 종료 때 임의 복원하지 않는다.

Restore/KeepFinalState는 채널별 resolver가 처리한다. Keep owner 종료 시 최종 기여와 priority를 pending commit으로 기록하고 마지막 owner가 해제될 때 가장 높은 priority·stable ID의 Keep commit을 base에 반영한다. Keep commit이 없으면 공통 base를 복원한다. 소유 중 gameplay 쓰기는 해당 채널의 UpdateBase API로만 허용하며 resolved output 직접 덮어쓰기는 거부한다. 종료·skip·cancel은 같은 release 경로를 정확히 1회 수행하고 대상 파괴·WorldEpoch 변경 시 stale 상태를 쓰지 않는다. input/camera/physics의 임시 제어권 채널도 같은 owner 집합을 사용하되 AlwaysRestore와 token auto-release로 고정한다. KeepFinalState는 명시 허용한 transform/property 값에만 적용하며 입력 잠금·camera possession·physics mode의 임시 제어권을 잔존시키지 않는다. camera pose 값 유지와 camera 제어권 반환도 서로 다른 정책이다.

A/B 시작 후 A/B 또는 B/A 종료, Restore/Keep 혼합, UpdateBase·skip·cancel·World unload를 검사한다. 동일 prefab/sequence를 2회 중첩하고 rebase·PIE 복제·save/load 후 대상과 event ledger가 구별되는지도 검증한다. camera cut에서는 temporal upscaler·TAA·motion blur history를 reset하고 shot blend에서는 연속 motion을 생성한다.

SequencePlayer는 필요한 asset bundle·World cell·animation·audio·VFX를 prefetch한다. 필수 항목이 준비되기 전 start를 지연하거나 명시 loading gate를 적용한다. 긴 sequence는 shot look-ahead window로 스트리밍하고 GC root·AssetLease를 필요한 구간만 유지한다. camera teleport·offscreen cast·새 shot 첫 frame에 geometry·shadow·GI가 갑자기 미상주하지 않게 pre-roll budget을 제공한다.

물리·AI·입력 제어는 gameplay와 독립된 control layer token으로 잠시 대체한다. 종료는 해당 control token을 해제하고 남은 owner를 재평가한다. 마지막 임시 제어 owner가 종료되면 AlwaysRestore로 base mode를 복원한다. root motion과 kinematic cinematic transform의 충돌을 track 설정에서 거부하거나 bake 정책으로 해결한다. script는 Play/Stop/Skip, marker 대기, completion result를 사용한다. 노드 기반 event graph는 제공하지 않는다.

멀티플레이 컷씬은 서버가 sequence asset/revision, binding, start tick, skip rule을 승인한다. client는 동기화된 timeline의 시각·음향을 평가하고 gameplay event는 서버만 확정한다. 늦은 참여는 현재 state로 seek하며 이미 실행된 reward/damage를 재발행하지 않는다. 개인 컷씬과 전체 플레이어 컷씬을 별도 정책으로 정의한다.

25.3 웹 제작 UI와 협업 데이터

타임라인은 track tree, clip section·trim·split·ripple 선택, keyframe, curve editor, snap, marker, zoom, frame-accurate playhead, shot thumbnail, waveform을 제공한다. viewport camera piloting, auto-key·manual-key, property key 생성, animation drag/drop, multi-track selection을 지원한다. 모든 편집은 stable track/key ID와 transaction을 사용하여 Undo/Redo·저장·충돌 검출에 연결한다.

dialogue는 localization key와 언어별 audio/subtitle timing을 분리한다. facial morph·viseme·look-at·gesture track을 지원하고 자동 lip sync는 선택적인 오프라인 도구로 후속 추가한다. take recorder는 입력·transform·pose·camera·event를 실제 timestamp로 기록한 뒤 sequence key로 bake한다. 외부 DCC camera/animation import·export는 동일 좌표·timebase 규약을 사용한다.

25.4 영상 출력과 검증

RenderJob은 sequence revision, asset manifest, deterministic seed, 출력 FPS·해상도·색 공간, frame range, warmup, motion-blur shutter, sample count를 고정한다. offline capture는 fixed-time evaluation과 필요 시 subframe sampling을 사용한다. 기본 출력은 PNG(SDR)·OpenEXR(linear HDR) image sequence와 별도 PCM audio이며 컨테이너 인코딩은 후처리 작업으로 연결한다.

멀티샘플 누적은 temporal AA/upscaler의 자체 history와 중복 누적하지 않도록 별도 capture profile을 제공한다. 기본 품질 기준 출력은 native/high-resolution render이며 vendor temporal upscaler 사용은 명시 job option과 비교 기준을 갖는다. frame generation은 deterministic offline frame 산출에 사용하지 않는다. cut·warmup·streaming·GI history 초기화가 재시작한 작업에서도 동일해야 한다.

오디오 파일은 rational frame range에 대응하는 sample 구간을 정확히 산정하고 device callback 시간에 의존하지 않는다. job 중단 시 완료 frame manifest를 기록하고 동일 설정으로 resume한다. output 공간 부족·누락 asset·shader compile 실패는 frame 성공으로 보고하지 않는다.

필수 데모는 **2분, 10개 이상 shot, 캐릭터 2개, nested sequence 1개 이상, dialogue·subtitle·VFX·script marker·skip·게임 복귀**다. preview/runtime의 camera·pose·event 결과를 비교하고 24/30/60 fps·앞/뒤 seek·loop·중단·late join·binding 대상 파괴를 시험한다. M6 목표는 10분 sequence에서 audio/video sync 오차 ±20 ms 이하, 기준 시점 pose 차이 tolerance 이내, skip 후 입력·카메라·physics mode 복구 누락 0건이다. 이 값은 미측정 초기 목표다.

25.5 합성용 출력·타일·작업 재개

capture는 Beauty·alpha·linear depth·world/view normal·motion vector·stable ObjectId/MaterialId AOV를 지원표로 관리한다. ID pass의 AA·blur·transparency 의미를 정의하고 기본 ID mask와 선택 Cryptomatte-compatible output을 구분한다. 모든 조명 항을 손실 없이 재조립할 수 있는 AOV를 제공한다고 가정하지 않는다. 참고 Movie Render Queue 자료도 pass별 한계를 명시한다. S34

high-resolution tile capture는 overlap/overscan과 lens/DOF/bloom의 sample 범위를 반영한다. screen-space GI/SSR·history가 tile 경계를 넘는 경우 full-frame context·제한된 지원 조합·별도 offline 경로 중 검증된 정책을 사용한다. 각 sample마다 simulation을 중복 진행하지 않고 fixed world evaluation과 shutter sample을 분리한다.

resume는 frame 번호뿐 아니라 shot pre-roll·GI/VFX cache·RNG state·audio sample range를 복구한다. 재생성 결과는 동일 장치/driver/build에서 tolerance를 비교하며 GPU bitwise 일치를 보장하지 않는다. partial EXR/PNG는 완료 manifest에 포함하지 않는다. headless capture와 editor capture는 같은 evaluator·resolved quality profile을 사용한다.

26 하이엔드 포스트이펙트 전체 기능군

26.1 기능 카탈로그와 지원 범위

포스트이펙트는 기본 효과 몇 개로 종료하지 않고, 하이엔드 게임·시네마틱 제작용 전체 기능군을 제품 범위에 포함한다. '전부'의 완료 범위는 아래 기능 카탈로그와 테스트 장면으로 관리한다. 업계 전체에 고정된 단일 목록은 없으므로 새로운 제작 요구는 같은 plugin/pass 계약으로 추가한다. 각 효과는 Runtime·Editor·Cinematic Capture에서 파라미터와 지원 profile을 공개한다.

Unreal의 공개 post-process 목록, 후처리 material 확장, cinematic DOF, HDR 출력 문서를 기준 자료로 검토하였다. 아래 목록은 자체 엔진에 대한 추가 구현 요구이며 특정 제품의 완전 호환을 뜻하지 않는다. S22, S23, S24, S25

기능군	필수 지원 항목	구현·검증의 핵심
노출	manual EV·physical camera exposure, histogram/basic auto exposure, 속도별 adaptation, exposure compensation curve, local exposure	pre-exposure·실제 광량 단위·cut/teleport reset·역광/실내외 전환
Tone mapping / 색	filmic·ACES 계열 선택, 3D LUT, lift/gamma/gain·offset/power/slope, contrast·saturation·white balance·temperature/tint, shadow/midtone/highlight, gamut mapping	linear HDR 기준·LUT color domain·double tone map 방지·색상 chart
출력·색 관리	SDR sRGB, HDR10/PQ, scRGB profile, paper white·peak brightness·UI luminance, OCIO 연계 capture 변환	display capability·metadata·OS HDR 전환·출력별 reference
Bloom / Glare	다단계 standard bloom, convolution bloom, threshold/soft knee, anamorphic streak, starburst·glare, lens dirt, halation·diffusion	HDR emissive·에너지·kernel normalize·upscale 전후 해상도

기능군	필수 지원 항목	구현·검증의 핵심
렌즈·필름	lens flare/ghost, chromatic aberration, vignette, film grain, barrel/pincushion distortion, calibrated STMap, anamorphic squeeze·bokeh, lens breathing	camera profile·center/edge·overscan·grain 해상도·TAA 순서
Depth of Field	cinematic bokeh DOF, aperture blade·shape·rotation, near/far gather/scatter, focus distance·f-stop, rack focus·autofocus, 가벼운 Gaussian 경로	CoC·foreground dilation·translucency·다중 레이어·camera cut
Motion Blur	camera+object velocity, shutter angle·offset, tile/neighbor max, per-object 제외·스케일, radial/zoom blur, 선택 rolling-shutter 근사	skinning/morph motion·disocclusion·UI 제외·subframe capture
AA / 재구성	TAA·TAAU, FXAA·SMAA 계열 spatial AA, Forward MSAA, temporal/spatial upscale, FSR·DLSS·XeSS, CAS 계열 sharpen	jitter·depth/motion·reactive mask·중복 sharpen·history 관리
AO / 접촉	SSAO·GTAO 계열 horizon AO, bent normal, multi-bounce 보정, contact shadow 합성, 선택 ray-traced AO	surface normal·thin geometry·distance falloff·필터·halo
반사·GI 후처리	SSR resolve, SSGI resolve, RT GI/reflection 합성, roughness filter, reflection probe fallback, specular occlusion	screen trace 누락·roughness별 품질·이력·표현 차이
디노이징	spatial bilateral·temporal variance, diffuse/specular 분리, disocclusion·firefly 억제, 선택 vendor ray-reconstruction adapter	raw signal 보존·ghosting·기하 경계·material ID·reset
안개·대기 합성	height/volumetric fog composite, aerial perspective, light shafts·god rays, cloud/atmosphere 합성	depth·transparent ordering·half-res edge·색 공간
물·날씨·왜곡	underwater absorption/scattering·fog·distortion, heat haze·refraction, rain drops·wet lens·frost, shockwave·damage distortion, 선택 caustic composite	mask·depth·particle reactive·stencil·시간 재구성
아트·게임 효과	custom depth/stencil outline·highlight·x-ray mask, toon·posterize·halftone·pixelation·dither, monochrome/sepia, night/thermal vision, glitch·scanline/CRT·VHS, letterbox·fade·damage mask	effect ID·order·camera/volume blend·UI 적용 범위
유틸리티·접근성	color-vision 보정 profile, flash/blur/grain 강도 제어, output-safe overlay, selection/picking debug, depth/velocity/CoC/exposure 가시화	사용자 설정 우선순위·게임 HUD·editor overlay 분리
확장	자체 PostProcess Material, typed pass plugin, scene color/depth/normal/velocity/custom stencil 입력, 사용자 parameter·curve	graph 자원 선언·명시 실행 위치·금지 read/write 검증

AO·GI·반사·volumetric·caustics의 원신호 생성은 Renderer의 별도 pass와 scene 표현을 요구한다. 이 표는 그 생성기와 resolve·합성을 함께 계획에 연결한다. 단순 full-screen 필터만 구현하고 ray tracing·조명 자체를 지원한다고 표기하지 않는다. film simulation·VHS 같은 아트 효과는 표준 preset과 parameterized custom material로 제공하며 모든 preset을 독립 shader system으로 중복 구현하지 않는다.

26.2 PostProcess Volume·레이어·커스텀 확장

PostProcessVolume은 bounds·unbound·priority·blend radius·weight·parameter override mask를 가진다. global default → local volume → camera → sequence shot → explicit gameplay layer → accessibility/output 정책의 합성 우선순위를 정의한다. 값별 linear/log/angle blend와 discrete 선택을 구분한다. exposure/pre-exposure와 adaptation history는 View/Camera별 상태다. HDR display metadata·paper white·swapchain color space는 OutputSurface별 상태로 분리하고, 여러 view를 합성할 때 공통 display-linear domain으로 변환한다.

```
struct PostEffectDesc {
    EffectId id;
    PostStage stage;
    ResolutionDomain resolution;
    ColorDomain color;
    RequiredInputs inputs;
    bool temporal;
};

struct PostProcessProfile {
    Span<const EffectSetting> effects;
    QualityTier tier;
    OutputProfile output;
};
```

custom 후처리 material은 BeforeLightingComposite, BeforeTemporalUpscale, AfterUpscaleHDR/BeforeToneMap, AfterToneMap, BeforeUI 등 명시 slot을 선택한다. AfterUpscaleHDR와 BeforeToneMap은 같은 HDR 단계의 alias이며 별도 실행 순서를 의미하지 않는다. 어떤 slot이 어떤 scene buffer를 읽을 수 있는지 type/capability 검사로 제한한다. 이전 scene color를 입력으로 받고 새 output을 생성하여 같은 texture의 위험한 read/write를 방지한다. full-screen/compute pass 모두 RenderGraph lifetime·barrier 계약을 따른다.

효과별 parameter는 reflected schema와 자체 언어 binding을 제공하고 컷씬 keyframe으로 조절할 수 있어야 한다. history는 effect instance+ViewId+HistoryCompatibilityKey로 식별한다. asset/profile revision은 진단·reload 추적 값이며 모든 연속 값 변경의 reset key로 사용하지 않는다. volume 변경은 연속 값 blend를 사용하고 history 의미가 바뀌는 discrete 변경은 reset한다. debug UI는 최종 값, 값의 출처, 지원 상태, GPU ms, history memory를 표시한다.

26.3 두 렌더 경로와 업스케일 조합

Deferred와 Forward에서 같은 camera/profile을 사용할 수 있어야 한다. Forward에 full GBuffer가 없을 때 depth·normal·roughness·material mask·velocity를 필요한 기능만 auxiliary pass로 공급한다. custom depth/stencil은 메인 depth와 별도 수명을 갖는다. Deferred-only data를 요구하는 effect는 Forward adapter를 제공한 뒤 정식 지원으로 등록한다.

MSAA resolve에서 color·depth·normal·motion의 edge 의미를 보존한다. 반투명 DOF·motion blur·refraction은 별도 depth/velocity 또는 명시 fallback을 사용한다. HDR 값에 tone mapping이 두 번 적용되거나, UI까지 bloom·motion blur가 적용되는 오류를 조합 테스트로 검출한다. temporal upscaler의 native jitter와 custom lens distortion·rolling shutter·blur 사이의 motion 규약을 문서화한다.

성능 profile은 Runtime60, Quality30, CinematicOffline으로 제안한다. 같은 효과군의 algorithm·resolution·sample count·ray budget·history 수를 profile별로 조절한다. 기능 구현 완료와 특정 장면에서 모든 효과를 최대값으로 동시에 실행하는 성능 목표를 별도로 계측한다. 어느 효과를 낮췄는지는 실제 resolved profile에 기록한다.

26.4 구현 단계와 합격 기준

단계	구현 범위	검증 gate
M3/M4	exposure·tone map·LUT·bloom·vignette·grain·FXAA/TAA·기본 AO/SSR, PostProcessVolume	두 경로와 컷씬 keyframe, 기준 영상·순서·색 관리
M5/M6	local exposure·cinematic DOF·object motion blur·lens flare/distortion·HDR·custom material·날씨/아트 preset	upscale·MSAA·자막/UI·카메라 cut·출력 resize matrix
M7/M8	convolution/anamorphic/halation·고급 렌즈·volumetric·water composite·고급 temporal denoiser	high-end 시퀀스·HDR reference·GPU/VRAM budget
M9	DynamicGI/RT 신호 합성·고급 reconstruction adapter·offline 영화 profile 안정화	실내외 noise/ghosting·capture 재현성·전체 catalog regression

표 37

색 chart·강한 역광·neon·가는 머리카락/철망·다층 반투명·근접 전경 DOF·빠른 skeletal motion·수중·빗방울·야간·camera cut의 기준 장면을 유지한다. motion blur는 velocity와 shutter에 대한 단위 테스트를 수행하고 DOF는 known depth의 CoC와 focus plane을 비교한다. HDR은 PQ/scRGB conversion 수학 테스트와 실제 HDR display의 시각 검토를 각각 수행한다.

runtime과 offline capture에서 NaN/Inf·잘린 highlight·밴딩·temporal trail·UI 오염·색 공간 불일치·구성되지 않은 pass를 검사한다. 성능은 effect별 p50/p95/p99 GPU 시간·메모리를 기록하고 quality preset 변경마다 image reference를 검토한다. catalog 각 행의 필수 세부 기능이 구현·지원 조합·회귀 영상에 연결된 상태를 전체 지원의 완료 기준으로 사용한다.

26.5 결정적인 합성과 지원 manifest

동일 stage의 효과는 dependency DAG → stage → priority → stable EffectId 순서로 결정하며 같은 priority의 순서를 플랫폼에 맡기지 않는다. temporal 이력은 매 frame 바뀌는 모든 값의 revision이 아니라 history 의미·buffer layout을 바꾸는 compatibility key로 무효화한다. 연속 exposure·volume weight 변경으로 history를 매번 초기화하지 않는다.

EffectSupport는 RenderPath·MSAA·upscaler·output/HDR·capture/AOV·required source buffer를 표현한다. 실행 시 선택된 정확한 algorithm과 fallback을 report한다. DynamicGI·water·cloud 등 원신호가 미구현인 조합은 전체 효과 지원 완료로 집계하지 않는다.

27 자체 빌드 도구·분산 빌드·반복 개발 최적화

27.1 목표와 단일 빌드 그래프

긴 compile/link와 반복 작업 비용을 제품의 핵심 성능 과제로 관리한다. 자체 BuildDriver(Rocks Build)은 UnrealBuildTool 계열의 module/target/configuration 관리, action DAG, 증분 판정, executor 추상화를 제공한다. UnrealBuildTool을 자체 엔진의 실행 의존성으로 포함하지 않는다. 참고 범위는 공개된 빌드 관리·Unity/PCH·executor 설정 개념이다. S40, S41

첫 Core 작업은 고정 toolchain과 작은 CMake/Ninja bootstrap으로 빌드한다. Core/Object Foundation Gate 통과 후 Rocks Build의 정식 구현을 시작한다. 그 이후 Rocks Build가 최종 action graph의 단일 소유자가 되고 CMake는 bootstrap·ThirdParty, Ninja는 생성된 local plan의 실행기로 사용한다. Rocks Build와 별도 CMake graph가 같은 output을 각각 생성하지 않는다. Ninja의 dependency·pool·dynamic dependency 모델을 local exporter에 활용한다. S49

ModuleManifest는 public/private dependency·include·define·PCH·link library·generated source·platform feature·plugin을, TargetManifest는 Editor/Client/Server/Tool/Test·configuration·architecture·link profile을 정의한다. 규칙 입력은 검증 가능한 선언형 데이터로 시작한다. 게임플레이 언어를 빌드 도구 bootstrap의 필수 의존성으로 만들지 않는다.

```
Bootstrap HostTools → Target/Module Plan → Native Binding Schema
 → C++/Script/Shader/Asset Actions → Module Link → Package/Validate
 → Local/Remote Execution → Artifact Manifest → Run/Attach
```

generated file은 먼저 생성 action과 출력 경로를 등록하고 compiler depfile/scan-deps가 실제 include closure를 보고한다. 소스 추가·삭제·rename·define·SDK 변경은 graph cache를 무효화한다. 출력 내용이 같으면 파일을 rewrite하지 않고 restat을 적용한다. 전역 generated header 하나의 변경으로 엔진 전체가 재컴파일되는 구조를 금지한다.

27.2 Action·캐시·증분의 정확성

generated C++에 formatter를 적용하는 action은 formatter executable/version digest, .clang-format과 실제 읽은 style 설정의 digest를 명시 입력으로 갖는다. 생성→format을 하나의 action 내부 단계로 실행하거나 raw와 formatted 산출물을 구분한 두 action으로 실행하며, 최종 경로의 producer는 하나다. formatter 변경 시 그 action과 후속 compile key가 무효화된다. 최종 바이트가 동일하면 출력 timestamp를 보존한다. CT12는 config만 변경·formatter binary만 변경·동일 출력의 세 분기와 원격 stale artifact 거부를 검증한다.

```
struct BuildAction {
    ActionId id;
    ToolchainDigest toolchain;
    ActionKind kind;
    InputTreeDigest inputs;
    OwnedArgList arguments;
    EnvDigest environment;
    Span<const ActionId> dependencies;
    ResourceRequirements resources;
    CachePolicy cache;
    ExecutionPolicy execution;
};

class BuildExecutor {
public:
    virtual Result<ActionTicket> Submit(const BuildAction&) = 0;
    virtual void Cancel(BuildSessionId) = 0;
};
```

Submit은 입력·인자를 소유 저장소로 snapshot한다. cache key는 compiler/linker/tool binary·plugin/SDK/sysroot·flag·define·환경 allowlist·include closure·생성 schema·source 내용·target ABI를 포함한다. timestamp만으로 hit를 판정하지 않는다. 경로 정규화·debug prefix mapping은 실제 compiler 출력과 일치하도록 구성하며 경로를 key에서 임의 삭제하지 않는다. 시간/난수/네트워크 등 비고정 입력을 사용하는 action은 명시하지 않으면 cache/remote 대상에서 제외한다.

action cache는 이전 depfile에 포함된 파일만 재해시하여 완전성을 가정하지 않는다. include 탐색 순서와 directory membership/negative lookup을 hermetic input tree에서 검증한다. 이를 추적하지 못한 adapter는 영향 변경 시 preprocessor·dependency 재스캔으로 전환한다. 새 shadow header·false→true의 __has_include·생성 header·SDK/sysroot 파일 존재 변화가 key 판정에 포함한다. graph cache 무효화와 action cache 재검증은 별개다. 이 반례는 ccache 공식 direct-mode 설명에도 제시된다. S54

새 우선순위 header·optional include 생성/삭제·동일 timestamp 변경·branch switch·generated producer 지연·다른 checkout의 shared hit를 clean build와 대조한다.

local content-addressed store → 팀 shared cache → 필요 시 승인된 remote executor 순으로 조회·실행한다. cache miss 이유를 file/flag/toolchain 단위로 보고한다. output·depfile·diagnostic·debug file을 완전한 artifact set으로 atomic publish한다. 실패·취소·부분 출력은 hit로 등록하지 않는다. 한 output의 중복 producer는 plan 오류다.

DDC는 asset/shader 파생 데이터, BuildArtifactCache는 object/PCH/generated/link 산출물을 관리한다. 공통 저장 엔진을 사용할 수 있으나 namespace·key·retention을 구분한다. original bulk store는 어느 캐시에도 대체하지 않는다. 동일 입력 full rebuild와 incremental rebuild의 의미적 결과를 비교하고 stale hit 0건을 필수 기준으로 한다.

27.3 Compile time 최적화

수단	적용 정책	정확성·성능 확인
Include 경계·IWYU	public header 최소화, forward declaration·Private 분리, 대형 implementation은 비공개 이동	include fan-out·재빌드 TU 수·비 Unity header 독립 compile
Template/생성 코드 축소	explicit instantiation·작은 binding stub·type별 generated unit, 과도한 inline 억제	compile trace·symbol/object 크기·schema 변경 영향
Shared/Explicit PCH	compiler·flag·macro 집합이 같은 군에서만 공유	PCH 생성/재사용·변경 비용·cache portability
Adaptive Unity	안정 파일 batching, 자주 편집하는 파일은 분리	파일 하나 수정 시 batch 범위, ODR·macro 오염·메모리
증분 메타·언어 compiler	include/type dependency별 재생성, body-only script는 해당 module IR만 갱신	무변경 output rewrite 0건·의존 누락 0건
C++ module/header unit	생성 헤더·macro·toolchain·cache 지원을 검증한 선택 실험	전면 필수화하지 않고 PCH 기준과 비교
Worker/agent 수 조정	CPU뿐 아니라 TU peak RAM·IO·PCH 비용·사용자 작업 여유 반영	swap/메모리 초과·editor 응답·critical path

표 38

Unity Build가 include 누락을 감추지 않도록 CI에 non-unity와 PCH 비활성 표본을 유지한다. adaptive 판단은 source-control writable 상태만이 아니라 최근 편집·TU 비용을 함께 사용한다. game module의 가벼운 변경이 Core public API·editor 전체를 재빌드하지 않게 module boundary를 지속 측정한다.

27.4 Incredibuild·FASTBuild·캐시 adapter

Executor/도구	위치와 도입 범위	제약·실패 시 동작
Local executor	항상 제공, CPU/RAM pool·중단·재개·trace	해당 host가 native toolchain을 지원하는 target은 외부 agent 없이 빌드 가능

Executor/도구	위치와 도입 범위	제약·실패 시 동작
Incredibuild adapter	Rocks Build action DAG를 검증된 XML/지원 interface로 변환, code·custom shader/cook worker	설치된 제품의 interface·toolchain 지원을 검사하고 unavailable이면 local 전환
FASTBuild adapter	선택 BFF/plan exporter, compile distribution·cache	Rocks Build와 output producer를 중복 소유하지 않음
sccache adapter	지원 compiler의 local/shared compile cache	distributed support와 단순 Windows cache 지원을 구분
자체 BuildAgent	content-addressed input·isolated workdir·toolchain staging·heartbeat	기존 executor로 요구를 충족한 뒤 도입 여부 판단

표 39

Incredibuild XML Interface는 command·input/output·dependency를 표현할 수 있으므로 custom engine adapter의 근거로 사용한다. Unreal 전용 통합만 설치하면 자체 엔진의 모든 작업이 자동 분산된다고 가정하지 않는다. S42

FASTBuild의 cache·distribution과 sccache의 compiler cache를 선택 지원한다. MSVC debug format·PCH·C++ module의 실제 지원은 선택 버전으로 검증한다. 특히 sccache의 분산 실행 문서는 Linux 중심이며 Windows에서 같은 구성이 검증되었다고 가정하지 않는다. S43, S44, S45

한 action은 하나의 executor와 한 cache owner를 갖는다. Incredibuild 안에 FASTBuild distribution과 sccache remote scheduler를 중첩하지 않는다. 조합이 필요하면 cache-only/execute-only 책임을 설정으로 분리하고 계측한다. agent 분실·timeout은 격리된 temp output에서 재시도하며 이전 session의 늦은 응답이 최신 build를 덮어쓰지 못하게 한다. signing·외부 publish 같은 side effect action은 임의 재시도·중복 실행하지 않는다.

27.5 Link time의 독립 최적화

컴파일의 원격 병렬화만으로 직렬 최종 link 시간을 해결할 수 없다. link·PDB·LTO·package를 따로 계측하고 module link 크기·public symbol·debug 정보·template 중복을 줄인다. DevModular는 작은 DLL/라이브러리 경계, Shipping은 profile별 monolithic/modular와 최적화를 선택한다. DLL 수를 과도하게 증가시켜 startup·ABI·배포 비용을 확대하지 않는다.

Windows는 clang-cl+lld-link의 fresh link와 MSVC LINK의 incremental link를 각각 benchmark한다. lld-link가 MSVC의 .ilk 기반 incremental 기능을 동일하게 제공한다고 가정하지 않는다. MSVC incremental이 비호환 flag나 큰 변경 때문에 full link로 전환되는 이유도 기록한다. S46, S48

Developer 기본 profile에는 full-program LTO를 적용하지 않는다. Shipping/성능 검증에는 ThinLTO 또는 선택 LTO·PGO를 사용하되 incremental cache·backend 병렬 수·RAM을 제한한다. ThinLTO 지원은 target/object format/toolchain 조합별로 검사한다. S47

debug 정보는 action cache/remote 환경에 맞는 format을 선택한다. /Z7·/Zi·PDB 생성은 trade-off가 있으므로 /Z7을 항상 빠른 설정으로 간주하지 않는다. PCH·compiler PDB 공유·link PDB 기록을 분리하고 경합을 계측한다. 변경 입력이 없으면 link 자체를 생략한다. link cache는 완전한 입력 key·debug artifact set을 검증한 경우만 사용하고, .ilk 같은 session 상태를 portable artifact로 무조건 공유하지 않는다.

macOS도 native linker·symbol/dSYM 생성·app bundle·서명·필요 notarization·프로세스 재시작/상태 복구 시간을 같은 end-to-end 지표로 측정한다. 개발 중 반복 경로와 배포 서명 경로를 분리하되 실제 완료 시간을 누락하지 않는다.

27.6 반복 개발 루프와 재시작 비용

변경 유형	개발 루프	기본 완료 경계
Script 함수 본문	dependency compile → verify → compatible code revision 교체	editor/PIE 유지, 기존 frame lease 보존
Tag/DataAsset/UI layout·prefab 값	validate → delta cook/compile → transaction 적용	해당 instance/view 갱신
Shader/material	영향 permutation만 worker compile → PSO 준비 → atomic 교체	compile 실패 시 이전 버전 유지
Texture/mesh/audio	의존 산출물만 recook → async lease 교체	dependent preview·resident 자원 갱신
Private C++ body	최소 TU compile → 해당 module link → 필요 프로세스 재시작/상태 재개	native patch가 검증되지 않으면 재시작
Public header/schema/layout	영향 graph 재계산·migration → compile/link → 안전한 재시작	old object·VM frame 무리한 재사용 금지

표 40

persistent build daemon은 graph·file hash·compile database·source map을 cache하고 file watcher는 overflow/branch 전환 시 재검증한다. 한 번의 Build/Run 요청이 import·language·shader·native·cook·launch를 조합하되 변경 없는 단계를 수행하지 않는다. UI에서 Rebuild All을 반복하도록 요구하지 않는다.

재시작은 editor session state·camera·selection·open asset·test map을 저장하고 backend 재기동 후 복원한다. 게임 런타임 snapshot은 검증된 schema에서만 선택 복구한다. editor frontend를 유지할 수 있어도 native engine state가 그대로 유지된다고 표시하지 않는다. full clean은 graph/toolchain 손상 또는 명시 요청에만 사용하고 일상적인 오류 복구 수단으로 삼지 않는다.

27.7 셰이더·쿠킹·테스트의 분산

ShaderWorker와 Cooker는 순수 action input manifest를 사용한다. shader include closure·DXC/options·RenderPath·MSAA·feature permutation을 key로 하며 stale result를 표시하지 않는다. asset cook은 dependency DAG의 독립 노드만 분산하고 package assembly·manifest publication을 직렬 commit한다. 같은 source file을 여러 worker가 직접 수정하지 않는다.

IO·memory·GPU 요구를 resource class로 선언한다. GPU capture와 CPU shader compile은 서로 다른 queue를 사용하고, editor interactive compile은 대형 background cook보다 우선한다. CPU compiler/cooker pool과 Runtime Task Graph는 OS 자원을 조율하지만 엔진의 frame worker queue에서 장시간 외부 compiler process를 동기 대기하지 않는다.

test sharding은 fixture dependency·World namespace·temp output을 격리한다. flaky test를 재실행하여 성공으로 숨기지 않고 첫 실패와 재현 정보를 보존한다. remote input에는 필요한 source·toolchain만 포함하고 빌드 machine 전체 환경을 자동 복제하지 않는다.

27.8 빌드 성능 계측과 합격 기준

time-to-first-correct-frame은 편집 저장 → graph 판정 → codegen → cache/compile → link/PDB → reload/restart → 첫 정상 frame까지다. compile 시간만 표시하지 않는다. Rocks Build는 build session·action critical path·queue wait·input upload·cache lookup·compile·link·artifact download·launch를 trace한다.

아래 수치는 **고정한 실제 중형 개발 fixture의 미측정 목표**다. M2 이후 engine commit·TU 수·preprocessed bytes·template 부하·module 크기·장치·local/remote agent·cache 상태를 고정한다. 현재 존재하지 않는 전체 엔진의 clean build 시간을 측정한 것으로 해석하지 않는다.

Scenario	초기 p95 목표	반드시 기록할 항목
I0 무변경 build	≤2 s, compile/link/codegen action 0개	graph scan·hash·daemon cold/warm
I1 script body 수정	compile+verify+실행 반영 ≤2 s	native TU/link 0개, 실행 오류·old frame 정리
I2 단일 UI/data/prefab 값	정상 preview 반영 ≤1 s	native compile 0개·undo·watcher·frame
I3 단일 private C++ body	첫 정상 frame까지 ≤30 s, module link p95 ≤10 s 목표	TU·link/PDB·restart를 분리, native patch 미가정
I4 대표 public header/schema	영향 graph 전체 정상 실행 ≤120 s	재빌드 TU fan-out·migration·최대 memory
I5 shader/material 변경	표준 preview fixture 정상 교체 ≤10 s	permutation 수·compile·PSO·GPU 준비
I6 clean/cache-restore 대형 build	M3 실측 baseline 뒤 목표 확정	local/분산/cold/warm, cache miss·agent·전송·link 별도

표 41

I6는 agent 수만으로 고정 배속을 약속하지 않는다. 원격 실행이 network/PCH/input 비용 때문에 느린 action은 local로 배치한다. 반복 fixture 20회 이상에서 p50/p95/max와 실패율을 기록하고, cache correctness와 non-unity 결과가 먼저 통과해야 속도 개선으로 인정한다. new workspace·branch switch·toolchain upgrade도 별도 workload로 유지한다.

27.9 도입 순서와 운영

Core 첫 단계에는 bootstrap·unit/stress runner·compiler timing만 준비한다. Foundation Gate 이후 M2 말/M3에 Rocks Build graph·local cache·include fan-out·PCH/Unity benchmark를 완료한다. M4에는 Incredibuild/선택 executor와 shader/cook 분산 adapter, M5에는 agent 장애·부분 cache·여러 개발자 동시 작업·반복 루프 SLA를 검증한다. M6 이후 link/LTO/PGO·fleet 자동 배치를 실측에 따라 확장한다.

CI는 no-op·부분 수정·header 변경·full clean·cache restore·cache poison fixture를 실행한다. 선택 executor와 local 기준 결과를 비교하며 속도 회귀 10% 이상 또는 재빌드 TU 수 증가를 추적한다. cache/network가 없어도 해당 host에서 지원되는 native toolchain target의 정확한 local build가 가능해야 한다. macOS→DXBC처럼 외부 compiler host가 필요한 target은 해당 worker 부재 시 CapabilityUnavailable로 실패한다. 잘못된 다른 compiler로 자동 대체하거나 cross-target 성공으로 보고하지 않는다.

27.10 패키징·플러그인·출시

plugin은 compatible compiler/CRT로 시작하고 binary 경계는 versioned C function table·opaque handle·명시 allocator/destroy를 사용한다. task·callback·script/native metadata lease 종료 전 unload하지 않는다. Target별 shipping/editor-only dependency closure를 검사하여 Server/Cooker/Client에 불필요한 renderer·WebView·LLVM이 링크되지 않게 한다.

PR은 Core/Object·메타·언어·헤드리스 sample·schema·증분 cook·금지 의존성 검사다. nightly는 sanitizer·GC/task/loader stress·network·UI·render/capture 회귀와 build scenario를 수행한다. Windows ASan과 지원 Linux ASan/UBSan/TSan을 구분한다. S18

배포물은 executable·package·schema/script/shader manifest·symbols·test/trace report·ThirdParty 고지를 포함한다. 새 환경에서 shared cache·source asset·editor 없이 실행을 검증한다. crash reporter는 build ID·module/symbol·script stack·최근 trace를 연결한다. commandlet은 validate/import/cook/package/test/capture/localize를 제공한다.

코드·에셋 branch 통합의 검증된 후보·충돌·게시 정책은 AAA07(33.6절)과 연결한다. 자동 merge 서비스 전체 구축은 후속 선택이며 격리 검증 계약부터 적용한다.

28 정식 헤드리스 실행·서버·자동화

28.1 실행 모드와 의존성

헤드리스는 창을 숨기는 임시 옵션이 아니라 정식 target/profile이다. display/window, GPU device, audio device, editor UI, network service를 독립 capability로 지정한다. CPU-only와 GPU offscreen을 구분하며 renderer를 제거한 NullRHI에서 영상 검증이 성공하였다고 보고하지 않는다.

Profile	필요한 기능	포함하지 않는 기본 의존성
CoreTest	Core·Task·Object·최소 schema/GC harness	World·GPU·window·WebView·audio device
DedicatedServer	World·script·physics/nav·network·game data	Renderer/RHI backend·window·게임 UI·audio device
Commandlet	작업별 asset/schema/cook/build·선택 World	기본 window·WebView·GPU·audio. 필요 capability는 명시 요청
Simulation/Replay	fixed clock·gameplay/AI/physics·state snapshot	presentation device, 선택 network
EditorBackend	asset·transaction·prefab/script schema·자동화 API	WebView·native window. renderer는 별도 선택
OffscreenRender	Renderer·GPU·texture target·readback·offline audio	visible window·swapchain present·physical audio device

표 42

DedicatedServer는 PlatformBase와 필요한 library만 링크한다. PlatformWindow·Renderer·WebView를 한 공통 DLL의 강제 import로 포함하지 않는다. NullRHI는 resource/state validation 또는 renderer smoke test를 위한 선택 backend이며 CPU-only target의 필수 dependency가 아니다. Asset 요청·World activation mask도 해당 profile에 맞게 검사한다.

28.2 공통 BootServices와 CLI

headless와 interactive는 같은 Object/World/Asset/Script 서비스를 조합한다. 모드마다 별도의 유사 엔진을 구현하지 않는다. 서비스 registry는 Required/Optional/Forbidden capability를 검증하고 불필요한 OS window·device·로그인 dialog·file picker를 생성하지 않는다. 모든 경로·설정은 CLI·manifest·환경 allowlist로 지정한다.

다음 명령은 **구현 예정 CLI 계약 예시**다.

```
rocks core-test --suite Core,Object --report result.json
rocks server --project Game --map TestWorld --fixed-hz 60 --no-audio-device
rocks cook --target Server --asset-manifest Assets.json --headless
rocks validate --project Game --prefabs --scripts --ui --sequences
rocks simulate --map AIStress --ticks 36000 --seed 42 --snapshot out.bin
rocks capture --sequence Intro --render offscreen-d3d12 --frames 0:240
rocks ui-test --widget MainMenu --layout-only --viewport 1920x1080
```

Rocks Build가 필요한 target/build를 준비한 뒤 위 실행 도구를 호출한다. engine run process가 자기 바이너리를 다시 link하여 덮어쓰지 않는다. project path·working directory와 독립된 asset mount를 사용하며 commandlet output은 작업별 디렉터리와 build/job ID로 격리한다.

28.3 Clock·수명·종료

headless simulation은 FixedTick clock, timeout은 real monotonic clock을 사용한다. 실행 속도가 실시간보다 빠르거나 느려도 tick 수와 seed를 일관되게 유지한다. cross-platform physics의 bitwise 결정성은 별도 보장이며 같은 상태에 대한 허용 오차·event 결과로 검증한다.

AudioNull은 command·수명·event만 처리하고 sample mix 정확성 검증을 대체하지 않는다. OfflineMixer는 실제 DSP를 고정 sample 수로 실행하여 PCM을 생성한다. GPU capture는 offscreen texture·fence/readback을 완료한 후에만 frame을 성공 처리한다. display 없는 GPU/driver 환경의 capability를 별도 probe하고 지원 실패를 보고한다.

종료는 stop admission → cancel → World deactivate → teardown command drain → physics/audio/GPU 완료 → root 반환·최종 GC → 추가 cleanup/backend drain·owner closure → output atomic commit → scheduler/type/writer 종료 → process exit 순서다. 3.2절의 GC/drain 반복과 수명 종료 규약을 같은 기준으로 적용한다. Ctrl-C/서비스 stop·부모 process 종료·deadline을 지원한다. graceful timeout 뒤 watchdog이 crash/stack/trace와 미완료 action을 보존하고 강제 종료한다. 취소 중 output을 정상 완료 artifact로 등록하지 않는다.

28.4 기계 판독 결과와 배치 운영

stdout은 선택 JSONL/progress protocol, stderr는 사람이 읽는 진단으로 구분한다. job 결과에는 status, exit reason, build/schema/asset revision, profile, seed, tick/frame/sample range, counters, artifact hash를 포함한다. success·failure·cancelled·unsupported·timeout·crash를 구분하고 console 출력 문자열로만 성공을 판정하지 않는다.

exit code 계약은 0 성공, 2 입력/설정, 3 asset/schema, 4 실행/검증 실패, 5 필수 capability 미지원, 6 timeout, 130 사용자 중단을 기본안으로 한다. OS crash code는 별도로 보존한다. optional GPU test의 skip은 계획에 선택으로 등록된 경우만 허용하며 필수 영상 job의 no-GPU는 실패다.

서버 service mode는 health/readiness·graceful shutdown·memory/tick/network counters·재시작 정책을 제공한다. 동시에 여러 World/job을 실행할 때 port·save path·AssetLease·temp output이 충돌하지 않아야 한다. source-control credential이나 대화형 계정 로그인에 의존하지 않는 사전 구성된 작업 profile을 사용한다.

28.5 완전한 헤드리스 검증

검증 묶음	필수 기준
CPU dependency	GPU/WebView/audio device 없이 boot. Linux display 환경 변수·window server 없이 실행. GUI device 초기화 0건
Foundation	Core/Object 정지형 GC·수명·동시성 stress를 CPU-only에서 재현하고 정확히 종료
Cook/Validate	metadata·script·prefab·UI·sequence·package 검사와 invalid fixture 오류 코드 검증
Server/Simulation	16 client·60Hz·travel/reconnect·fixed tick·shutdown·soak, old World root 정리
GPU offscreen	Deferred/Forward·업스케일·post·UI·컷씬의 실제 영상/trace 생성. interactive와 같은 profile 비교

검증 묶음	필수 기준
Fault/Cancel	IO hang·script loop·missing asset·device loss·cancel/timeout 중 partial output·메모리·프로세스 잔존 검사

표 43

UI layout·tag/prefab·script conformance는 CPU-only fixture로 실행하고, font shaping은 고정 font asset과 renderer 독립 text backend를 사용한다. 픽셀·HDR·실제 음향 장치·DPI/focus 검증은 해당 device를 사용하는 별도 job으로 유지한다. 모든 검증을 NullRHI 한 종류로 대체하지 않는다.

CoreTest는 첫 작업부터, M3에는 asset Commandlet·CPU boot, M4에는 단독 fixed-tick World/physics/AI simulation과 server service boot/shutdown을 검증한다. 16-client replication·travel/reconnect·soak는 선행 기능이 완성되는 M5 gate다. offscreen render/UI/capture는 M4–M6의 기능별 suite로 운영한다. 각 suite의 단계·필수 사례를 manifest에 열거하여 M4 boot 성공을 M5 multiplayer 완료로 보고하지 않는다. 클라우드 GPU·Linux Vulkan·컨테이너 provider는 지원 target·driver·image를 고정한 후 확장한다.

29 통합 검증·프로파일링·성능 예산

29.1 계측 체계

Trace는 CPU scope·Task dependency·GC phase·allocation tag·IO request·asset state·GPU pass·network packet·audio block·script function·sequence playhead를 동일 session ID로 기록한다. CPU와 GPU clock은 calibration pair로 연결하고 오디오 sample time·server tick은 명시 변환 테이블을 유지한다. 서로 다른 clock을 같은 숫자로 해석하지 않는다.

ring buffer는 bounded이며 event loss count를 기록한다. profiling 자체의 비용·buffer 용량을 표시하고 shipping은 최소 crash breadcrumb 수준으로 구성한다. profiler UI는 critical path, worker idle/starvation, GC barrier 대기, residency, object root chain, DDC miss reason, shader permutation 수, post-effect resolved value를 조회한다.

29.2 재현 가능한 시험 장면

모든 수치는 **미측정 초기 목표**다. M0에서 실제 CPU·GPU·RAM·SSD·OS·driver·power profile을 고정하고 scene manifest·camera path·seed·build hash를 저장한다. 기준 장치는 12–16 physical core급 CPU, RAM 32 GB, NVMe SSD, D3D12 12 GB VRAM급 GPU를 가정한다. 최소·권장·고급 profile은 이후 실측으로 확정한다.

시험 ID	고정 workload	필수 측정과 초기 합격 기준
B0 · Core/Task	1,000만 container operation, 1,000만 task·취소·의존성 변형	누락/중복 실행·data race·leak 0건, worker별 공정성·overhead 기록
B1 · Object/GC	100만 객체·400만 edge·1천 roots, 초당 1만 create/destroy·30분 mutation	reachable 오수거 0건, mark slice p99 ≤0.5 ms·총 safepoint p99 ≤1 ms 목표
B2 · 기본 3D	10만 render instance·2,000만 source triangle, 100 light·8 shadow caster	Deferred/Forward 각각 native 1080p 기준 GPU p95 ≤14 ms·p99 ≤16.67 ms 목표
B3 · Game	100 animated character·1,000 active rigid body·200 AI·128 voice	CPU critical path p95 ≤12 ms, server 60Hz, audio underrun 0건 목표
B4 · Streaming	4×4 km tile world, 30분 이동·반복 teleport·cold/warm cache	RAM/VRAM 상한 유지, active cell 준비 오류 0건, game IO 동기 wait 0건

시험 ID	고정 workload	필수 측정과 초기 합격 기준
B5 · 제작	태그·component·nested prefab·script 편집, 1,000회 PIE	종료 후 resource baseline 복귀, undo/save/reload 결과 정합
B6 · Network	16 client, RTT 0–200 ms·loss 최대 5%·jitter/reorder	ID alias·권한 위반·세션 간 stale 적용 0건, tick/bandwidth/correction 기록
B7 · Cinematic/Post	25절 데모, 26절 효과 scene, 24/30/60 fps·4K capture	A/V ±20 ms 목표, skip 복구 오류 0건, frame 누락·NaN 0건
B8 · Upscale	1080p/1440p/4K output, 고정 품질·동적 render scale	양 경로의 지원 matrix 통과, cut 잔상 0 frame, quality·latency·VRAM 기록
B9 · High-end	dense static city·실내외 GI·night emissive·streaming	VG crack/누락 0건, GI artifact threshold·카탈로그 기능별 image 승인
B10 · Build/Iteration	I0–I6·branch/toolchain/cache/executor 변형	stale hit·잘못된 증분 결과 0건, end-to-end latency·link 분리 측정
B11 · Headless	28절 profile·cancel/device/missing fixture	capability·exit/report 정합·GPU 영상 job의 허위 성공 0건
B12 · UMG	Designer/game/CPU-layout/GPU-pixel 동일 WidgetAsset	binding·IME/focus·DPI/HDR·cook·수명과 21.5절 budget

표 44

B2/B3의 CPU·GPU 시간은 overlap되는 각각의 critical path이며 합산값이 frame time은 아니다. B2의 2,000만 source triangle은 모든 삼각형의 동시 raster 비용을 뜻하지 않는다. visible triangle·shaded pixel·overdraw·material complexity·output resolution을 함께 기록한다. B1의 root snapshot·final remark·sweep cutoff도 시간 측정에서 제외하지 않는다. 대규모 root·거대 참조 배열·긴 VM frame은 별도 B1 확장 fixture에서 최대 정지와 backlog를 측정하며 기본 1천-root 목표를 그대로 일반화하지 않는다.

Runtime60의 GPU 14 ms p95 예산 예시는 geometry/depth 3.0, shadow 2.0, lighting/GI 3.0, transparency/VFX/fog 1.5, upscale 1.5, output post/UI 1.5, 여유 1.5 ms다. 합계 14 ms이며 효과 조합에 따라 재배분한다. 초기 전체 VRAM 목표는 12 GB 장치에서 10 GB 이하, RAM steady-state 24 GB 이하로 제안하며 OS·driver reserve를 실측한다. history·upload·transient peak와 double-buffered streaming도 포함한다.

Quality30과 CinematicOffline은 별도 target이다. Runtime60 달성을 위해 지원 catalog의 기능을 삭제하지 않고 sample·quality·resolution·update rate를 profile로 조절한다. 성능 회귀는 동일 scene·동일 profile에서 p95가 5% 이상 악화되면 원인을 검토한다. 중간값만 보고 hitch·최대값을 숨기지 않는다.

29.3 정확성·장애 주입

병렬 구현은 직렬 oracle, GC는 full collector, skinning은 CPU reference, culling은 conservative CPU set, shader는 기준 image와 비교한다. 픽셀 비교는 비결정적인 GPU 차이를 고려한 tolerance와 temporal sequence 평가를 사용한다. render validation layer 결과, crash·hang·leak, data race·OOB는 성능과 무관하게 실패다.

IO 지연·부분 read·DDC 손상·shader compile 실패·GPU memory budget 초과·asset 순환·오디오 장치 제거·network loss·script 무한 loop·save 도중 종료를 주입한다. 각 장애는 terminal error·fallback·cleanup·재시도 한도가 있는지 검증한다. bug 재현 asset와 최소 script·trace·seed를 regression fixture로 보존한다.

29.4 필수 로깅·프로파일러·상태 진단

로깅·프로파일러·시스템 상태 조회 도구는 정식 필수 기능이다. Rocks Lens는 라이브 세션과 저장된 capture의 원인을 분석하는 도구이며, 단순 FPS 표시나 binary trace 생성만으로 완료하지 않는다. R0/R1의 console/report부터 R2의 오프라인 viewer, M3/M4의 editor 통합·GPU timeline으로 단계화한다. 개발 중 원인을 관찰할 수 없는 기능은 해당 milestone을 완료한 것으로 집계하지 않는다.

계층	필수 정보·조작	초기 제공 시점
구조화 로그	level/category·UTC+monotonic time·thread/session·task/asset/entity correlation, 필터·검색·파일 rotation·stack link	R0/C06 console/file, M3 viewer 통합
CPU·Task	scope·frame p50/p95/p99·hitch·worker별 실행/대기·dependency/critical path·lock wait	R0 최소 scope, R2 timeline/Task 연결
메모리·Object·GC	tag별 current/peak·allocation/free·retained·live object/root·GC phase/pause·root chain·retire backlog	R1 summary, R2 history/diff·선택 상세 allocation
자산·스트리밍·빌드	load/IO/decode/upload 상태·mip demand/residency/pool·DDC miss·shader variants·compile/link/cook 시간	M3/M4 해당 기능과 함께
GPU·렌더링	pass/queue timestamp·GPU/CPU clock 상관·resource budget·transient peak·draw/triangle/material·history	M3 초기 GPU scope, M4 queue/pass viewer
기타 시스템	network RTT/loss/queue·audio underrun/voice·physics/nav/animation/script·입력 loss·컷씬 상태	각 시스템 최초 통합 Gate
상태 조작	capture 시작/종료·채널/level 변경·bookmark·현재값 snapshot·diff·export·watchdog dump	R2 CPU headless, M3 editor 연결

표 45

로그는 category별 rate limit·중복 집계와 compile/runtime level을 제공한다. error/fatal도 bounded storage·별도 emergency 경로를 사용하며 저장 무한 보장을 선언하지 않는다. 로그나 trace를 위해 AudioRT/allocator/GC의 임계 구간에서 file IO·heap allocation·일반 lock wait를 수행하지 않는다. 중요 사건은 사전 확보 breadcrumb에 남기고 손실 수·구간·원인을 viewer에서 표시한다. 로그와 trace의 공통 correlation ID는 유지하되 모든 문자열 로그를 고율 trace로 복제하지 않는다.

TraceCapture/TraceAnalysis/LensViewer를 분리하고 수집과 분석이 웹 UI에 의존하지 않게 한다. channel manifest·event schema/version·build ID·module symbol ID·clock quality·buffer high-water·loss 범위를 capture에 포함한다. session-owned metadata를 복사/보존하여 module unload 이후 분석할 수 있게 한다. allocation 추적은 예약 storage와 재귀 guard를 사용하고 callstack symbolization은 offline 또는 별도 worker에서 처리한다. 상세 memory channel을 중간에 시작한 경우 이전 할당은 snapshot baseline으로 구분하며 완전한 lifetime 기록으로 표시하지 않는다.

Rocks Lens는 frame 선택→CPU/Task/GC/GPU/IO 구간 상관→관련 로그/asset/object 조회→capture 비교의 흐름을 지원한다. built-in diagnostic command는 typed registry에 등록하고 read-only snapshot을 기본으로 한다. 파괴적인 runtime 변경은 개발 모드 capability를 요구하며 shipping에는 필요한 최소 상태·crash breadcrumb만 포함한다. session 연결은 기본 local이며 외부 연결은 명시 endpoint·인증·권한 설정을 따른다. 비밀번호·token·사용자 입력 전문을 기본 로그 항목으로 수집하지 않는다.

정상 종료는 producer 종료→최종 GC 상태 기록→trace drain/flush→writer join→allocator 해제다. crash/hang의 수집은 사전 확보 storage와 제한 시간의 best effort이며 truncated/incomplete 표시를 남긴다. 잠긴 heap·registry를 crash handler가 다시 순회하지 않는다. GPU timestamp disjoint·device lost·clock calibration 오차와 event loss는 추정값·미완료 span으로 표시하고 임의로 완전한 timeline을 합성하지 않는다.

Epic의 Trace 및 Memory Insights 문서는 수집/분석/viewer 분리와 allocation·tag·callstack 조사 기능의 비교 자료다. Rocks의 format·API·분석기는 독립 구현이며 UE trace 파일과의 호환을 기본 요구로 두지 않는다. S66, S67

29.5 관측 도구 수용 기준

ID	시험	완료 판단
OB01	category/level·burst·rotation·disk full·writer 지연	필터 정확성, 실행 thread 비차단, 누락/실패 표시
OB02	CPU/Task/GC/memory fixture capture→viewer	올바른 span·dependency·counter·root 원인 조회
OB03	queue 포화·metadata unload·partial file·중간 capture	UAF 0건, loss/truncated·baseline 표시, 손상 구간 이후 안전한 분석
OB04	allocator/AudioRT/GC 내부 계측·recursive log	계측 재귀·교착 0건, hot path 추가 allocation/IO 없음
OB05	shutdown·hang·crash·device loss	bounded 종료·유효 chunk 복구 또는 명시 실패, 허위 완전성 0건
OB06	native/headless/live/offline·두 build capture 비교	같은 schema 해석·symbol ID 일치·clock 불확실성 표시·overhead 보고

표 46

기본 수집·상태 report·읽을 수 있는 offline timeline은 R2 필수다. sampling·압축·대형 trace 탐색·계측 overhead의 미세 최적화만 후속 선택이다. 각 지원 target에서 계측 on/off의 CPU·RAM·IO·frame latency 차이를 측정하며 임의의 0% overhead나 무손실을 약속하지 않는다. OB01–06은 기능 구현 단계에 맞는 CPU subset부터 시작하고 GPU/실장비 부분은 별도 통과 결과를 남긴다.

29.6 안정성 우선과 누수 방지 Gate

Rocks의 우선순위는 안정성·수명 정확성, 확장성, 성능이다. 메모리 누수 0을 출시 Gate로 두고 정상 실행뿐 아니라 실패·취소·rollback·재로딩·장치 손실·종료 경로를 검사한다. 계획과 유한한 시험만으로 모든 환경에서 결함이 없음을 입증하였다고 표현하지 않는다. 발견된 누수·UAF·double free·data race를 성능 수치로 상쇄하지 않으며 원인 미해결 상태를 정상 완료로 처리하지 않는다.

모든 allocation/resource에 owner scope·allocator/해제 API·generation·취소·최종 회수 조건을 명시한다. 메모리 ledger는 native heap, managed graph, task/callback, IO/staging, GPU resource/descriptor, plugin metadata, trace buffer, built-in/persistent asset의 별도 회계를 제공한다. raw pointer는 짧은 비소유 접근으로 제한하고 ownership 이전은 실패 시 잔존 소유자까지 정의한다. nonmoving GC는 native/GPU/SDK resource 회수를 자동 보장하지 않으므로 backend lease와 RAII를 연결한다.

Core의 shutdown은 추적 가능한 엔진 소유 allocation의 잔존 0을 목표로 한다. World/PIE stop은 해당 scope owner 0과 명시한 공유 cache/builtin baseline 복귀로 검사한다. 정상 cache·pin은 owner·상한·만료/해제 조건이 있는 allowlist로 분리하며 증가하는 누수를 cache로 재분류하지 않는다. 외부 SDK/driver가 보고하는 메모리와 엔진 ledger의 관찰 범위를 구분하고 미해명 증가도 조사 대상으로 유지한다.

cancel·timeout만으로 외부 callback/GPU 완료를 가정하지 않는다. 회수 불가능한 진행 중 lease는 bounded quarantine과 retained reason으로 표시하고 신규 요청을 제한한다. 안전한 backend 종료/복구 경로로 정리하며 무기한 증가·무제한 retry를 허용하지 않는다. deadlock이나 장치 손실에서 억지로 free하여 수치를 0으로 만드는 구현은 금지한다. 종료 실패·불완전 회수는 실패 결과로 남긴다.

ID	필수 검증
NL01	모든 allocation 단계의 N-fail·부분 생성/등록 rollback: publish·leak·소유권 유실 0
NL02	root cycle·weak pin·container 재배치·GC cutoff·retire: 오수거 0, root 제거 후 회수
NL03	task/IO/compile/upload 취소·늦은 callback·module unload: terminal 1회·lease 종료 전 free 0
NL04	PIE 1,000회·asset/template reload·streaming 왕복·profile 교체: owner/baseline 복귀·증가 기울기 조사
NL05	GPU device loss·backend failure·quota 초과: bounded quarantine·재생성·실패 보고·정상 pool 오반환 0
NL06	정상 shutdown·중간 실패·강제 crash: 정상 ledger 잔존 0, crash의 복구 범위·truncated 상태 명시
NL07	지원 sanitizer·독립 allocator ledger·graph oracle·seed stress: 툴별 관찰 범위/NOT RUN 공개
NL08	최적화 on/off·1/N worker·cache cold/warm: 의미·소유권 동일, 성능 개선 전 정확성 통과

표 47

N-fail·rollback·baseline·원인 report는 Core CF02/CF07–CF12에서 시작한다. leak sanitizer 지원 플랫폼에서 상세 검사하고, 미지원 target은 자체 ledger·반복 scope 종료·heap/resource snapshot을 병행한다. 일정 압박으로 correctness gate를 생략하지 않는다. lock-free·allocator cache·parallel GC·async queue 등 최적화는 검증된 기준 경로와 비교 가능한 선택 switch를 갖는다.

29.7 모든 구현의 테스트 하네스 의무

모든 구현 변경에는 기능 계약과 대응하는 유닛·계약·회귀 하네스를 함께 제출한다. API 존재·화면이 한 번 표시된 결과·추정한 정상값으로 완료를 판정하지 않는다. 성공·실패·경계·취소·수명·thread·호환성의 관련 사례와 실제 실행 target, 미실행 사유를 작업 manifest에 연결한다. 기존 테스트 재사용은 허용하지만 해당 변경을 검출하는지 입증해야 한다.

하네스는 TestClock·DeterministicExecutor·N-fail Allocator·ScriptedIO·FakeGpuCompletion·NetworkFault·SeededGraph·GoldenArchive를 제공한다. 가짜 backend는 오류·지연·재정렬·중복 completion을 제어하기 위한 것이며 실제 장치 conformance를 대체하지 않는다. 기대값은 명세·독립 reference model·수동 검증 golden artifact에서 도출하고 구현을 그대로 복제한 테스트만으로 승인하지 않는다.

compile-time ABI/header 검사, property/fuzz, differential oracle, allocation 단계별 failure injection, scheduler interleaving, packaged integration, 실제 GPU/입력/audio target 테스트를 계층화한다. critical invariant는 의도적 결함 주입으로 하네스가 실패를 검출하는지도 확인한다. 예를 들어 barrier 누락·잘못된 generation·중복 callback·streaming 조기 free·한 tick 입력 중복을 넣은 negative control은 반드시 실패해야 한다.

PR gate는 작은 deterministic subset, 야간은 seed·stress·sanitizer·failure matrix, release gate는 지원 OS/API/packaged/PIE 조합을 실행한다. 실패 seed·최소 재현 입력·로그·trace·artifact hash·toolchain·build/source/asset revision을 저장한다. flaky test는 재시도 통과로 숨기지 않고 원인을 수정한다. quarantine한 필수 테스트는 통과 집계에서 제외하며 대체 근거 없이 출시 Gate를 닫지 않는

다.

ID	하네스 자체 검증
HT01	작업→계약→test ID→실행 결과의 추적 누락 0
HT02	정상/실패/경계·상태 전이·소유권 테스트 및 독립 oracle 확인
HT03	지정 fault/negative control이 실제 실패로 검출됨
HT04	동일 seed·clock·executor schedule 재실행의 실패 경로 재현
HT05	fake/device·CPU/GPU·Win/Mac·지원 API의 PASS/FAIL/NOT RUN 분리
HT06	test 종료 자체의 task/root/allocation/handle 잔존 검사·보고서 검증

표 48

line coverage는 누락 후보 탐색 수단이며 안정성 증명이 아니다. 수명·오류 경로와 상태 전이의 검증을 우선한다. 검증 비용을 줄일 때는 영향 suite 선택·병렬 실행·재현 cache를 사용하고 필수 사례를 제거하지 않는다. 이 요구는 첫 C00/C06부터 적용하며 엔진 구현을 시작한 뒤 추가하는 후속 과제가 아니다.

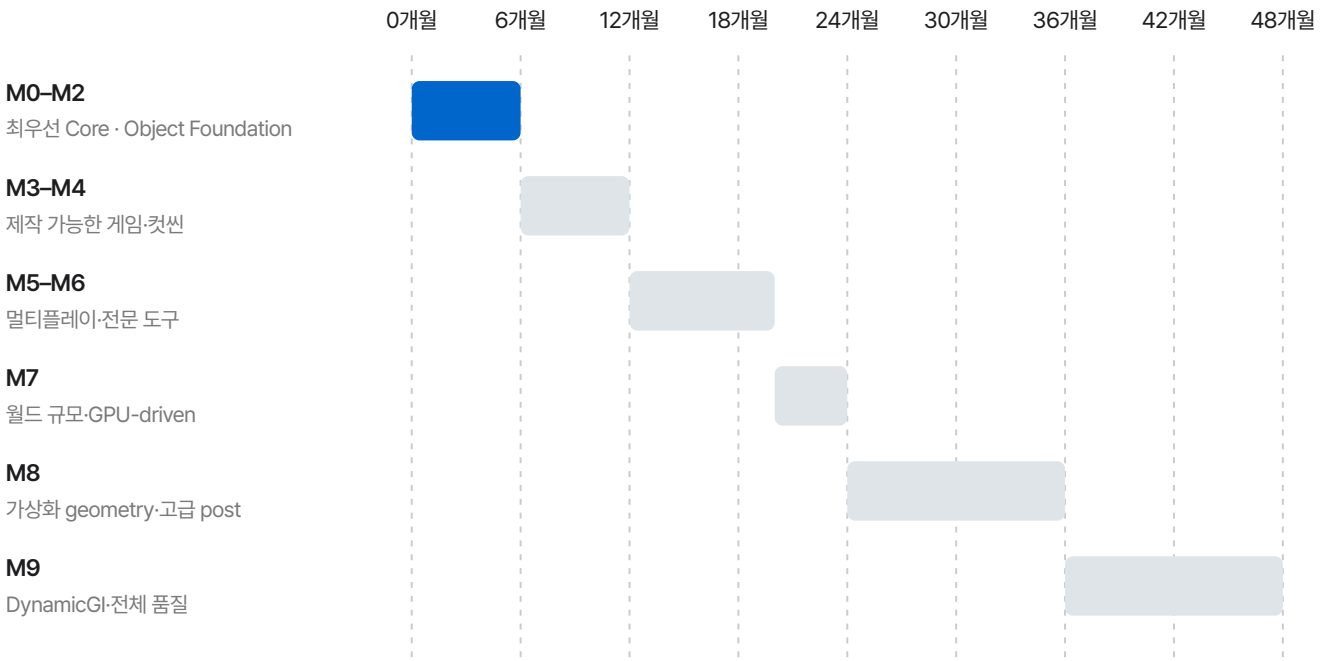
입력→화면 지연, 에셋별 검증·비용 피드백, 시스템 간 예산 조정은 AAA04–AAA06(33.6절)의 공통 상관 ID·측정 상태·시험 기준으로 확장한다.

30 단계별 마일스톤과 인력

30.1 일정 가정

기존 일정은 **Core Foundation 선행 6개월, 전체 48개월의 미측정 참고선**이다. 5종 RHI·macOS full Studio·bake·템플릿·진단 요구가 추가된 현재 범위의 확정 일정으로 사용하지 않는다. Foundation 결과와 각 전문 기능 spike를 근거로 인력·기간을 다시 산정한다. 첫 작업은 Core/Object 전담 3–5명이 수행하며 후속 전문 인력은 gate 이후 단계적으로 합류한다. 완전 확장 단계는 전담 16–22명과 별도 콘텐츠/TA·QA 지원을 가정한다. Core/Object 3–4, Renderer 3–4, Asset/Editor/UI 3–4, Gameplay/Network/Physics/Animation 3–4, Language 2, Build/Infra 2, QA automation 1–2의 역할을 일부 겸임한다. 실제 인력 배치·콘텐츠 검증 능력에 따라 기간을 재산정한다.

아래 개월은 순차 gate를 설명하는 기준선이다. 추가 요구를 반영하여 1.0의 36개월 가정을 48개월로 개정한다. 범위·인력·콘텐츠 난도에 따라 42–60개월 이상으로 재산정할 수 있으며 UE5 전체 동등성의 달성 시한이 아니다. 1–3명 규모는 M4의 작은 제품까지 범위를 축소하고 고급 연구를 장기로 분리하는 별도 일정이 필요하다. 승인된 예산이나 인력 확정으로 해석하지 않는다.



기존 48개월 참고선 · 추가 범위 재산정 · Core/Object Gate 우선

그림 4 · 기준 마일스톤 (개념도)

단계·기준 기간	선행 조건	핵심 산출물	통과 gate
M0 · 1개월	착수	Core bootstrap·PlatformBase·allocator·Array/Span·assert/trace·CPU runner, 최소 객체 계약	F0–F2 초기 fixture, GPU/웹/언어 VM 의존 0
M1 · 2–3개월	M0	Core library·thread/task/IO·Registry·handle/reference·manual TypeInfo·Object 생성/파괴	F1–F4 정확성·stress·no-op baseline
M2 · 4–6개월	M1	ClassDefaults/instancing·full GC·barrier API·Archive·Clang 최소 생성기·Foundation 통합	CF01–CF12·B0/B1 정확성 통과, 후속 착수 gate
M3 · 기존 7–9개월 참고	Foundation Gate	Rocks Build/local cache·asset/DDC·5종 RHI/RG 기본 개발 ·Deferred/Forward·texture/builtin ·Material MAT0/MAT1·웹/UMG 기초·script L0/L1·spatial upscale	import→edit→save→cook→run, Win/Mac Studio·CPU headless·기본 PIE·iteration trace
M4 · 기존 10–12개월 참고	M3	5종 RHI 기본 Gate·lightmap/probe·정식 PIE·템플릿·애니메이션·물리/audio/input/AI·UMG runtime/Designer·Material MAT2·ScriptComponent·TAAU·기본 nested 컷씬·post, Incredibuild/분산 worker	gameplay+2분 컷씬, 두 렌더 경로 ·API/TX/BR/PI 전체 기본 profile 및 LT/TT/MT의 단계별 필수 subset·offscreen·UI callback·build adapter

단계·기준 기간	선행 조건	핵심 산출물	통과 gate
M5 · 13–16개월	M4	replication/subobject·travel·nested prefab·능력 기초·script debugger/async·FSR·UMG binding/animation·Material layer MAT3·adaptive probe/scenario·고급 post	B3/B5/B6/B8·I0–I6, GC 병렬/점진 최적화·agent 장애·headless server
M6 · 17–20개월	M5	컷씬/capture·EXR/AOV·HDR/post·DLSS/XeSS·ability/feature·UMG 전문 Designer·Material MAT3·기본 고급 재질	B7·UMG/headless GPU 회귀·script reload·feature teardown·link profile
M7 · 21–24개월	M6	World Partition·HLOD·대량 AI·rig/retarget·terrain·PCG·audio graph, GPU-driven 정식화	B4, long traversal·teleport·producer 경험·SoA budget
M8 · 25–36개월	M7	VirtualGeometry·virtual shadow/texture 연구, advanced lens/post·denoiser	page fault fallback·crack·memory·두 경로 호환, 효과 catalog 중간 승인
M9 · 37–48개월	M8 및 GI 사전 연구	DynamicGI 고도화·reflection/RT·high-end post catalog·품질/출시 안정화	B9, 전체 기능 traceability, production sample·장시간 QA

표 49

M8/M9의 고급 기능은 M3부터 별도 prototype scene으로 타당성을 평가하되 main branch 제품 gate에 조기에 결합하지 않는다. 연구 실패 시 conventional mesh·probe/기본 GI·표준 shadow를 유지하여 제작 가능한 제품을 보존한다. 해당 연구 기능은 미완료로 표시하고 일정·인력을 재산정한다.

30.2 임계 의존성과 병행 경로

임계 의존성: Core library·기반 Task → 최소 TypeInfo·객체·참조·정확한 GC → Foundation Gate → 메타/언어·Rocks Build·asset/prefab → UMG/에디터·샘플 게임/컷씬 → 규모·속도 최적화 → 고급 renderer. M2는 Object/GC/native schema 계약을 고정하며 완전한 VM·에디터를 Foundation 선행 조건으로 요구하지 않는다.

RHI·웹·언어·전문 빌드 도구의 본격 구현은 Foundation Gate 뒤에 착수한다. 이후 animation/physics/audio adapter는 POD fixture로 World 완성 전 검증할 수 있다. 웹 timeline·Inspector·UI Designer·Material graph는 mock schema로 준비할 수 있으나 실제 native edit·undo·preview gate를 별도로 통과한다. GPU 연구 prototype은 독립 benchmark로 관리하고 stable renderer에 통합할 때 두 경로·upscaler·post·cutscene 회귀를 모두 수행한다.

각 gate의 완료는 'API가 존재함'이 아니라 실행 artifact·재현 scene·성능 trace·test report·지원 제한·샘플 제작 과정을 포함한다. 담당자는 미완료 항목과 다음 단계로 이월할 근거를 기록하며 기능 이름만 체크하지 않는다.

31 위험·우선순위·대응

ID·우선순위	실패 조건·관찰 신호	예방·완화와 판단 시점
R01 · P0	참조 변경 barrier 누락·GC 조기 회수	mutation API 제한·meta 진단·full oracle. M2 정확성 미통과 시 점진화 중단

ID·우선순위	실패 조건·관찰 신호	예방·완화와 판단 시점
R02 · P0	자체 container/lock-free/allocator의 UAF·ABA·overflow	단순 잠금 기준 구현·sanitizer·fuzz·epoch 검증. 성능 이전 정확성 gate
R03 · P0	VM와 native 사이 수명 누락·언어 개발 범위 확대	단일 객체 GC·root provider·제한 언어/인터프리터 우선. M2 native/GC 계약 검토, M3/L1 VM 통합에서 범위 재산정
R04 · P0	Type/Field ID drift·prefab override 오적용·save 손상	stable ID·3-way rebase·schema fixture·atomic save. migration 없이 release 금지
R05 · P0	task capture·module unload·cancel 완료 누락	owning capture·root token·drain gate·exactly-once terminal stress
R06 · P1	Deferred/Forward 기능 차이·shader permutation 증가	공통 IR·기능 manifest·cooker 오류·pairwise/full 주요 matrix. M3부터 회귀
R07 · P1	temporal upscale·DOF·motion blur·렌즈 효과 ghosting	motion 규약·보조 buffer·cut reset·temporal 시퀀스. M4부터 영상 검토
R08 · P1	native viewport와 웹 UI의 focus/DPI·레이어 문제	Foundation 이후 M3 초기 spike·다중 monitor 검증·host ownership. 필요 시 dock 구조 조정
R09 · P1	컷씬 seek/skip의 side effect·상태 복원 누락	pure evaluation·event ledger·control token·prefetch. M4 필수 데모
R10 · P1	오디오 deadline·physics/nav lock 경합	RT 무할당·audio mailbox·physics read gate·nav context 격리·callback 지연 추적. B3 장시간 테스트
R11 · P1	streaming·GC·render·audio의 해제 순서 불일치	lease/root/epoch/fence를 각 도메인별로 완료 확인. 강제 unload stress
R12 · P1	모든 high-end 효과의 비용·품질 조합 확대	catalog·tier·자동 matrix·effect별 GPU budget. 단계별 완료·전체 M9 승인
R13 · P2	virtual geometry crack·page miss·material 비용	parent fallback·page pin·독립 cook/test·conventional LOD. M8 기술 gate
R14 · P2	GI leak/noise/coverage·비싼 RT/SDF 제작	probe baseline·시각 debug·콘텐츠 제한·caps 분리. M9 이전 장면 검증
R15 · P2	ThirdParty STL·ABI·toolchain 변경 전파	adapter·고정 manifest·allocator 소유권·Host/Target 분리·업데이트 fixture
R16 · P2	기능 증가로 Foundation·제작 검증 지연	Core/Object gate 우선·M4 game/cutscene·다중 backend·Mac 개발 범위를 반영한 전체 일정 재산정
R17 · P0	compile/DDC cache stale hit·원격 늦은 output 오염	완전한 key·원자 artifact set·session ID·local/clean 대조
R18 · P1	원격 compile만 개선하고 link/restart가 병목 유지	I0–I6 전체 시간·module link/PDB·실행 복구·비native 반복 경로
R19 · P1	NullRHI로 영상/UI 성공을 오판	CPU-only/GPU offscreen profile·capability 실패·실제 image 비교

ID·우선순위	실패 조건·관찰 신호	예방·완화와 판단 시점
R20 · P1	UMG designer/runtime 차이·binding root 누수	동일 evaluator/layout·typed binding·preview/cooked/headless 통합

표 50

기능 계획의 P0는 안정성 선행 조건, P1은 제작·실행 필수, P2는 규모·품질 확장, P3는 연구·특수 기능이다. 위 위험표의 우선순위는 대응 시급성으로 별도 사용한다. 후속 전문 기능을 P3로 분류하여도 제품 범위에서 삭제한 것으로 해석하지 않는다. 다만 첫 실용 버전과 장기 전체 범위의 완료 시점을 구분한다.

32 착수 4주 실행 항목과 요구 추적

32.1 첫 4주

주차	결과물	확인 기준
1주	Core 전용 target·PlatformBase·assert/log·CPU-only runner, allocator·Object contract	새 checkout bootstrap·금지 의존·headless 실행·실패 재현
2주	기준 heap/page·Array/Span·고정 ID·allocation ledger/trace	정렬·overflow·실패 주입·nontrivial lifetime fixture
3주	Atomic/queue/Task 최소형·청크 Registry·generation·NewObject/Destroy	task terminal·publish·stale handle·반복 생성/삭제
4주	Root/Object/Weak·manual TypeInfo·ClassDefaults/visitor·full GC prototype	root/순환 참조/약한 pin·instancing·종료 테스트, CF backlog 확정

표 51

4주 항목은 첫 Core 작업의 prototype이며 production Foundation 완료는 CF01–CF12로 판정한다. 이 4주에 언어 VM·renderer·viewport·UI Designer·분산 build를 추가하지 않는다. 결과는 설계 결정 기록(ADR), 실행 sample, test report로 보존한다. 최우선 ADR은 STL/예외·allocator·참조/GC·Registry publication·Task shutdown·TypeId/FieldId·instancing 계약이다. renderer·언어·웹/UMG·컷씬·build executor ADR은 해당 Foundation 계약을 입력으로 후속 작성한다.

32.2 요구사항 추적표

요구	상세 절	핵심 완료 gate
C++23·STL 비사용 Core·할당기	2–4	M1 / B0
Object·청크·Nonmoving 병렬·점진 GC	5–6	M2 정확성, M5 최적화 / B1
Clang 메타·직렬화·schema	7	M2 / migration corpus
Task·비동기·수명·trace	8·29	M1부터 / B0·B1·전체 stress
비동기 에셋·쿠킹·DDC·패키지	9·27	M3 / clean package·corrupt data fixture
Scene·World Partition·큰 월드	10	M3 기반, M7 규모 / B4
Deferred·Forward 둘 다 정식	11	M3 기본, 전 단계 기능 matrix / B2

요구	상세 절	핵심 완료 gate
업스케일·동적 해상도·AA	12	M4–M6 / B8
Static/Skeletal·Animation·GPU-driven	13·15	M4 기본, M7 정식 / B2·B3
Nanite/Lumen 유사·shadow/VT	13–14	M8–M9 / B9
충돌·물리·이동	16	M4 / B3·B6
네트워크·예측·replay	17	M5 / B6
오디오·nav·AI	18–19	M4 기본, M7 확장 / B3
태깅·컴포넌트·프리팹·게임플레이	20	M4 기본, M5 중첩·변형 / B5
VFX·UI·환경·툴	21–22	M4 기본, M6–M8 확장 / B5·B7
Blueprint 제외·자체 언어	23	L0–L4 / VM corpus·script sample
Unreal 참고 체계 추가	24	항목별 M2–M8 / integration fixture
컷씬 제작·실행·출력	25	M4 핵심, M6 완성 도구 / B7
하이엔드 post 전체 기능군	26	M4–M9 catalog / B7·B9
빌드·반복 개발 최적화	27	Foundation 후 M3–M5 / I0–I6·B10
헤드리스·server·offscreen	28	CoreTest부터 / B11
UMG runtime·UI Designer	21.2–21.5	M3–M6 / B12
테스트·프로파일링·위험	29–31	전 milestone / 재현 artifact
첫 작업 Core/Object	1.1–1.2·4–7·32.1	M0–M2 / CF01–CF12

표 52

33 전체 검토 결과·Unreal 비교·잔여 항목

33.1 개정 1.1의 설계 정합성 수정

아래 '수정'은 계획서의 계약과 선행 관계를 정리한 상태다. 엔진을 구현하여 오류가 없음을 입증한 의미가 아니다. 실제 구현 완료는 각 gate의 실행 결과로 판정한다.

ID	1.0에서 확인한 충돌·미명시	1.1 수정과 검증 위치
A01	첫 작업에 Core 외 언어·viewport prototype이 함께 배치됨	첫 작업을 F0–F6 Core/Object로 고정, CF01–CF12 뒤 후속 구현. 1·30·32절
A02	scheduler 제출 차단 뒤 자원 정리 task가 필요한 종료 순서	일반 admission 차단·teardown capability 허용·마지막 drain 후 scheduler 종료. 3·8·28절
A03	generation만으로 lifetime/race 안전성을 보장할 위험	Root/Borrow·GC gate·WorldId/Epoch·논리 destroy를 구분. 5–6절

ID	1.0에서 확인한 충돌·미명시	1.1 수정과 검증 위치
A04	GC 최종 정지 구간의 전체 슬롯 Retiring scan이 시간 예산과 충돌	atomic epoch cutoff·pin 금지·chunk별 후속 sweep·root/VM flush. 6.4절
A05	비예외 container와 throw 가능한 생성/이동의 계약 누락	noexcept element 규약·Result Initialize·합법적 relocation 제한. 4.3절
A06	C++ 함수 본문 무재시작 갱신에 필요한 code lifetime 부재	native 기본 재시작, script/shader/asset만 compatible reload, native patch는 별도 연구. 7·23·27절
A07	RHI Submit 시그니처가 실패를 반환할 수 없음	`Result<SubmissionResult>` 로 통일. 11.2절
A08	Forward에서 current-color SSR/SSGI를 opaque lighting 이전에 계산	base opaque color 이후 trace·별도 contribution 합성, graph cycle 거부. 11.4절
A09	HZB culling이 자신의 depth prepass 비용을 줄이는 것으로 해석될 여지	예비 culling→depth→current HZB→후속 culling으로 명시. 11.4절
A10	view별 shadow projection 차이와 공유 조건 누락	light/caster/projection/coverage가 같은 view family만 공유. 11절
A11	카메라 exposure와 global HDR output의 상태 구분 모순	View exposure/history와 OutputSurface color/metadata 분리. 26.2절
A12	같은 post stage alias·동일 priority 실행 순서 미정	stage alias 통일·dependency/priority/stable ID 정렬·compatibility history key. 26.5절
A13	nested sequence를 M6에 도입하면서 M4 데모에 요구	기본 nesting M4, 고급 nesting/curve/take M6로 정합. 24–25절
A14	take recorder M6와 M7 도입 설명 불일치	basic transform/camera/event M6, mocap 연계 M7 이후. 24–25절
A15	물/volume 합성 일정에 필요한 원신호 생성기 일정 부재	water/fog M7·cloud M8·효과 입력 manifest와 연결. 21·26절
A16	immutable physics query snapshot이 구현된 것처럼 가정	live broadphase ReadScope와 step-boundary async batch로 명시. 16.3절
A17	서버의 substep overflow가 tick/input drop으로 이어질 위험	authoritative tick 유지·과부하/시간 정책 protocol/replay에 기록. 16–17절
A18	default subobject 공유·nested prefab ID alias 가능성	instance별 clone/remap, instance 경로 포함 stable key. 5.3·20.5절
A19	VM 중단·오류가 native lock 선점/외부 효과 rollback으로 해석될 위험	safe poll boundary·native call 계약·staged transaction·revision lease. 23.4절
A20	모든 asset ready가 GPU를 요구할 가능성	target mask·server bundle·unsupported 검증, AudioRT 일반 callback 금지. 9.4·28절
A21	DDC와 source bulk virtualisation의 내구성 혼동 가능성	authoritative 원본 bulk store·재생성 DDC·build cache 분리. 9·27절
A22	빌드가 CMake orchestration에 한정되어 새 요구 미충족	UBT 계열 Rocks Build·Incredibuild/선택 executor·link 독립 최적화·I0–I6. 27절

ID	1.0에서 확인한 충돌·미명시	1.1 수정과 검증 위치
A23	모든 headless를 NullRHI로 처리할 위험	CPU-only·Server·Backend·GPU offscreen capability와 결과 구분. 28절
A24	UI runtime 설명에 비해 편집·binding·cook 완료 기준이 약함	WidgetAsset/Designer·자체 언어 controller·IME/focus·layout/pixel tests. 21.2–21.5절

표 53

33.2 Unreal 비교의 기능 범위

'필수'는 전체 제품의 계획 범위, '후속'은 명시적 확장, '제외'는 사용자 정책 또는 별도 제품 영역이다. 모든 행의 구현 상태는 현재 **계획 / 미구현 검증**이다. 공식 문서에 있는 기능을 이 계획서에 기재한 것만으로 동등 지원으로 판단하지 않는다.

기능군	반영 범위·계약	완료 단계·추적
Core·메모리·Task·Object·Reflection·GC	첫 작업, archetype/default subobject·참조·세대·수명·schema	필수 M0–M2 CF gate, 병렬/점진 최적화 M5
빌드·코드 생성·분산·캐시·link	Rocks Build·local/remote·Incredibuild adapter·PCH/Unity·iteration trace	필수 Foundation 후 M3–M5, 고급 link M6+
Headless·server·commandlet·offscreen	capability별 target·결과·timeout·drain	필수 CoreTest부터 단계별 확장
Framework·입력·camera·save·travel	MatchRules/State·PlayerState·LocalPlayer·possession·epoch·snapshot	필수 M4–M6
Tag·component·prefab·feature plugin	계층 query·등록/tick·variant/rebase·load/unload token	필수 M4–M6
커스텀 언어·debugger·RPC·reload	static type·verified VM·GC roots·native binding·언어 서비스	필수 Foundation 후 L0–L4
Blueprint·범용 gameplay visual scripting	UI/AI/sequence 이벤트도 C++/자체 언어로 연결	명시 제외
Asset Manager·DataAsset/Table·DDC·virtual assets	primary/bundle·stable schema·source bulk·incremental import/cook	필수 M3–M6
Deferred·Forward·RHI/RG·shader·GPU Scene	두 경로 정식·공통 IR·typed pass/queue/fence·capability	필수 M3부터 전체 회귀
필수 머티리얼 에디터·instance/function/layer	node graph·live preview·typed IR·진단·비동기 compile·DDC·cook	필수 MAT0–MAT2 M3/M4, MAT3 M5/M6; 14.5–14.9 MT gate
고급 surface·transparency·capture	skin/SSS·clear coat·cloth·eye·foliage·water·planar/scene capture	필수 M6–M8, layered closure 별도 연구
VirtualGeometry·GI·VSM·VT·RT	대체 경로·page/representation·memory·capture 지원표	필수 장기 M8–M9 연구 gate
전체 post·upscale·HDR	26절 catalog·두 경로·FSR/DLSS/XeSS·volume/shot·history	필수 M4–M9

기능군	반영 범위·계약	완료 단계·추적
Skeletal·animation·rig·retarget·morph	blend/montage/sync/socket·IK/warp·pose search·skin/velocity	필수 M4–M8, 고급 cloth/groom P3
Physics·collision·query·nav	channel/material·constraint·character·async query·tile snapshot	필수 M4–M7; 파괴/softbody/vehicle 개별 P3
AI·EQS·State/Behavior·SmartObject·Mass	task budget·script predicate·예약·SoA/Entity bridge	필수 M4–M7
Gameplay Ability·effect·cue	태그·속성·stack·cost·duration·prediction·취소	필수 M5–M6
Network·subobject·replay·online	authority/conditions·spawn/fixup·travel/reconnect·provider	필수 M5–M7, 서비스 운영 통합 별도 gate
Audio·DSP·dialogue·clock	voice/bus·concurrency·graph·sample scheduler·offline mix	필수 M4–M7
UMG 계열 runtime/Designer	widget hierarchy·canvas·binding·animation·IME·focus·cook	필수 M3–M6, Blueprint 제외
VFX·simulation cache	CPU/GPU·ribbon/mesh·data interface·reaction/motion·scrub	필수 M4–M8, 고급 fluid/sparse volume 후속
Landscape·spline·foliage·PCG·환경	edit layer·terrain/collision/nav·stable generated IDs·water/cloud	필수 M7–M8
모델링·DCC·geometry cache	blockout/UV/collision·geometry API·glTF/FBX·선택 USD/Alembic	필수 기본 도구 M6–M7, 고급 interchange M8+
Sequencer·camera·take·render jobs	기본 nested·dialogue·events·skip·AOV/EXR·tile/resume	필수 M4–M6, mocap·reference path trace 후속
Editor transaction·source control·PIE	asset diff·journal·conflict·script/UI designer·World 격리	필수 M3–M6
Test·trace·crash·localization·quality	CPU/GPU/audio/script/build timeline·headless matrix·schema/텍스트	필수 첫 Core 작업부터
Multi-user/remote collaboration	revision·transaction을 기반으로 후속 동시 편집	후속, 단독 편집/저장 gate 이후
Mobile·console·XR/stereo·Linux GPU editor	platform/caps·입력·UI·SDK·렌더 profile 확장	후속 플랫폼 gate; 기본 Windows/macOS 개발·Linux server 범위와 구분
nDisplay·DMX·LED stage·전문 방송	일반 게임/컷씬과 다른 장비/동기화 제품 범위	별도 확장, 현재 기본 완료 조건에 미포함

표 54

33.3 잔여 설계·성능 판단

1.1 검토와 1.2 독립 교차 검토의 발견 사항은 본문 계약으로 보완하였다. 계획 수정만으로 구현 정확성이 입증된 것은 아니다. 구현 전에 확정할 항목은 language 상세 문법, GPU별 최소 caps/최종 SDK, vendor adapter의 실제 조합, BVH/SDF GI 경로, 고급 재질 encoding, 전문 플랫폼 범위다. 이를 이미 결정된 지원으로 기록하지 않는다.

특히 0.5 ms GC slice, 60Hz의 전체 high-end profile, I0–I6 iteration 목표, 많은 효과의 동시 사용 품질은 실측이 필요하다. Core Foundation의 정확성 gate와 연구·속도 gate를 구분하고 실패 원인을 기록한다. 미지원/미검증 기능은 feature manifest에서 정식 완료 집계에 포함하지 않는다.

1.0의 36개월 기준선은 추가 빌드·UI·헤드리스·고급 콘텐츠 요구와 Core 우선 인력 배치를 반영하여 **48개월 기준 계획선**으로 개정한다. 30절의 인력·일정은 미확정 제안이며, M0–M2의 Foundation 결과와 실제 build/iteration 측정 후 다시 산정한다. 기능 목록의 증가만으로 기존 일정에 모두 완료되는 것으로 처리하지 않는다.

33.4 서브에이전트 교차 검증과 개정 1.2

검토 입력은 개정 1.1 원본이다. Core/Object 담당은 6건, 렌더링·콘텐츠 담당은 5건, 빌드·헤드리스 담당은 6건을 제시하였다. Core-first 일정 중복 1건을 통합하여 **필수 수정 16건** (P1 8건·P2 8건)으로 정리하고, API 상세화·고급 조명·문구 통일의 **별도 보완 3건**을 반영하였다. P1은 해당 시스템 구현 전 안전성 계약 확정, P2는 통합 전 의미·일정 일치가 필요한 항목이다. 구조 전체의 즉시 재작성 사유인 P0는 이번 검토에서 확인하지 못하였으나 모든 결함의 부재를 증명한 것은 아니다.

각 담당은 수정 전 원문 위치·실패 시나리오·수정안·검증 기준을 독립 기록하였다. 종합 담당자는 본문과 공식 근거를 대조한 뒤 수정하고 담당자에게 수정 초안을 다시 검토시켰다. Core 파괴 계약은 빌드·헤드리스 담당에게도 재대조하여 admission 차단 순서·cleanup reader 경계·retire self-root 금지를 추가하였다. 2차 검토에서 RootSnapshot 초기화·asset unload/fence 순서, 늦은 GPU 제출의 fence/lease 원자 이관, 컷씬 임시 제어권의 AlwaysRestore까지 정정하였다.

조치 ID·우선순위	원검토	확인한 문제·보완	본문 조치	구현 시 필수 검증
X01 · P1	C01	cutoff 뒤 Resolve/edge 연결이 pin/root 차단을 우회	6.5 공통 판정·gate·Borrow 경계	cutoff 직후 모든 acquisition 거부
X02 · P1	C02	지연 sweep 중 epoch/bitmap 재사용 규약 누락	6.5 단일 분류 cycle·별도 retire record	chunk 중단·새 cycle·지연 fence
X03 · P1	C03	명시 destroy와 strong 보존·reader cleanup 조건 불일치	5.2 admission 우선·보존/논리 종료 분리·6.5 self-root 금지·9.2 해제 순서	재진입·reader 정지·root/fence 순서·1회 소멸
X04 · P1	C04	미완성 객체·자식 graph의 GC 보호 방식 누락	5.4 constructor no-safepoint·ConstructionContext	단계별 강제 GC·생성/remap 실패
X05 · P2	C06	implicit-lifetime을 relocation 보장으로 오해 가능	4.3 portable trivially-copyable 조건·확장 분리	nontrivial/self-pointer/over-aligned fixture
X06 · 보완	Core 추가 확인	Task 제출 거부·실패 의존성 및 GC worker 진행 상세화	8.1 Result admission·RunOnTerminal, 6.5 GC 참여	shutdown·cancel 전파·1/2-worker 진행
X07 · P2	C05 + BH05	M0 언어/viewport·M2/L1 표기가 Core-first와 충돌	23.1·31절·SDK 착수 시점 통일	M0–M2에 후속 구현 선행 없음
X08 · P1	RC02	recording부터 Submit 이전의 GPU 자원 보존 공백	11.2 RenderResourceLease·취소/부분 제출·device loss	record pause와 unload/실패 경합
X09 · P1	RC01	occlusion 과오 방향 반대·이전 HZB 탈락 복구 미정	11.4 현재 frame recovery·13.1 $V_{ref} \subseteq V_{submit}$	느린 이동·이동 occluder·가시 누락 0건

조치 ID·우선순위	원검토	확인한 문제·보완	본문 조치	구현 시 필수 검증
X10 · P2	RC05	TRS와 affine/shear 및 KeepWorld 실패 규약 누락	2.3 Core Transform·10.1 transaction 오류	비균일/음수/0 scale·reparent/Undo
X11 · P2	RC03	컷씬 prefab NodeGuid 단독 주소의 중첩 충돌	25.1 PrefabNodeAddress·25.2 sequence instance path	동일 prefab/sequence 2회 중첩
X12 · P2	RC04	중복 sequence의 종료 순서별 상태 복원 미정	25.2 공통 base·owner 집합·Keep/Restore resolver	A/B 양 종료 순서·gameplay base 갱신
X13 · P1	BH01	Detour query state·이웃 tile 링크 격리 누락	19절 전용 context·초기 read/write gate·독립 snapshot	sliced query·tile 교체/unload·TSan
X14 · P1	BH02	유한 audio ring의 critical 무조건 보존·void 제출	18절 TryEnqueue·Backpressure·세대 mailbox·ACK	포화·stop storm·slot 재사용
X15 · P2	BH03	runtime package 순환과 cook DAG/hash의 관계 미정	9.1 참조/산출물 graph 분리·SCC key	2/3-node cycle·clean/incremental 비교
X16 · P2	BH04	과거 include closure만으로 cache hit 판정 가능	27.2 negative lookup·탐색 경로·재스캔	shadow/optional header·branch 전환
X17 · P2	BH06	M4 headless gate가 M5 multiplayer를 선행 요구 가능	28.5 boot/simulation·16-client 단계 분리	suite manifest·완료 범위 보고
X18 · 보완	RC06	고급 직접 조명의 제작 기능 범위 보강	14.2 Rect/area·IES·광량 단위·cookie	양 렌더 경로·조도/분포/그림자
X19 · 보완	RC 문구 통일	history key의 profile revision 표현 잔존	26.2 HistoryCompatibilityKey로 통일	연속 값은 유지·layout 변경 reset

표 55

위 19개 조치의 상태는 **설계 반영·실행 검증 대기**다. 서브에이전트의 수정안 재검토는 문서 계약이 제시된 반례를 처리하는지 확인한 결과이며 엔진의 unit/stress/영상/성능 시험을 통과하였다는 의미가 아니다. 기본 요구와 기존 장을 삭제하지 않았으며 첫 Core/Object 작업·CF01–CF12·Blueprint 제외·커스텀 언어를 유지한다.

Core 착수 시 X01–X07·X10을 CF gate의 구체 fixture로 우선 구현한다. X08–X09는 RHI/culling, X11–X12는 컷씬, X13–X17은 각 서비스/도구의 첫 통합 gate에 포함한다. 48개월 기준선과 모든 성능 목표는 계속 미확정·미측정이며 검토 건수만으로 완료 일정이나 안정성을 보장하지 않는다.

33.5 개정 1.3의 머티리얼 에디터 필수화

기존 14.1절의 shader compiler와 22절의 전문 도구 언급만으로는 편집 기능의 완료 범위가 불명확하였다. 14.5–14.9절에서 그래프·instance·function/layer·미리보기·진단·compile/cook를 각각 정의하고 M3/M4의 필수 제품 Gate와 연결한다. Blueprint 제외는 gameplay visual scripting 정책이며 머티리얼 그래프를 제외하는 근거로 사용하지 않는다. 첫 Core/Object Gate는 그대로 유지한다. 렌더링 담당 독립 검토에서 함수 호출별 parameter identity와 Host-only MaterialCook 경계를 보완하였다. 이는 문서 계약 검토이며 실제 에디터·GPU 실행 결과가 아니다. 새 요구의 인력·일정 영향은 MAT0 결과에서 재산정하며 기존 48개월 계획선을 확정 납기로 간주하지 않는다.

33.6 상용 AAA 엔진 공개 기술과의 범위 비교

Snowdrop, IW 관련 Call of Duty 기술, RAGE의 공개 개발 자료를 비교하여 **10개 계약 보완 항목**을 계획에 반영한다. 아래의 필수는 Rocks의 제품 요구이며 해당 엔진 내부 API의 복제를 뜻하지 않는다. 게임 적용 사례·엔진 공통 기능·스튜디오 제작 인프라를 구분한다. 공개되지 않은 GC·객체 ABI·전체 build time·콘솔 구현은 unknown으로 유지한다. 일반 오픈소스 엔진의 비교를 추가하지 않는다. 기존반영은 계획에 포함된 상태이며 구현 완료를 의미하지 않는다.

ID·Rocks 보완 범위	공개 근거·범위	핵심 계약·도입 단계	필수 수용 기준
AAA01 · 지속 월드 상태·환경 필드	Massive, An Evolving Pandora: 시설 상태에 따른 기하·이동 영역 및 게임플레이/셰이더 환경 값. Division 2→Avatar의 게임 적용 사례; 발행일 미표시, 2026-11-10 확인	**필수 보완, P2/M7–M8.** 기존 Data Layer·save·replication에 `WorldStateSet` 의 stable ID/schema/authority/revision을 연결한다. asset·collision/nav 준비 후 simulation 경계에서 활성화하고 render/audio는 명시 revision과 지연 정책을 사용한다. 게임 규칙은 커스텀 언어로 작성	상태 전환 중 save/load·late join·cell unload/reload·실패/취소를 시험. 오래된 완료의 새 상태 덮어쓰기·권한 불일치·잔존 lease 0건; Game lane의 GPU/audio 완료 대기 금지
AAA02 · 절차 배치와 수동 보정 보존	Massive, Crafting Pandora: 다층 표면·관계 기반 배치와 생성 후 편집. Snowdrop 공통 기능 편입을 명시; 발행일 미표시, 2026-11-10 확인	**필수 보완, P2/M7–M8.** 기존 PCG에 surface provider·영향 영역·cell 경계 소유권·생성 provenance·수동 override/rebase를 추가한다. 게임 플레이 시각 언어를 도입하지 않는다	동일 seed/revision의 editor/headless·작업 순서별 stable ID 집합 일치. 재생성 후 수동 수정 보존, 충돌 명시, 경계 중복·stale publish 0건
AAA03 · RT/cache용 재질 표현	Ubisoft 개발자, Scene Representation and Custom BVH, GDC 2024: Avatar 규모의 장면·재질 표현 사례	**필수 보완, P2/M8–M9.** 기존 Material IR·RayScene에 `RayMaterialProfile` 을 추가하여 opacity/masked/two-sided/emissive·UV·time/instance 입력의 정확·근사·정적·미지원 상태를 선언한다. proxy와 cache key에 source/layout/profile revision 반영	raster/GI/reflection 차이와 근사 오차를 시각화. dynamic material 변경·cell unload·늦은 GPU 완료에서 이전 proxy 재게시 금지; 미지원 재질의 허위 정상 표시 0건
AAA04 · 입력→화면 지연 계측	Activision, Controller to Display Latency, 2020-11-06 게시: CoD 지연 측정 발표; 특정 IW 세대의 공통 사양은 확인되지 않음	**필수 보완, P1/M4부터.** InputEventId/FrameId로 입력 sample·simulation·submit·present를 연결하고 queue depth·VSync/VRR·frame cap을 기록한다. 소프트웨어 timestamp와 실제 input-to-photon 측정을 분리	CPU/GPU 병목과 표시 모드별 p50/p95/max·누락 frame 기록. present 반환을 광출력 시점으로 표시하지 않으며 입력/tick 유실이나 수명 barrier 생략으로 지연 목표 달성 금지
AAA05 · 콘텐츠 예산 조정의 통합	Activision, Lima Oscar Delta, 2020-09-21 게시: Modern Warfare의 정적·동적 콘텐츠 LOD 제작 사례	**필수 보완, P2/M5–M7.** mesh/HLOD·animation·AI·VFX·dynamic resolution의 기존 제어기를 공통 budget arbitration에 연결한다. 비용 근거·우선순위·최저 품질·히스테리시스·변화율·복구 순서 선언	이동/전투/컷씬 fixture에서 품질 진동과 budget 초과 원인을 기록. notify·root motion·서버 권한 판정 생략 금지. 결정적 시험은 resolved profile을 고정

ID·Rocks 보완 범위	공개 근거·범위	핵심 계약·도입 단계	필수 수용 기준
AAA06 · 에셋별 제작 검증·비용 피드백	Rockstar, The Sound of GTA V, GDC 2014, 24쪽: RAGE Audio 제작 도구의 자동 최적화·비용·에셋 시험	**기존반영+필수 보완, P1/M3부터.** §29 하네스·관측 체계를 `AssetValidationProfile` 로 AssetId/revision·target·검사 버전·예산에 연결한다. validator는 각 subsystem의 첫 통합 단계에 제공하고 editor/headless가 공유	의도적 불량 fixture 검출, source 수정/worker crash/중복 완료 시험. 추정·offline benchmark·실장비 측정을 구분하고 미실행 target PASS·stale 승인·원본 자동 훼손 0건
AAA07 · code/data branch 통합	Activision, MergeBot, GDC 2024-03-21 발표·2024-06-24 게시: 스튜디오 Perforce 제작 인프라이며 runtime 기능이 아님	**필수 제작 계약, P2/M5 이후.** Git/Perforce adapter의 격리된 통합 후보에 code/asset/schema diff·conflict·build/cook/test 결과와 작업 이력을 연결한다. 외부 branch 게시·충돌 해결은 명시 정책만 허용	schema rename·binary conflict·부분 실패·재시도·구버전 산출물 시험. 미검증 후보 게시·중복 적용·무관한 사용자 변경 손상 0건
AAA08 · 이동 환경 샘플·애니메이션 제작 이력	Rockstar, Making the Believable Horses of RDR2, GDC 2021: 말 시스템 사례. 공식 PDF 직접 열기는 제한되어 검색 색인의 terrain/feature 추출 구간만 확인; 범용 API는 unknown	**필수 계약 보완, P1/M4; 제작 확장 P2/M6–M7.** `LocomotionContext` 는 terrain/nav/physics revision·tick·origin epoch·허용 age와 Ready/Stale/Unavailable을 구분한다. `AnimationFeatureBake` 는 clip·extractor·좌표/단위·override provenance를 보존	nav/shape 미준비·origin 변경·cell unload·재임포트와 늦은 완료를 시험. 필수 조회 실패 시 이동 보류/승인 fallback, 오래된 결과 적용 0건. feature 자동 추정과 수동 annotation을 구분
AAA09 · 대기·반사·GI의 환경 일관성	Rockstar, Creating the Atmospheric World of RDR2, SIGGRAPH 2019 공식 초록: main view·reflection·sky irradiance 통합. 전체 PPTX 내부는 미확인	**필수 계약 보완, P1/M4; 품질 확장 M7–M8.** `EnvironmentState` snapshot에 단위·revision을 부여하고 각 소비자의 허용 age·이전 결과·history reset을 선언한다. 컷씬 override는 priority/blend/scope token을 사용하며 baked scenario와 runtime atmosphere revision을 구분	weather/capture/PIE 2세션/seek·stop/늦은 GPU 완료 시험. stale 경고·광량 중복 방지·동일 revision 기준 영상, 종료 후 override/GPU lease 0건
AAA10 · 재생 준비 시점과 실패 결과	Rockstar, The Sound of GTA V, GDC 2014, 4·10쪽: RAGE Audio의 예측 가능한 streaming·sample 시점·decode 지연 보상	**필수 계약 보완, P1/M4부터.** 기존 ReadyMask·sample clock·컷씬 gate에 `PlaybackReadinessRequest` 의 sample 구간·clock domain·pre-roll·준비 목표·MissedDeadline/Unavailable/Cancelled를 연결한다. 월드 streaming 적용은 Rocks의 후속 일반화	지연/부분 read·decode 실패·seek/stop·장치 변경·완료 재정렬 시험. AudioRT 무대기·무할당, 취소 voice 부활·허위 ready·잔존 request/voice 0건. OS/장치의 hard deadline 성공은 보장하지 않음

표 56

라이트맵·프로브 제작 과정, RenderGraph, 코드/에셋 지속 검증은 이미 필수 범위에 반영하였다. 추가 시스템 수로 중복 집계하지 않고 §14.10·§11·§29의 acceptance와 연결한다. 근거는 Infinite Warfare 사전 계산 조명, 2017-08-08, Activision Task Graph Renderer, 2023-06-05, CoD 자동 테스트, 2018-06-26이다. 게시일과 원 발표 연도는 구분하며 Activision 공통 발표를 모든 IW 버전의 기능으로 귀속하지 않는다.

선택 확장은 별도로 유지한다. Snowdrop GPU 음향, GDC 2025는 M8 이후 비동기 AcousticScene 연구이며 CPU/volume 경로와 DedicatedServer의 RHI 독립성을 보존한다. Modern Warfare software VRS, 2020-09-21는 M7 이후 고급 shading 선택이며 hardware VRS·upscale과 동등 기능으로 합치지 않는다. 장르별 자연 지형 traversal, 실측 재질 저작, ML 성능 예측은 기본 계약 이후의 선택 기능이다. 이 선택 기능의 미구현으로 Core나 기본 renderer 완료를 지연하지 않으며, 연구 결과를 실제 기기 검증으로 표시하지 않는다.

각 보완 항목의 첫 구현에는 unit·contract·fault·concurrency 하네스와 종료 후 자원 검사를 함께 배치한다. Core R0/R1/Foundation은 allocator·container·Object 수명·직렬 기준 GC·trace·test를 먼저 완성한다. AAA 비교 항목은 이 기반 위의 모듈에 추가하며 Foundation에 renderer·Studio·전문 asset tool의 역의존을 발생시키지 않는다. 공개 기능의 이름 수보다 실패 경로·수명·편집과 headless의 일치 여부를 완료 기준으로 사용한다. C++23/STL 비사용, `rocks::Object/Entity/Component`, 자체 게임플레이 언어, Blueprint 제외 원칙을 유지한다.

33.7 Rocks 추가 요구의 구현·검증 추적

필수 제품 범위와 첫 Foundation의 구현 범위를 분리한다. 아직 지원하지 않는 기능은 capability와 미완료 gate로 표현하며, 확장을 위해 첫 단계에 모든 backend·editor·SDK를 링크하지 않는다. 안정성·데이터 보존·정확성을 선행하고 그 안에서 확장성·성능을 최적화한다.

요구	규약·착수 위치	완료 근거
가독성·빈 줄·스페이스 4칸·clang-format	3.4·상세 4.3·C00/CT01	버전 고정 설정·탭 0·포맷 재실행 동일성·리뷰
Rocks 자체 명명·접두사 비교	2.4·Core 상세 4절, C++ namespace·외부 ABI 실험	public header·C/custom-language glue·symbol 충돌·과거 asset ID
검증된 수학·자체 API·SIMD	3절 수학 정책·상세 C15	MX 시험, 양 ISA·정확성/성능/컴파일 측정·vendor 타입 경계
Windows/macOS 개발·다중 플랫폼	2.5·상세 플랫폼/입력 작업	두 OS checkout→build→Studio→PIE→cook→test; native Mac CI
5종 RHI와 Deferred/Forward	11.5·14.1·M3/M4	API01–06, actual GPU 영상·지원 manifest
가변/고정 tick·dilation·멀티스레딩	8.3/8.4·Core C15/C17	TK01–06, pause/step·debt·replay·직렬/병렬 결과
texture streaming·내장 리소스	9.5/9.6·M3/M4	TX01–08·BR01–05, 실제 memory·resident mip·device 실패
노드 기반 머티리얼 에디터	14.5–14.9·M3/M4	MT01–10, compile/preview·instance·function·cook
lightmap·범용 probe	14.10·M4–M6	LT01–06, CPU baker·이동 객체·scenario·headless
템플릿·반복 제작 최소화	22.3–22.5·Core C00부터	TT01–08, 사용자 카탈로그·3-way update·복구·CLI
PIE 필수	22.6·M3/M4	PI01–06, 세션 격리·single-step·1,000회 stop 후 소유권 복귀
logging·profiler·상태 관찰	29.4/29.5·C06/C18/C26	OB01–06, 기록·분석·viewer와 손실·clock 오차 표시
누수 방지·안정성·촘촘한 하네스	29.6/29.7·모든 작업	NL01–08·HT01–06, fault·수명·병렬·negative-control 회귀

요구	규약·착수 위치	완료 근거
AAA 공개 기술 비교	33.6·후속 시스템별 gate	Snowdrop/IW/RAGE 공개 범위·unknown·자체 계약·검증 구별
Blueprint 제외·커스텀 언어	23절·전체 editor/runtime 경계	gameplay VM conformance; material·UI·cutscene 전문 편집은 유지

표 57

입력 상세안은 keyboard·mouse·gamepad·hotplug·rebinding·IME·UI/PIE routing·haptics·replay·console provider 경계를 포함한다. 콘솔 실지원은 해당 SDK·개발 장비·실제 인증 시험이 확보된 단계에 판정하며, provider skeleton만으로 완료하지 않는다. 라이선스나 장비가 없는 실행은 NOT RUN으로 남긴다.

위 항목은 계획상의 필수 gate다. 유닛·하네스·영상·성능 시험의 실행 결과는 구현 후 산출하며, 문서 교차 검증과 구분한다.

33.8 개정 1.3 최종 계약 교차 검증

서브에이전트 3개가 Core/Object, 렌더·콘텐츠·수학, 빌드·헤드리스·플랫폼 관점에서 통합본과 상세안·작업 데이터를 대조하였다. CF ID 문제의 중복을 합친 16개 보완 사항을 반영하고 지적 반례를 다시 확인하였다. Resolve context, publication 이후 실패, 최종 GC/drain, Foundation 단계·ID, CPU bake 재질, 논리 mip, GPU device domain, PIE scene 격리, 메모리 포화 진척, host별 build fallback, template TypeId, 입력 통합 조건을 구체화하였다. 별첨 교차 검증 결과에 항목별 반영 위치를 기록한다.

이후 담당 영역을 교환한 2차 검토에서 정상 AssetLease의 unload admission, task capture와 완료 observer의 수명 분리, metadata 소유 복사, InputReadKey의 scope/clock 격리, Sync.h의 C02 도입, formatter/config 캐시 입력을 추가 보완하였다. 3차 역방향 검증에서는 queue 포화 시 cleanup 진척과 asset callback의 gate 재진입을 추가 발견하여 승인 시 자원 예약·추가 할당 없는 cleanup·gate 밖 callback·비동기 Stop을 명시하였다. 4차에서 세 담당자가 기존 및 추가 반례의 해결을 확인하였다. 총 24개 보완 사항이며 이번 검토 범위의 미해결 지적은 없다. 전체 구현의 무결함 증명으로 해석하지 않는다.

모든 상태는 문서 계약 반영·구현 검증 대기다. 이번 배포에서 실제 수행한 검사는 문서의 구조·배치·참조·작업/파일 데이터와 C++ 예제의 clang-format 서식 검사다. 엔진 unit/stress/sanitizer·실제 장치·누수·영상·성능 시험은 수행하지 않았다. '사람이 알아보기 좋은 코드 스타일 유지' 원칙과 스페이스 4칸·탭 금지·빈 줄·버전 고정 formatter를 3.4절 및 상세안 4.3절에 명시하였다.

34 참고 자료와 설계 근거

이전 대화는 사용자 제공 참조 「3D 엔진 설계」와 현재 대화의 추가 요구를 기준으로 한다. 아래 공식 문서는 기술 개념과 API 경계를 확인하기 위한 자료다. 일정·API·예산·구현 단계는 이 계획의 제안이며 참고 자료의 성능 보증이 아니다. 온라인 문서는 변경될 수 있으므로 실제 구현 착수 시 선택 SDK 버전의 문서·commit을 build manifest에 고정한다. 조회일은 2026.11.10이다.

ID	1차 자료	적용 범위
S1	Epic · Incremental GC	점진 도달성·write barrier
S2	Clang · LibTooling	AST 기반 host tool
S3	Clang · JSON Compilation Database	실제 compile option 재현
S4	Microsoft · Enhanced Barriers	access·layout·동기화 구분
S5	meshoptimizer · 공식 저장소	mesh cook·meshlet 후보
S6	Microsoft · Indirect Drawing	GPU command buffer·ExecuteIndirect
S7	SIGGRAPH 2021 · Nanite 발표	가상화 geometry 공개 발표
S8	Epic · Lumen Technical Details	trace·surface cache 개념

ID	1차 자료	적용 범위
S9	ACL · 공식 저장소	animation compression 후보
S10	Jolt · 공식 저장소	physics integration 후보
S11	miniaudio · Manual	audio backend 후보
S12	Recast/Detour · 공식 저장소	tiled nav build·query
S13	Microsoft · WebView2 Security	host message·navigation 경계
S14	Epic · Gameplay Ability System	ability·attribute·effect
S15	Epic · Asset Management	primary asset·bundle·audit
S16	Epic · Game Features	독립 feature lifecycle
S17	Epic · Cinematics and Sequencer	컷씬 timeline·camera·shot·track
S18	Clang · ThreadSanitizer	data race 검사 지원 범위
S19	AMD · FSR Upscaler Integration	motion·jitter·mask·resource 입력
S20	NVIDIA · Streamline DLSS Guide	SR adapter·입력 convention
S21	Intel · XeSS-SR Guide	jitter·motion·dynamic resolution
S22	Epic · Post Process Effects	volume·effect 기능 목록
S23	Epic · Post Process Materials	custom effect·삽입 위치
S24	Epic · Cinematic Depth of Field	camera·bokeh·CoC
S25	Epic · HDR Display Output	HDR·색 변환·display profile
S26	Epic · Unreal Object Handling	기본값·참조·객체 처리
S27	Epic · Gameplay Framework	권한·scope·controller/state
S28	Epic · Object Replication	component·subobject 복제
S29	Epic · Multiplayer Travel	whole-world travel
S30	Epic · Virtual Assets	source metadata/bulk 분리
S31	Epic · Landscape Edit Layers	비파괴 terrain 편집
S32	Epic · Shading Models	고급 surface 모델
S33	Epic · Substrate Overview	layered material 연구
S34	Epic · Cinematic Render Passes	AOV·EXR·출력 한계
S35	Epic · Path Tracer	reference/offline 렌더링
S36	Epic · Online Services	온라인 provider 경계
S37	Epic · UI Invalidation	UI cache·retainer
S38	Epic · Motion Warping	root motion 목표 정합
S39	Epic · Modeling Tools	geometry 제작
S40	Epic · UnrealBuildTool	target/module 빌드 관리
S41	Epic · Build Configuration	PCH/Unity·executor 개념
S42	Incredibuild · XML Interface	custom action 분산
S43	FASTBuild · Caching	선택 cache/executor
S44	Mozilla · sccache	compiler cache
S45	Mozilla · sccache Distribution	지원 플랫폼별 분산
S46	LLVM · LLD	fresh link backend
S47	Clang · ThinLTO	선택 LTO/cache

ID	1차 자료	적용 범위
S48	Microsoft · Incremental Link	MSVC .ilk·비호환 옵션
S49	Ninja · Manual	local plan·dependency/pool
S50	Epic · UMG Editor Reference	UI Designer
S51	Epic · UMG Viewmodel	typed push binding
S52	C++23 N4950 · basic.types (초안 렌더링)	byte copy·객체 수명 구분
S53	Microsoft · D3D12 CPU Efficiency	command recording 자원 수명
S54	ccache · Direct Mode	negative include·cache 반례
S55	Detour · NavMeshQuery 정의	query context·node pool
S56	Detour · NavMesh 구현	tile 연결·인접 링크 변경
S57	Epic · Visibility and Occlusion Culling	보수적 가시성·시간차
S58	Epic · Rectangular Area Lights	면광원 제작 범위
S59	Epic · IES Light Profiles	배광 제작 범위
S60	Epic · Material Editor Guide	graph 기반 재질 제작 과정
S61	Epic · Material Editor UI	preview·live update·shader code·platform stats
S62	Epic · Material Instances	parent·parameter override·instance 제작
S63	Epic · Material Layers	layer stack·blend·instance 편집
S64	Epic · Custom Templates	제작 템플릿
S65	Epic · Texture Streaming	mip 요구량·예산·resident
S66	Epic · Trace	계측·분석 구조
S67	Epic · Memory Insights	할당·수명 진단
S68	C++23 · Language linkage	C export와 namespace
S69	Microsoft · Shader targets	D3D11 shader model
S70	Microsoft · DXC	DXIL·SPIR-V toolchain
S71	Khronos · SPIRV-Cross	MSL·GLSL 변환 후보
S72	Apple · Shader precompile	Metal offline library
S73	Microsoft · D3D11 devices	context thread 계약
S74	Unity · APV runtime lighting	baked scenario·runtime 변화 구분

표 58

EOD